

INFORMATIQUE EN CLASSE DE 1ÈRE C-D

NOUVEAU PROGRAMME

100%
GRATUIT

Ne peut être vendu!



Par le groupe WhatsApp
Les grandprofs  Informatique

Par des enseignants

 **BOUKAR**
 **KAMGA** Serges
 **KENGNE** Patrick B.
 **MAMNO** Marie T.
 **NGAFOE** Thierry
 **NGAKO** Armelle
 **NGOUABA** Paul
 **NUINKEU** Jean Jules
 **NKONDA** Steve F.
 **NTAKENDO** Emmanuel

 **OBAMA** Christian
 **OUAFO** Peguy
 **PELEU** Anicet
 **SIMO** Sandrine
 **SOPTSI** Voltaire
 **TAKAM** Abdel
 **TCHONGOUANG** Rostand
 **TOUZA** Isaac
 **YAYA** Ibrahima



NOUVEAU PROGRAMME

cours et exercices selon le nouveau programme en vigueur



COURS

Cours détaillés et illustrés selon l'Approche Par Compétence (APC)



EXERCICES

Des exercices de savoir, savoir-faire et savoir-être après chaque leçon

Ce livre est sous licence libre dont vous êtes libres de le modifier et de le diffuser !

Édition : Version 1.0 Août 2019

Infographie et couverture : NTAKENDO Emmanuel

Pour toutes suggestions ou préoccupations écrire à :

Le responsable des ateliers : (+237) 699 38 62 59

La communicatrice : (+237) 699 32 26 76

Avant-propos

La collection des livres du groupe WhatsApp « Les grandprofs^{Info} » a été conçue pour vous permettre d'entrer de manière efficace et efficiente dans les APC dans les classes de secondes et de premières ESG et EST, adaptés aux nouveaux programmes informatiques, ces ouvrages vous offrent entre autres :

-  **Des cours détaillés** (chaque module contient des unités d'apprentissage, et dans chaque unité d'apprentissage, des unités d'enseignement)
-  **Des situations problèmes contextualisées** qui permettront à l'apprenant d'exécuter des tâches déterminées à travers des consignes explicites dont l'issue est la construction de ses propres compétences
-  **Des activités d'intégration** : qui permettront à l'apprenant de réinvestir ses acquis dans un module donné
-  **Des exercices d'application** : à la fin de chaque unité d'enseignement il est question de vérifier la compétence à travers des petits exercices

Ce travail a été fait par un groupe d'enseignants. Le déroulement (l'organisation) des travaux a eu lieu dans un groupe WhatsApp (« Les grandprofs^{Info} »), et dans ce grand groupe sont nés d'autres petits groupes ou ateliers de travail, ainsi dans chaque atelier, les membres formaient des duos pour produire des résultats des objectifs fixés au préalable. À noter qu'il n'y a pas eu de rencontres physiques entre les membres.

Les contenus de ces livres répondent donc aux objectifs tant du côté enseignant que du côté apprenant, c'est une véritable référence exploitable à toutes fins utiles

Nous sommes ouverts à toutes les suggestions, remarques et même critiques constructives afin d'améliorer nos œuvres. Ne dit-on pas souvent que « la perfection n'est pas de ce monde ? »

La communicatrice du groupe Mme Mbassi Né Mendouga Philomène

SOMMAIRE

Avant-propos	3
MODULE 1 : ENVIRONNEMENT NUMÉRIQUE, SÉCURITÉ, MULTIMÉDIA ET GESTION DES DONNÉES	
U.A 1 : INSTALLATION D'UN SYSTÈME D'EXPLOITATION	
U.E 1 : Installation d'un SE	7-16
U.E 2 : Les opérations post-installation d'un SE	17-30
U.A 2 : UTILISER L'INVITE DE COMMANDES	
U.E 1 : Introduction à l'invite de commandes	32-34
U.E 2 : Utilisation de l'invite de commandes	35-39
U.A 3 : APPLIQUER LES CONCEPTS FONDAMENTAUX DE LA SÉCURITÉ INFORMATIQUE	
U.E 1 Introduction à la sécurité informatique	41-47
U.A 4 : UTILISER LES FICHIERS MULTIMÉDIAS	
U.E 1 : Utiliser les fichiers multimédias	49-54
U.A 5 : DÉCRIRE LES CONCEPTS DE BASE DES SYSTÈMES D'INFORMATION	
U.E 1 : Introduction aux systèmes d'information	56-62
U.E 2 : Méthodes d'analyse et de conception des systèmes d'information	63-65
U.A 6 : DÉCRIRE LES CONCEPTS DES BASES DE DONNÉES	
U.E 1 : Généralités sur les bases de données	67-69
U.E 2 : Les caractéristiques d'une table	70-72
U.E 3 : Les fonctions d'une base de données	73-76
MODULE 2 : ALGORITHME ET PROGRAMMATION	
U.A 7 : UTILISER LES STRUCTURES ALGORITHMIQUES	
U.E 1 : Les structures itératives	78-81
U.E 2 : Les structures conditionnelles	82-83
U.E 3 : Généralités sur les structures de données	84-86
U.E 4 : Les tableaux	87-92
U.A 8 : UTILISER LES FONCTIONS ET PROCÉDURES	
U.E 1 : Notions de procédure et fonction	94-98
U.A 9 : PROGRAMMER EN HTML	
U.E 1 : Insérer les fichiers multimédias dans une page web	100-101
U.E 2 : Insérer des tableaux et formulaires en HTML	102-106
U.E 3 : Organiser les dossiers du site et mise en place du CSS	107-110

U.A 10 : PROGRAMMER EN JAVASCRIPT

U.E 1 : Introduction au JavaScript	I12- I14
U.E 2 : Notions de base du JavaScript	I15- I21
U.E 3 : Structures de contrôles, tableaux et fonctions en JavaScript	I22- I32
U.E 4 : Les événements en JavaScript	I33- I36

U.A 11 : PROGRAMMER EN C

U.E 1 : Introduction au langage C	I38- I46
U.E 2 : Les structures de contrôle en C	I47- I54
U.E 3 : Les tableaux en C	I55- I58
U.E 4 : Les sous-programmes en C	I59- I63
U.E 5 : Exécution d'un programme C	I64- I68



INSTALLATION D'UN SYSTÈME D'EXPLOITATION

I
N
S
T
A
L
L
E
R

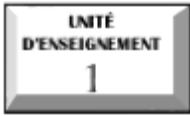
U
N
S
E

 U.E 1 : Installation d'un SE

 U.E 2 : Les opérations post-installation d'un SE

Situation problème :

L'ordinateur de votre papa utilise comme système d'exploitation Windows XP. Il voudrait désormais travailler sous Windows 7. Il fait appel à vous pour changer son système d'exploitation mais voudrait qu'à la fin tout puisse fonctionner correctement comme sous Windows XP. Comment allez vous procéder ? Quelles sont les opérations que vous allez effectuer ?



Installation d'un SE

Compétences :

- ☑ Configurer l'ordre de bootage des supports
- ☑ Partitionner le disque dur lors de l'installation d'un SE
- ☑ Choisir la partition d'installation d'un SE
- ☑ Configurer les comptes utilisateurs au cours de l'installation d'un SE

I. CONFIGURER L'ORDRE DE BOOT (DÉMARRAGE) DES SUPPORTS

A. Changer l'ordre de démarrage dans les paramètres du BIOS

Pour entrer dans le BIOS, allumez votre ordinateur. Dès le démarrage de votre système d'exploitation, maintenez enfoncée la touche « F2 ». Si vous possédez un PC de bureau et que cette manipulation ne vous a pas permis d'accéder au BIOS, nous vous invitons à éteindre votre ordinateur et à l'allumer en tapotant sur la touche « Suppr ».

2. Appuyez la touche directionnelle FLÈCHE DROITE pour sélectionner Boot (Démarrage).
3. Appuyez la touche directionnelle FLÈCHE BAS pour sélectionner Boot Order (Ordre de démarrage).
4. Utilisez la touche F5 ou F6 pour changer l'ordre.
5. Appuyer sur la touche F10 pour enregistrer les changements et redémarrer l'ordinateur.

B. Changer l'ordre de démarrage dans le menu de démarrage

1. Vérifiez que la fonction de Menu de démarrage F12 est activée sur votre système: veuillez consulter l'article de notre base de connaissances Activer le menu du périphérique de démarrage.

2. Mettre sous tension votre ordinateur. Lorsque le premier écran de logo apparaît, appuyer plusieurs fois la touche F12 pour accéder au menu de démarrage.

3. Utilisez la touche directionnelle FLÈCHE HAUT ou BAS pour sélectionner le lecteur à partir duquel effectuer le chargement du système d'exploitation, et ensuite appuyez sur la touche ENTRÉE pour confirmer.

II. PARTITIONNER LE DISQUE DUR LORS DE L'INSTALLATION D'UN SE ;

Afin d'installer un nouveau système, il faut formater votre disque dur ou votre partition "C:". A l'aide du DVD Windows 7, vous pouvez formater, créer ou supprimer une ou plusieurs partitions avant l'installation. Nous vous recommandons de créer au moins deux partitions. Si vous n'avez pas de DVD d'installation il faudra alors en créer un.

La première partition C ou partition principale: (disque local C:) ou sera installé le système et une deuxième partition D: ou E: ou secondaire pour sauvegarder vos documents (images, musique et vidéos). Dans le cas d'un crash du PC, vous pourrez réinstaller facilement Windows 7 sans effacer vos documents.

Avant de commencer l'opération, sauvegardez vos fichiers. Sauvegardez vos documents sur un disque dur externe, un DVD ou un CD, un lecteur flash USB ou dans un dossier réseau. Sauvegardez également votre compte utilisateur et bien-sûr assurez-vous que votre ordinateur présente la configuration requise pour faire tourner Windows 7

I. Préparez le disque d'installation de Windows 7

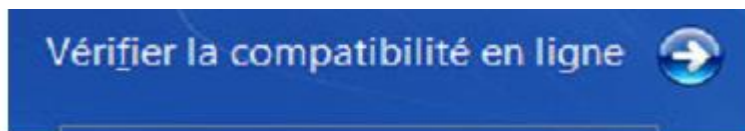
Avant toute chose, commencez par repérer votre clé de produit Windows de 25 caractères. Vous la trouverez sur une étiquette collée sur l'unité centrale du PC ou sur le boîtier du CD d'installation Windows. Branchez votre PC sur votre modem ou routeur (Box) à l'aide du câble Ethernet afin que Windows recherche pendant l'installation, les mises à jour disponibles pour Windows 7 et les pilotes périphériques. Une installation sans mise à jour ne devrait pas poser de problème dans la mesure où vous débranchez tous les périphériques externes avant l'installation. Vous devrez faire les mises à jour après l'installation Windows 7 via le Menu démarrer / Tous les programmes / Windows Update.

Choisissez entre l'installation de l'édition 32 bits ou 64 bits de Windows 7. Pour pouvoir exécuter la version 64 bits, votre ordinateur doit être équipé d'un processeur 64 bits.

2. Insérez le DVD d'installation Windows 7 dans votre lecteur et redémarrez le PC.

À la demande, appuyez sur une touche du clavier pour démarrer à partir du DVD. Si la page "Installer Windows" ne s'affiche pas ou que vous n'êtes pas invité à appuyer sur une touche pour démarrer à partir du DVD, il peut être nécessaire de sélectionner votre lecteur/graveur DVD au démarrage dans le Bios.

3. Une fois le DVD lancé, choisissez le langage et cliquez sur "Installer maintenant"



4. Télécharger les mises à jour ou pas

Obtenir les mises à jour importantes pour l'installation



Télécharger les dernières mises à jour pour l'installation (recommandé)

Procurez-vous les mises à jour les plus récentes, telles que les mises à jour de sécurité et les pilotes matériel, pour optimiser l'installation de Windows. L'ordinateur restera connecté à Internet tout au long de l'installation.



Ne pas obtenir les dernières mises à jour pour l'installation

Sans ces mises à jour, l'installation peut échouer et votre ordinateur risque d'être plus vulnérable aux menaces de sécurité.

5. Acceptez les termes du contrat Windows. Choisissez Installation personnalisée (options avancées).

Quel type d'installation voulez-vous effectuer ?



Mise à niveau

Procédez à la mise à niveau vers une version plus récente de Windows et conservez vos fichiers, paramètres et programmes. L'option de mise à niveau n'est disponible que lorsqu'une version existante de Windows est en cours d'exécution. Sauvegardez vos fichiers avant de poursuivre.



Personnalisée (option avancée)

Installez une nouvelle copie de Windows. Cette option ne conserve pas vos fichiers, paramètres et programmes. L'option permettant de modifier les disques et les partitions est disponible quand vous démarrez votre ordinateur à l'aide du CD d'installation. Sauvegardez vos fichiers avant de poursuivre.

6. Où installer Windows 7 sur un disque déjà partitionné?

Si vous avez un disque dur neuf ou formaté complètement sans partition, passez directement à l'étape suivante. Si votre disque dur est déjà partitionné, il vous faudra sélectionner l'espace disque où Windows 7 sera installé. Cliquez sur Options de lecteurs (avancées). (Voir l'image ci-dessous).

Où souhaitez-vous installer Windows ?

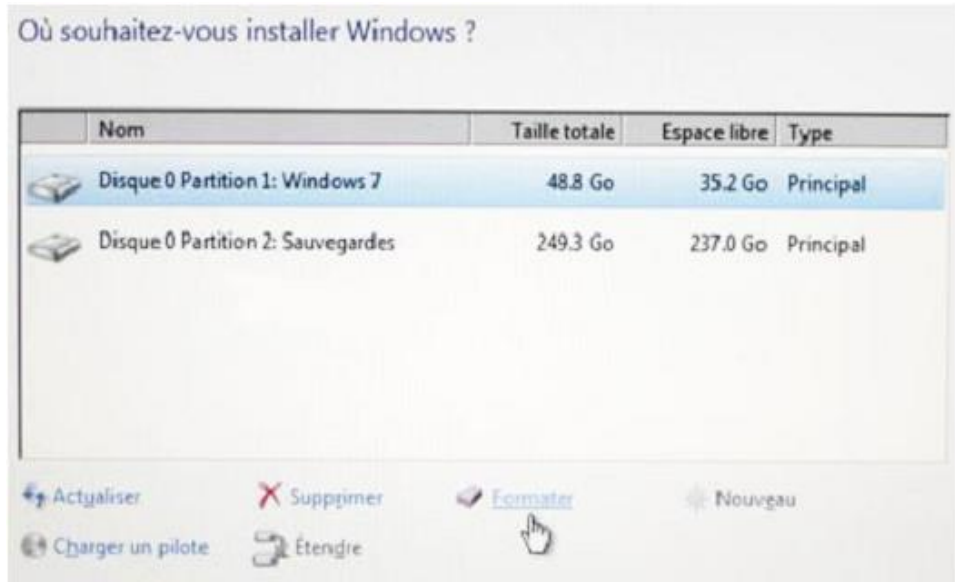
Nom	Taille totale	Espace libre	Type
 Disque 0 Partition 1: Windows 7	48.8 Go	35.2 Go	Principal
 Disque 0 Partition 2: Sauvegardes	249.3 Go	237.0 Go	Principal

 Actualiser
 Charger un pilote

[Options de lecteurs \(avancées\)](#)

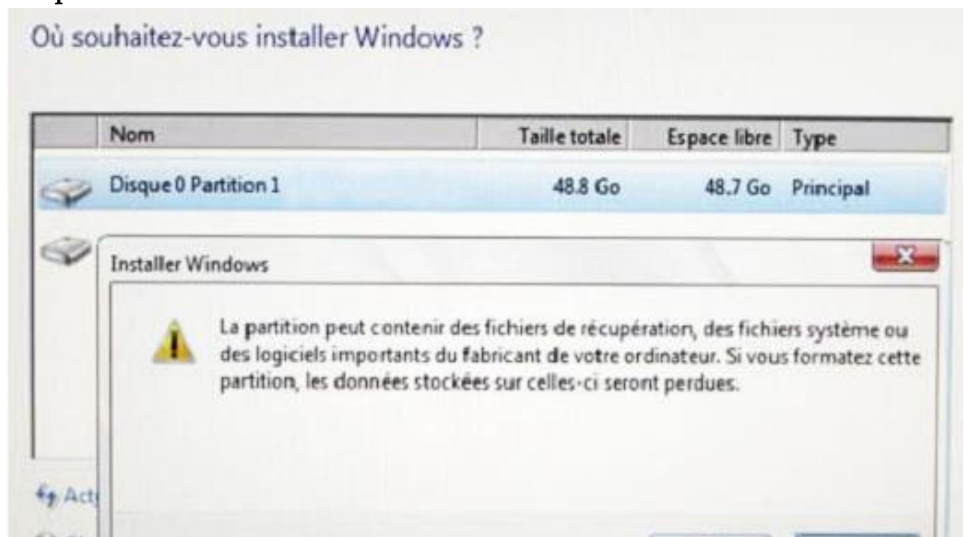
Pour ne pas réinstaller Windows 7 sur un système déjà installé, formatez la partition Disque 0 Partition 1 (Voir l'image ci-dessous). Cela évite de se retrouver avec un

dossier Windows.old (dossier de sauvegarde de l'ancien système) sur la partition C:



Toutes les données sur cette partition seront effacées.

Cliquez sur "OK"



Sélectionnez Disque 0 Partition 1 et cliquez sur l'onglet suivant pour installer Windows 7 maintenant.

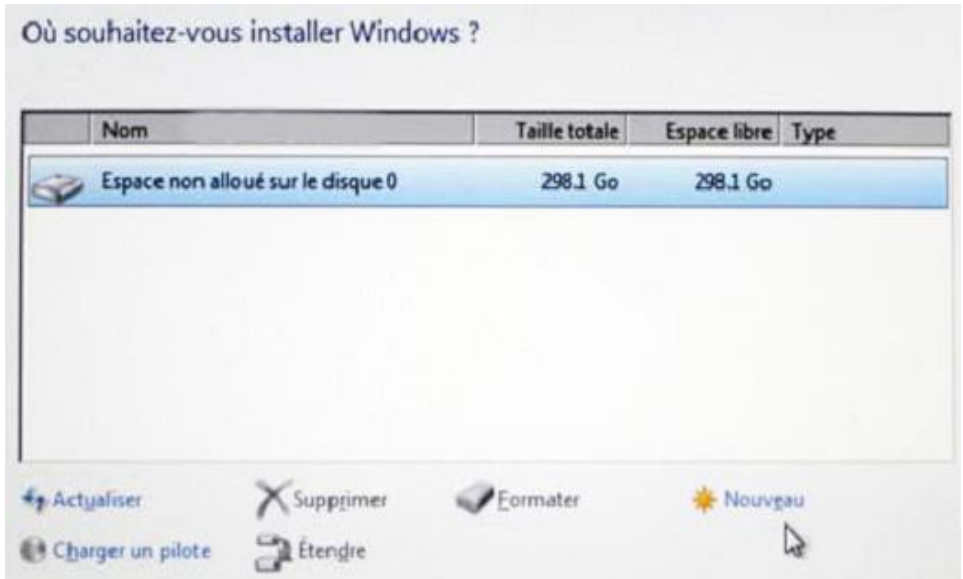
Si votre disque dur est neuf ou déjà formaté, nous vous recommandons de créer au moins deux partitions. La première partition Disque 0 Partition 1 ou sera installé le système Windows 7 et une deuxième partition Disque 0 Partition 2 pour sauvegarder vos documents (images, musique et vidéos). Dans le cas d'un crash Windows, vous pourrez réinstaller Windows 7 sans effacer vos documents et fichiers personnels.

Si vous ne créez pas de partition, sélectionnez Disque 0 Partition 1 et cliquez sur l'onglet suivant pour installer Windows 7 maintenant.

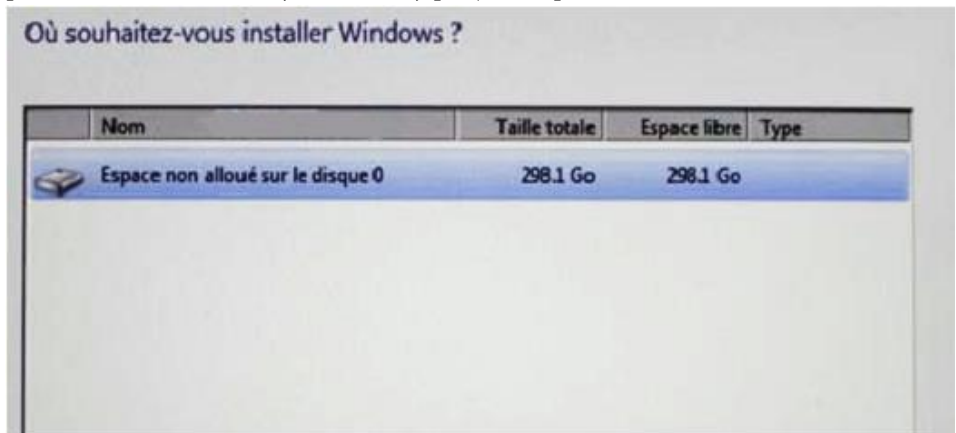
7. Création d'une ou plusieurs partitions. Cliquez sur Options de lecteurs (avancées). (Voir l'image ci-dessous).



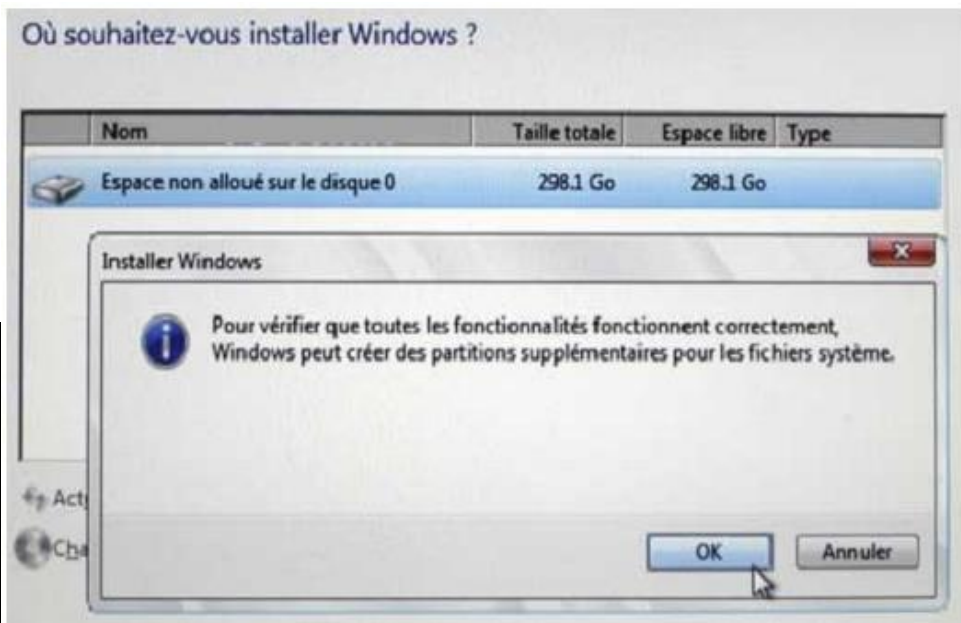
8. Cliquez sur "Supprimer" pour pouvoir modifier la partition principale sur le disque 0 (Voir l'image ci-dessous).



Tapez la taille de la partition sur laquelle Windows 7 sera installée et cliquez sur l'onglet "Appliquer". (Voir l'image ci-dessous). Pour une installation standard, 60Go (60000 Mo) sera suffisant. Si vous installez des Jeux, il faudra prévoir environ 20 Go (20000 Mo) par jeu en plus.



Cliquez sur "OK"



Contrairement à Vista, Windows 7 va créer automatiquement une partition supplémentaire de 100 Mo. Cette partition "System Reserved" peut être utilisée dans le cas d'un problème de démarrage Windows afin de réparer le système.

Remarque: Vous pouvez annuler cette option, si vous possédez déjà 3 partitions principales. Vous pourrez toujours réparer le système à partir du DVD d'Install. Sélectionnez "Espace non alloué sur le disque 0", cliquez sur "Nouveau" tapez une taille inférieure pour créer une autre partition ou appliquez la taille d'origine pour terminer. (Voir l'image ci-dessous).

9. Sélectionnez Disque 0 Partition 2 pour installer Windows 7, cliquez sur "Suivant"



Windows 7 va copier les fichiers nécessaires à l'installation et va redémarrer le PC.

N'appuyez plus sur une touche du clavier pour démarrer à partir du DVD.

Le programme de mises à jour paramètre les registres, après un 2ème redémarrage, l'installation est en cours.. Patientez 15 à 20 mns. Windows 7 démarre, entrez un nom d'utilisateur et un nom pour votre ordinateur et cliquez sur "suivant".



Tapez la clé du produit, 5 x 5 caractères, et cliquez sur “ suivant”

Entrez votre clé de produit

Vous trouverez la clé de produit Windows 7 Édition Intégrale sur le boîtier du disque d'installation situé à l'intérieur du package Windows. L'activation enregistrera la clé de produit sur cet ordinateur.

La clé de produit ressemble à celle-ci :

CLÉ DE PRODUIT : XXXXX-XXXXX-XXXXX-XXXXX-XXXXX

[Où trouver ma clé de produit Windows ?](#)

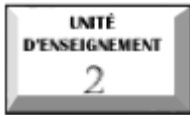
Clé du produit :

L'installation est terminée, vous êtes enfin sur le bureau Windows 7. Retirez votre DVD d'installation Windows 7

EXERCICES

Exercice 1

Exercice 2



Les opérations post-installation d'un SE

Compétences :

- ☑ Choisir le mode d'installation du SE
- ☑ Activer un système d'exploitation
- ☑ Créer un point de restauration
- ☑ Restaurer un SE
- ☑ Gérer les pilotes

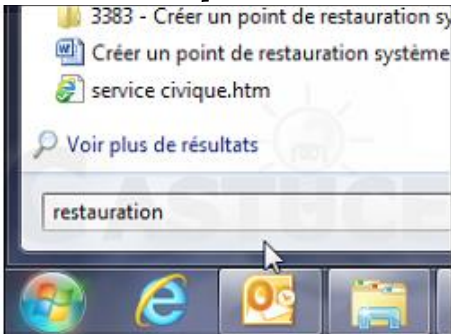
I. CRÉER UN POINT DE RESTAURATION ;

Avec l'installation d'un logiciel ou une modification importante de votre système, nous vous conseillons de créer manuellement un point de restauration. Ainsi, en cas de problème, vous pourrez revenir à l'état précédent fonctionnel.

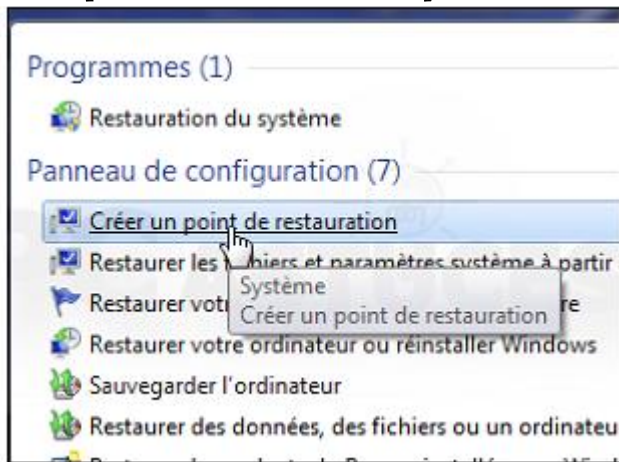
1. Cliquez sur le bouton Démarrer.



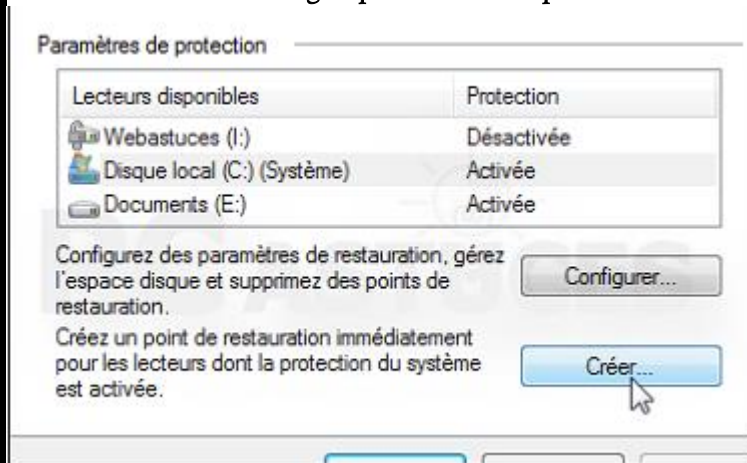
2. Dans le champ Rechercher, saisissez la commande restauration.



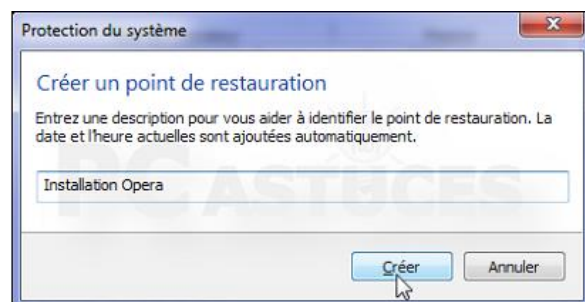
3. Cliquez sur la fonction Créer un point de restauration qui apparaît.



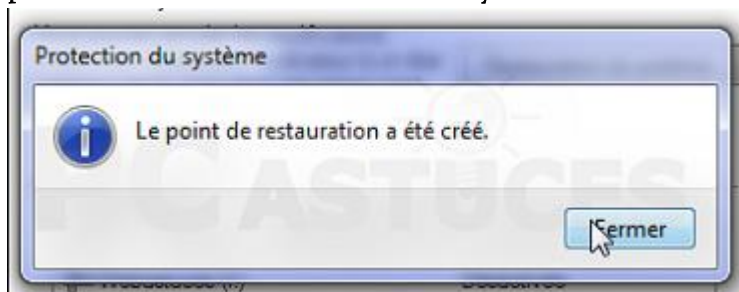
4. Dans la boîte de dialogue qui s'affiche, cliquez sur le bouton Créer.



5. Saisissez une description pour mieux identifier un point de restauration et cliquer sur Créer



6. Cliquez enfin sur **Fermer**. En cas de soucis, vous pourrez facilement revenir à ce point de restauration et retrouver votre système en état.



II. RESTAURER UN SE ;

La restauration du système est utile en cas de dysfonctionnement de l'ordinateur. Elle permet de "restaurer" l'ordinateur à une configuration précédente au problème rencontré.

La restauration du système consiste à :

- créer automatiquement des points de restauration à chaque mise à jour du système (cela se fait par Windows, sans intervention de l'utilisateur)
- en cas de problème, "restaurer le système", c'est-à-dire revenir à une configuration précédente.

Les points de restauration sont des "clichés" du système. Cela consiste à sauvegarder les fichiers les plus importants du système

En cas de problème, la Restauration du système remplace les fichiers modifiés par les anciens, pour revenir à une configuration de l'ordinateur antérieure au problème et retrouver un fonctionnement normal de l'ordinateur.

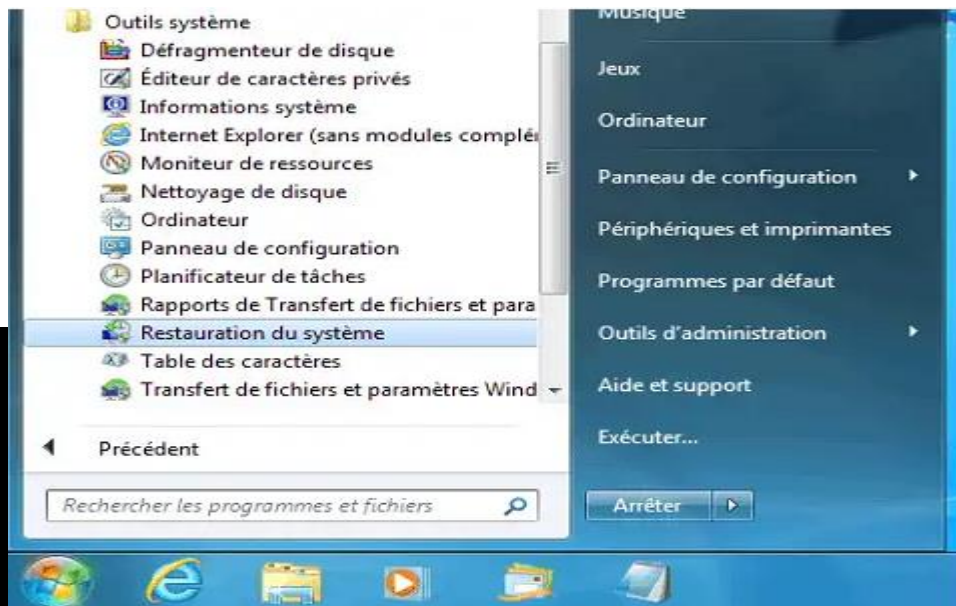
On peut faire la restauration en passant :

I - Par le menu Démarrer :

Situation normale : l'ordinateur est en état de fonctionner. Mais il y a des dysfonctionnements.

On accède à la Restauration du système par le menu Démarrer

- **menu Démarrer**, Tous les Programmes, Accessoires, Outils système, Restauration du système.

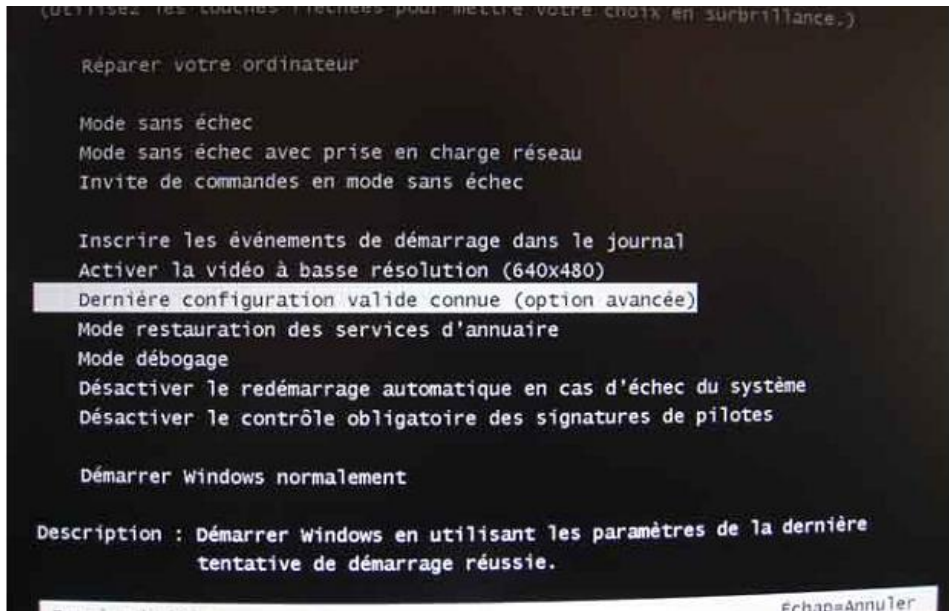


2 – par le mode sans échec : L'ordinateur ne démarre pas. Impossible de lancer Windows.

Lorsque Windows ne peut démarrer, on peut tenter d'utiliser la Restauration du système au démarrage de l'ordinateur.

Au démarrage de l'ordinateur, après le premier écran du Bios :

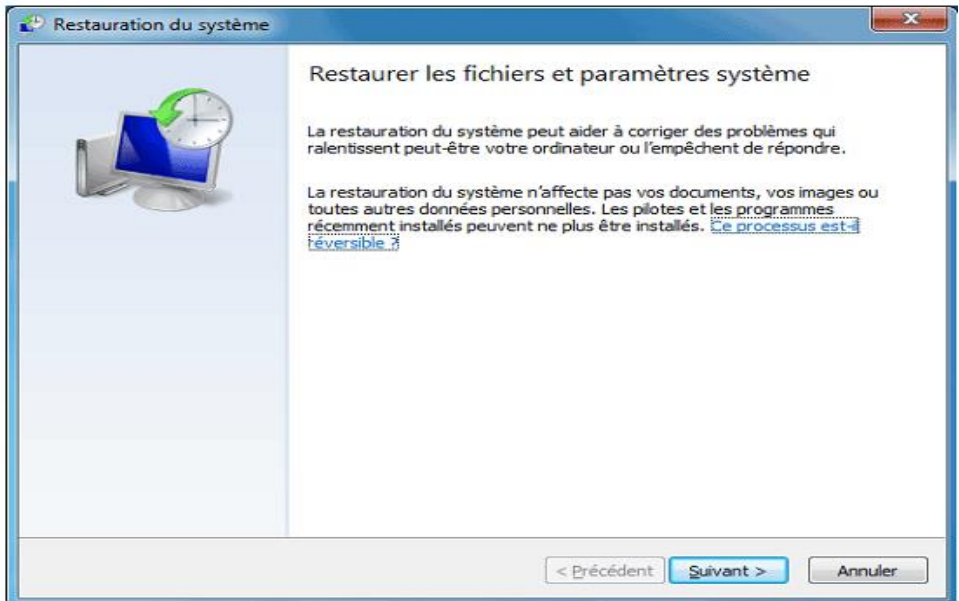
- "tapoter" sur la touche F8 jusqu'à apparition du menu de démarrage
- Dans le menu de démarrage :
- sélectionner "Dernière bonne configuration connue" avec les touches fléchées (le nom peut être : "Dernière configuration valide connue")
 - appuyer sur la touche Entrée pour valider le choix.



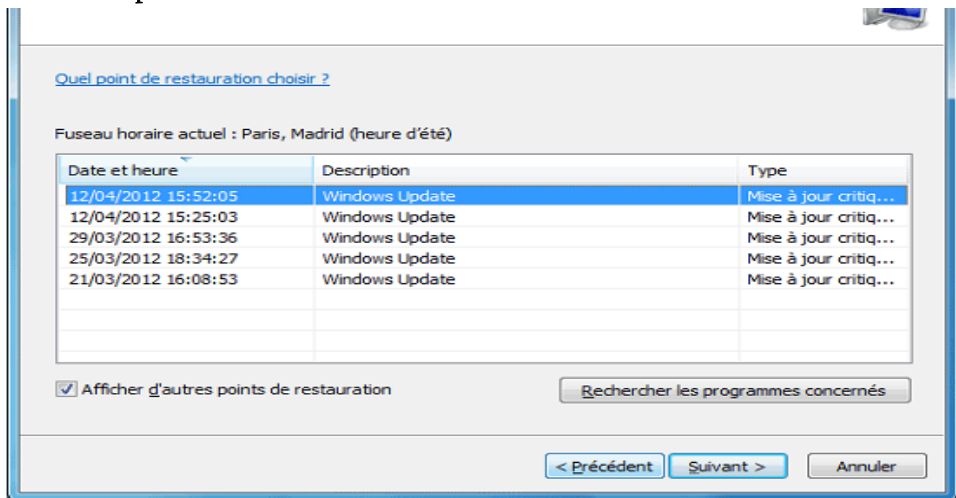
Utilisation de la Restauration du système :

Lorsqu'on accède à la Restauration du système par le menu Démarrer ou par le mode sans échec.

Au premier écran :
- cliquer sur Suivant



Choisir un point de restauration :



- sélectionner un point de restauration,
- cliquer sur Suivant.

Normalement, on pourra essayer successivement plusieurs points de restauration.

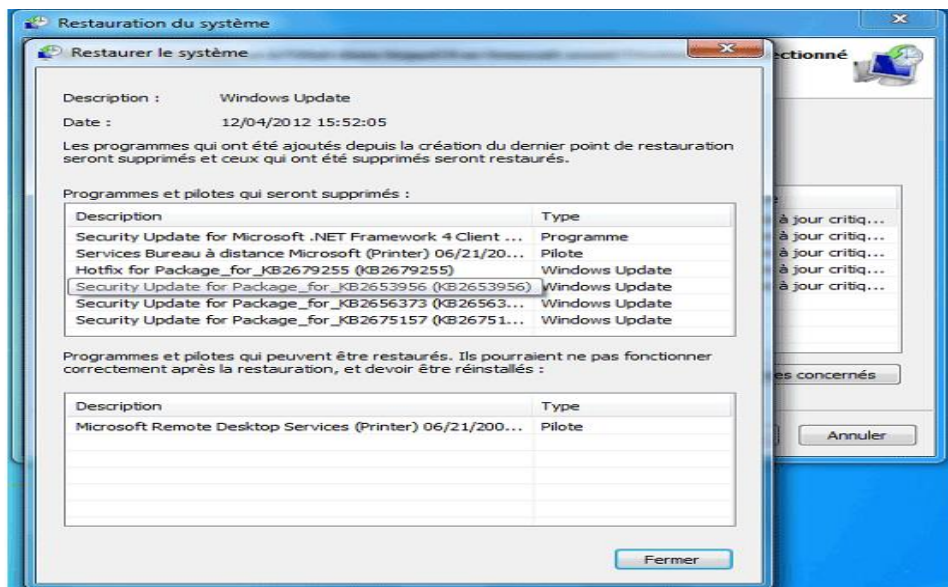
Même si la Restauration du système n'affecte que les fichiers système :

Au préalable, effectuer une sauvegarde de ses données importantes.

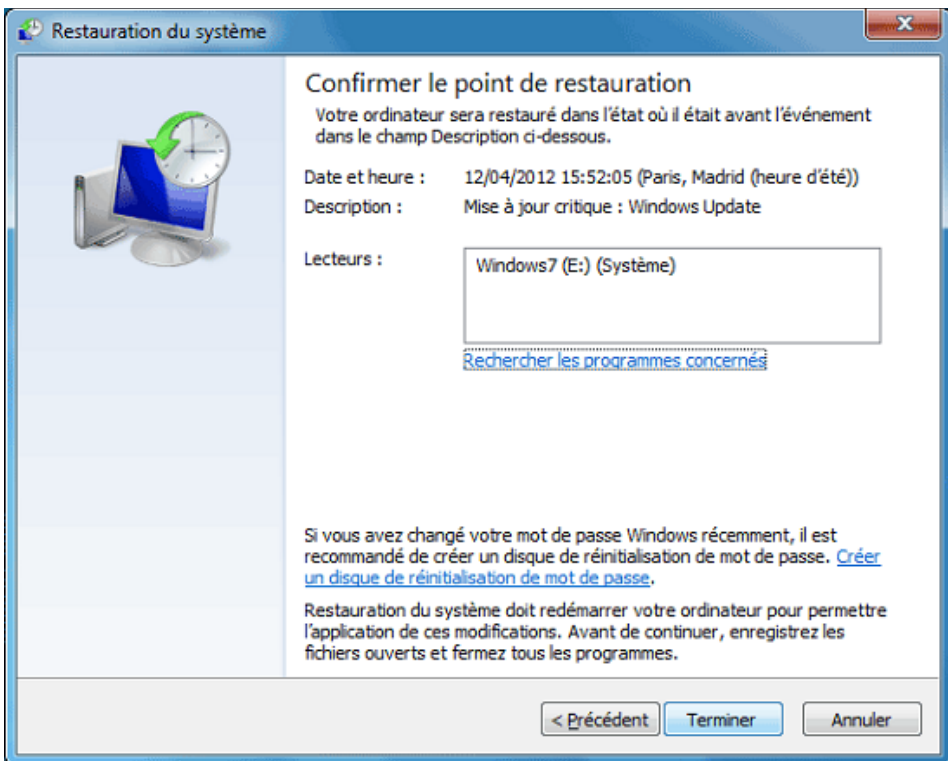
La Restauration du système permet de supprimer proprement tout programme malencontreusement installé.

Si on veut connaître l'incidence de la Restauration du système sur les programmes (facultatif)

- cliquer sur "Rechercher les programmes concernés"



- cliquer sur "Terminer" pour lancer la Restauration du système



ATTENTION : Dès que l'on clique sur Terminer, l'ordinateur s'éteint et redémarre. Sauvegarder, au préalable, tout document en cours. Et fermer tous les programmes et fenêtres ouvertes.

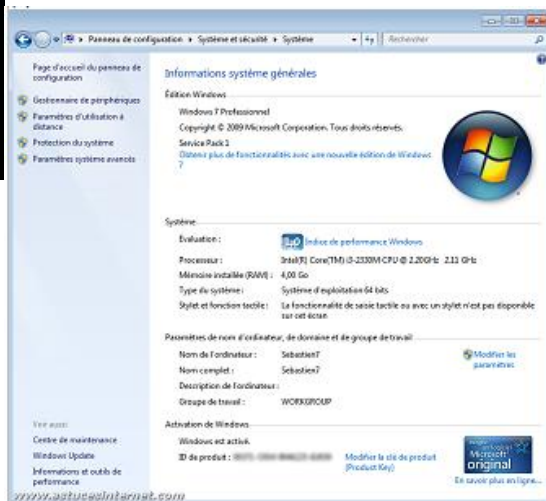
III. ACTIVER UN SYSTÈME D'EXPLOITATION;

Le module Système donne un aperçu des informations matérielles de votre ordinateur. Ainsi, vous aurez accès à la page Informations système générales qui affichera certaines informations matérielles de base (nom de l'ordinateur, processeur, quantité de RAM installée, etc.). Aperçu :



La page Informations système générales affiche certaines informations de votre ordinateur, déclinées en quatre catégories :

- Edition Windows ;
- Système ;
- Paramètres de nom d'ordinateur, de domaine et de groupe de travail ;
- Activation de Windows.



Edition Windows : Affiche la version Windows en cours d'utilisation. Dans notre exemple, la version utilisée est Windows 7 Professionnel, avec le service pack 1 installé.

Le lien **Obtenir plus de fonctionnalités avec une nouvelle édition de Windows 7** va vous rediriger vers une fenêtre Mise à niveau express vous permettant :

- De vous connecter au site de Microsoft afin d'acheter une version de Windows ayant davantage de fonctionnalités;
- D'entrer une nouvelle clé Windows si vous avez déjà acheté une nouvelle version.

Aperçu :

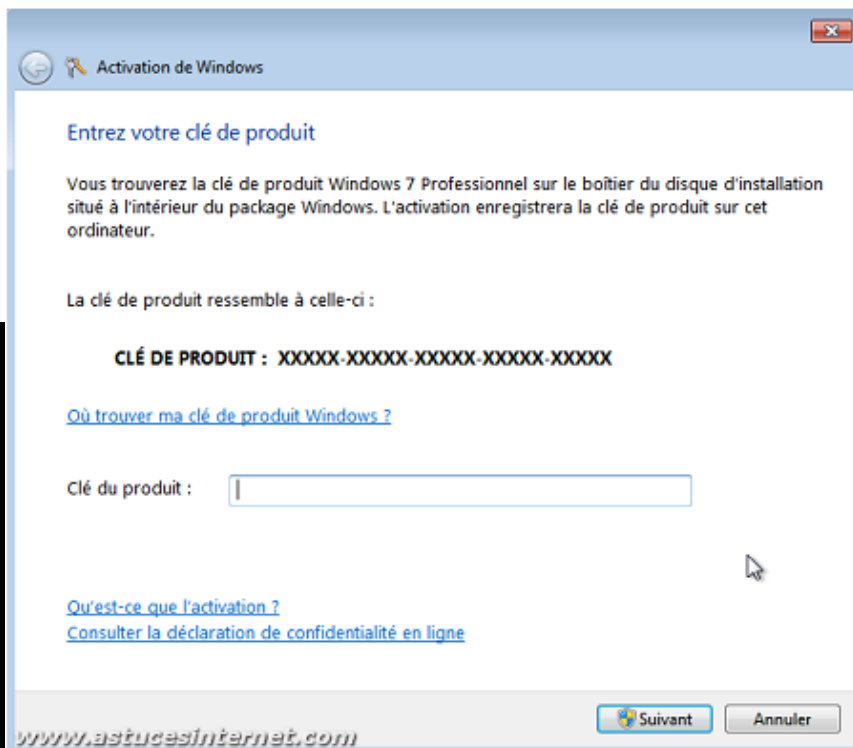
Système : Affiche l'indice de performance Windows, le processeur et la quantité de RAM installée, le type de système d'exploitation (32 ou 64bits) et si le système dispose d'une interface tactile.

Le lien **Indice de performance Windows** redirige vers la page Informations et outils de performance. Rendez-vous sur la page suivante pour obtenir des informations sur le module Informations et outils de performance : Informations et outils de performance

Paramètres de nom d'ordinateur, de domaine et de groupe de travail : Affiche le nom de votre ordinateur et les informations de groupe de travail ou de domaine. Vous pouvez modifier le nom de l'ordinateur ainsi que son groupe de travail en cliquant sur Modifier les paramètres.

Activation de Windows : Indique si votre copie de Windows est authentique et activée.

Vous avez la possibilité de saisir une clé produit en cliquant sur Modifier la clé de produit (



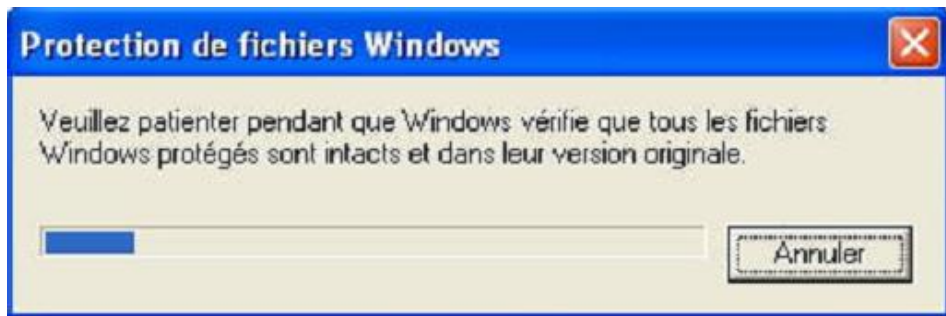
IV. CHOISIR LE MODE D'INSTALLATION DU SE (MISE À NIVEAU, NOUVELLE INSTALLATION, RÉPARATION)

Parfois, lors de l'installation d'un logiciel ou suite à un problème de disque dur, une partie d'un fichier système peut être endommagée. Le système ou un logiciel particulier peuvent s'en plaindre et refuser de démarrer.

Windows possède un utilitaire fort utile et pourtant bien peu connu : **le vérificateur de fichiers système** (SFC - System File Checker).

Pour l'utiliser :

- Cliquez sur Démarrer / Exécuter.
- Saisissez et validez la commande suivante en respectant l'espace entre la commande et son argument : `sfc /scannow`
- La boîte de dialogue Protection de fichiers Windows s'ouvre alors et l'analyse des fichiers système commence.



En cas de fichier corrompu, le support d'installation peut être demandé pour remplacer le fichier par sa version originale

Si la procédure ci-dessus ne permet pas de résoudre le problème, il reste la possibilité de réparer l'installation de Windows en se munissant du support d'installation.

Concernant Windows 10 / 8 / 7, il est possible de télécharger l'image ISO de Windows afin de créer une clé USB bootable. Attention: Cette réparation peut nécessiter la réactivation de Windows auprès de Microsoft dans certains cas. À noter: vous aurez besoin de clé produit de Windows pour finir la réparation du système. Elle se généralement trouve sur le côté de l'ordinateur.

Si vous ne la trouvez pas, l'utilitaire SIW pourra facilement retrouver ce numéro de série, dans Logiciels > Licences > Windows Product Key.

a. Démarrez votre PC avec le support d'installation de Windows :

- Insérez le support d'installation, et redémarrez votre PC. Si rien ne se passe, passez à l'autre étape.
 - Au démarrage de votre PC, essayez avec la touche qui affiche la liste des périphériques permettant de démarrer votre PC (F8 ou F12 en général). Puis choisissez le lecteur correspondant au support d'installation.
 - Si vous ne trouvez pas la touche, il faut alors passer à la configuration du BIOS :
 - Accéder au setup du Bios
 - Commandes habituelles du BIOS
 - Modifier l'ordre des périphériques de démarrage
- b. Appuyez sur une touche dès que le système vous y invite.
- c. Patientez pendant la détection du matériel.

- d. Quand vous aurez cet écran, appuyez sur ENTRÉE.
- e. Appuyez sur F8.
- f. L'installation va détecter automatiquement la présence d'une ou plusieurs versions de Windows. Sélectionnez celle que vous voulez réparer et appuyez sur R.
- g. La réparation devrait commencer.
- h. Ensuite, la procédure est exactement la même que celle de l'installation de Windows, mais à partir de l'étape redémarrage.
- i. Lorsque la réparation est terminée, retirez le support d'installation de Windows.
- j. Laissez l'ordinateur redémarrer normalement.
- k. Ne vous inquiétez pas. Vous ne perdez rien de votre Windows et le système devrait fonctionner à nouveau...

V. GÉRER (INSTALLER, METTRE À JOUR, ACTIVER) LES PILOTES

En informatique, **un pilote** (appelé driver en anglais) est un programme informatique qui permet faire fonctionner un périphérique (imprimante, clavier, souris, carte graphique, carte son etc...) avec le système d'exploitation de votre ordinateur. Il agit comme le lien, l'interface entre le matériel et le système d'exploitation.

En gros, sans pilote, le périphérique ne pourra pas remplir sa fonction car le système d'exploitation de l'ordinateur ne sera pas capable de donner des instructions au périphérique afin de l'utiliser.

Chaque matériel informatique connecté à un ordinateur utilise un pilote spécifique au système d'exploitation : un pilote spécifique à Windows 10 ne fonctionnera pas la plupart du temps avec une autre version de Windows (Windows 7, Windows 8, Windows Vista) et encore moins avec un Mac ou une machine sous Linux.

a. Où trouver le bon driver pour faire fonctionner mon matériel ?

Le plus souvent, le driver est fourni avec le matériel, souvent sous la forme d'un CD (et oui, votre lecteur de CD-ROM peut vous servir à autre chose que convertir vos CD en mp3). Mais avec la démocratisation d'internet, il arrive fréquemment de devoir aller chercher le pilote adéquat sur le site du constructeur

b. Installer un pilote sur Windows

Lors de l'ajout d'un nouvel périphérique, Windows va rechercher s'il possède un driver compatible avec votre matériel. Si c'est le cas, il va automatiquement installer le pilote sans intervention de votre part, votre périphérique sera utilisable immédiatement. C'est-ce qu'on appelle en anglais le Plug and Play qu'on pourrait traduire littéralement par Branchez et Jouez.

Il est aussi possible d'installer un driver directement depuis le système d'exploitation, sans passer par son navigateur internet.

Windows permet l'installation de drivers via son gestionnaire de périphériques qui liste tous les périphériques présents sur votre ordinateur.

Ouvrir le gestionnaire de périphérique en faisant un clic droit sur l'icône Windows en bas à gauche de votre écran.

Dans le gestionnaire de périphériques, le matériel qui ne possède pas de driver est signalé par un point d'exclamation sur fond jaune avec la désignation Périphérique inconnu.

Faire un clic droit sur le périphérique non reconnu puis cliquer sur Mettre à jour le pilote...

Cliquer sur Rechercher automatiquement un pilote mis à jour

Windows va rechercher sur votre ordinateur et sur internet un pilote pour faire fonctionner le matériel.

S'il trouve le driver, Windows l'installera automatiquement.

S'il ne le trouve pas, vous devrez aller chercher vous-même le pilote sur le site du constructeur de votre matériel et l'installer manuellement.

c. Mettre à jour les pilotes sous Windows

Utiliser un logiciel de drivers comme **Driver Booster** pour mettre à jour ses drivers. Il existe donc une solution qui convient bien aux utilisateurs les moins avancés ou à ceux qui n'ont pas envie de s'embêter avec les MAJ de pilotes : utiliser un logiciel qui détecte et met à jour pour vous vos pilotes comme le logiciel IObit Driver Booster.

EXERCICES

Exercice 1

Exercice 2

Exercice 3

Exercice 4

Exercice 5

I
N
S
T
A
L
L
E
R

U
N

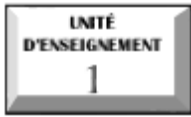
S
E



UTILISER L'INVITE DE COMMANDES

 U.E 1 : Introduction à l'invite de commandes

 U.E 2 : Utilisation de l'invite de commandes



Introduction à l'invite de commandes

Compétences visées :

- Donner l'importance de l'utilisation de la ligne de commande
- Accéder à la ligne de commande.

Situation problème :

I
N
V
I
T
E
D
E
C
O
M
M
A
N
D
E
S

I. IMPORTANCE DE L'UTILISATION DES LIGNES DE COMMANDE

Les ordinateurs ont longtemps été cantonnés à des interfaces en mode texte uniquement. Puis, la popularisation par Apple des premiers systèmes d'exploitation « graphiques », suivie par les débuts de Windows, ont amorcé la transition d'une ère du « tout clavier » à celle actuelle du « tout souris », où la plupart des opérations simples sont réalisables en quelques clics bien placés.

Indépendamment de ces avantages indéniables, la ligne de commande permet de réaliser simplement et rapidement des opérations qui seraient autrement très fastidieuses et répétitives, voire impossibles, comme :

- 📁 créer une sauvegarde de tous vos fichiers modifiés cette semaine ;
- 📁 extraire la première page d'une série de fichiers PDF pour en faire des images

📁 En cas de gros problèmes elles permettent parfois de se dépanner en lançant l'outil de restauration du système à partir d'un démarrage sans échec avec l'invite de commande.

📁 Ces commandes sont essentielles pour faire des fichiers de commandes “.bat ” ou “.cmd ” bien utiles pour automatiser les tâches répétitives.

📁 Renommer rapidement les fichiers d'un dossier selon une règle simple. Par ailleurs, l'utilisation de la ligne de commande est indispensable dans certains cas, en particulier lors de connexions distantes à un autre ordinateur. S'il est possible d'utiliser des interfaces graphiques à distance grâce à des

protocoles spécialement étudiés, ces derniers sont par essence plus gourmands en ressources réseau. Pour une connexion lente, le dialogue est nécessairement plus fluide en ligne de commande.

II. ACCÉDER À L'INVITE DE COMMANDE

a L'invite

L'**invite** ou **prompt** du DOS est un message d'attente qui permet de donner le signal que le système est prêt à recevoir des commandes ; il se présente comme ceci : **C:\>_**

L'invite indique la position du répertoire actif : le nom du disque par défaut ici est **C:**, suivi du chemin complet à partir du répertoire racine représenté par le caractère « back-slash » ****.

Le caractère d'attente « **>** », lui marque la fin du nom du répertoire courant et enfin le curseur qui clignote indique à un utilisateur où est-ce que les caractères du clavier seront tapés.

b La ligne de commande

Une ligne de commande est en quelque sorte une phrase ou un groupe de mot qui se tape juste devant l'invite. Elle a la structure suivante : **commande** [**\option**] [**argument**]

La ligne de commande est analysée par interpréteur de commande (COMMANDE.COM ou CMD.EXE), bon bref dans une ligne de commande, les **arguments** et les **commandes** proprement dite sont séparés par les **espaces**. Donc vous devez prendre l'habitude de ne pas oublier ces espaces, puisque dans certaine version du DOS ils tolèrent alors dans d'autre pas.

📖 Une ligne de commande commence toujours par un mot clé, ce dernier représente une tâche à exécuter par le système d'exploitation

📖 Viennent ensuite les options, par fois appelé les paramètres ou commutateurs, elles sont facultatives et toujours représenté par un « slash » suivi d'une lettre, la fonction de l'option est de modifier la manière dont la commande va se réaliser

📖 Les arguments viennent en complément ; dans la présentation de la syntaxe de la commande les arguments sont représentés entre crochet pour dire que leur présence n'est pas toujours indispensable.

Exemple : voilà quelques exemples de commande :

C:\> CD \Documents\recherche, la commande CD accompagné de l'argument \Documents\recherche, permet à un utilisateur de se déplacer dans le répertoire racine (C:\>), plus précisément à l'emplacement \Documents\recherche.

C:\> TIME, est une commande sans argument permettant d'afficher l'heure exacte ;

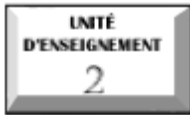
C:\> cls, est une commande (clear screen) qui permet de nettoyer l'écran de l'invite de commande ;

C:\> VER, elle affiche la version du système d'exploitation installé sur votre ordinateur.

EXERCICES

Exercice I

- 1- Donner deux importances de la ligne de commande ;
- 2- Donner la syntaxe de l'invite de commande ;
- 3- Comment accéder à la ligne de commande ?



Utilisation de l'invite de commandes

Compétences visées :

- ☑ Utiliser quelques commandes de manipulation de répertoires et fichiers (créer, modifier et supprimer, parcourir... un répertoire et fichiers)
- ☑ Utilisation de la commande d'aide.

Situation problème :

Entre amie, l'un 'entre eux te demande de lui envoyer dans une clé USB quelques fichiers (audio, vidéo ...), qui sont disposés à des endroits différents de ton ordinateur qu'il avait vu dans ton ordinateur et qu'il a maîtrisé les noms des fichiers simplement, comment satisfaire la demande de ton ami ?

- 📁 Qu'est-ce que la ligne de commande ?
- 📁 Comment ouvrir un logiciel qui n'a pas de raccourcis au bureau ?
- 📁 Comment ouvrir une ligne de commande ?
- 📁 Quelles sont les procédures à suivre s'il s'agissait des objets physiques, dont tu as besoin d'envoyer à quelqu'un ? **réponse attendue : je rassemble les objets, les mets dans un seul emballage et enfin je les expédie.**
- 📁 Quelles sont les différentes opérations que tu as utilisées
- 📁 A quoi correspondent ces différentes actions en informatique ?

I. LANCEMENT DE L'INVITE DE COMMANDE

Tout comme linux, Windows dispose d'une invite de commande permettant de lancer le programme « cmd », autorisant d'entrer directement les commandes au clavier pour appeler le programme ; alors le programme « cmd » peut être lancé de 2 façons différentes :

Première façon :

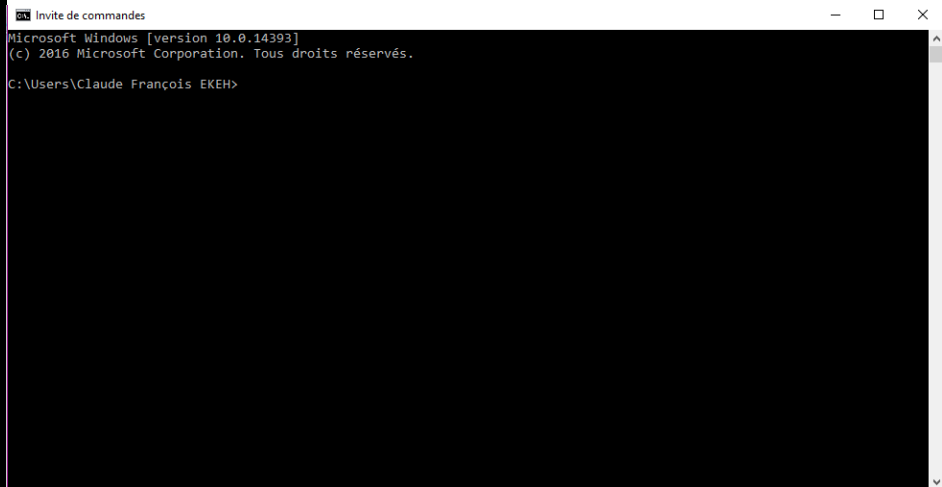
- 📁 Aller dans le menu démarrer
- 📁 Programme
- 📁 Accessoires

Invite de commande

Seconde façon :

-  Aller dans le menu démarrer
-  Exécuter
-  Entrer dans la zone le nom du programme, à savoir **cmd**
-  Enfin valider 'entrer'.

Et ensuite il s'affiche à vous une fenêtre de la forme suivante :



L'on constate qu'on peut entrer une commande, lorsque le curseur clignotant se trouve sur la ligne qui affiche en générale le nom du répertoire courant, suivi d'un chevron ">".

II. LES COMMANDES

L'invite de commande permet de lancer tout programme donc l'exécutable se trouve dans un répertoire listé dans la variable d'environnement « PATH » de l'invite. Si l'on tente de lancer un programme qui n'existe pas ou qui ne se trouve pas dans le chemin d'accès, la réponse est la suivante :

Par exemple si l'on tape « infor » comme programme voila ce qui s'affiche : **'infor'** n'est pas reconnu entend que commande interne ou externe, un commande exécutable ou un fichier de commandes.

I Création d'un répertoire

La commande de création d'un répertoire est la suivante : MD ou MKDIR, en anglais « make directory », qui signifie créer un répertoire ; la structure est la suivante : **MD nomDuRepertoire**

Exemple : C:\> MD Coursinfo ; cette commande crée un répertoire nommé **coursinfo** dans l'emplacement courant.

2 Changer de répertoire

La commande CD (change directory), est celle qui permet de se déplacer d'un répertoire à l'autre, la structure est la suivante : **CD cheminVersleRepertoireVoulu**

Exemple : C:\> CD ubuntu, après avoir tapé cette commande il s'affiche ceci : C:\ubuntu> le répertoire \ubuntu devient alors le répertoire par défaut.

Cas particulier de commande : CD .., qui veut dire aller au répertoire parent et CD \, veut dire aller au répertoire racine.

3 Supprimer un répertoire/fichier

a. Suppression d'un répertoire

RD ou RMDIR, qui signifie « Remove directory », supprimer un répertoire, cette commande ne fonctionne que si le répertoire est vide

Exemple : C:\Documents> RD coursinfo, cette commande supprime le sous-répertoire coursinfo qui se trouve Documents

b. Suppression d'un fichier

DEL ou ERASE est la commande de suppression d'un fichier, la structure est la suivante : **DEL nomDuFichier**

Exemple : C:\Documents> DEL coursinfo.txt

4 Renommer un répertoire/fichier

a. Renommer un répertoire

La commande pour renommer un répertoire ou un fichier est la suivante : **REN** ou **RENAME** et la structure est **REN Nom1 Nom2** (Nom1= fichier à renommer et Nom2= nouveau nom du fichier)

Exemple : C:\Documents> REN coursinfo coursmaths

b. Renommer un fichier

La commande pour renommer un fichier est pareil que celle pour un répertoire, mais au niveau du nom du répertoire l'on doit préciser ici le nom du fichier

Exemple : C:\Documents> REN courinfo.txt coursinformatique.doc

5 Afficher le contenu d'un répertoire

Pour afficher le contenu d'un répertoire ou sous répertoire et fichier à l'emplacement courant, on doit utiliser la commande **DIR** (qui signifie **Directory**) accompagné du nom du dit répertoire, la structure est la suivante :

DIR emplacementDuRepertoire

Exemple : C:\Documents> DIR, cette commande affiche tout répertoire et fichier que contenu dans Documents.

NB : on peut utiliser les options * et ?. Ainsi, ? permet de représenter un caractère et * permet de représenter une chaîne de caractère.

Exemple : DIR b ??.* affiche tous les fichiers ayant trois caractères et dont le nom commence par b.

DIR *.com affiche tous les fichiers ayant pour extension .com.

DIR b*.c* affiche tous les fichiers dont le nom commence par b et l'extension par c.

6 Déplacer un répertoire ou un fichier

Pour copier-coller un fichier vers un emplacement quelconque, on doit utiliser la commande **COPY**. Sa structure est la suivante :

COPY emplacementDeDepart nouvelEmplacement

C:\Documents> COPY Noël.pdf C:\Courier. Cette commande copie le fichier Noël.pdf dans le répertoire Courier.

NB : Pour couper-coller un fichier vers un emplacement quelconque, on doit utiliser la commande **MOVE**.

La commande **XCOPY** permet de copier aussi bien les fichiers que les répertoires

III. AIDE POUR UNE COMMANDE PARTICULIÈRE

Pour avoir une aide sur une commande précis, on peut utiliser la commande **HELP**, ou la structure suivante : **commande / ?**

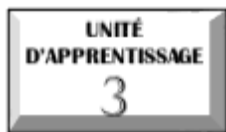
Exemple : pour avoir une idée de ce qu'est ça fait la commande **DIR**, il te faut juste taper : **DIR /?** Ou **HELP DIR**.



EXERCICES

Exercice I

- 1- Qu'est-ce que l'invite de commande ?
- 2- Qu'est-ce que la ligne de commande ?
- 3- Donner la structure générale d'une ligne de commande ;
- 4- Donner les commandes permettant d'exécuter ces tâches suivantes :
 - se déplacer dans un répertoire ;
 - copier un fichier ;
 - créer un fichier ;
 - renommer un fichier ;
 - supprimer un fichier ;
 - créer un répertoire
 - renommer un répertoire ;
 - supprimer un répertoire ;
 - afficher le contenu d'un répertoire courant.
- 5- Quand on a des difficultés avec certaines commandes, quel utilise-t-on pour avoir de l'aide ?

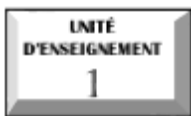


APPLIQUER LES CONCEPTS FONDAMENTAUX DE LA SÉCURITÉ INFORMATIQUE

S
É
C
U
R
I
T
É

I
N
F
O
R
M
A
T
I
Q
U
E

 U.E I : Introduction à la sécurité informatique



Introduction à la sécurité informatique

Compétences visés :

- ☞ Définir les concepts fondamentaux de la sécurité informatique (confidentialité, disponibilité, intégrité, non répudiation)
- ☞ Définir les concepts de cybercriminalité, cybersécurité
- ☞ Décrire les techniques de protection des données

Situation problème1 :

Vous recevez un message d'une personne inconnue X vous disant que votre numéro de téléphone à été tiré au sort vous donnant ainsi la possibilité de gagner une importante cagnotte de 1000000 frs(un million de francs). Mais pour entrer en possession de votre argent l'inconnu X vous propose de composer et de valider un code qu'il va vous donner. Très ému de cette offre mielleuse, vous composez et validez le code qu'il vous a transmis. Quelques secondes plus tard vous recevez un message mobile money vous stipulant que votre compte à été vidé. C'est alors que vous comprenez que vous venez de vous faire arnaquer. Comment cela à t'il été possible ? comment appelle-t-on ce genre de personnes et de délits commis à travers les réseaux informatiques ?

Situation problème2 :

La société Alpha est une entreprise qui fait dans la recherche et le développement. A la fin d'une étude basée sur le développement rural, le fruit de ses recherches est divulgué sur internet. Ce posant plusieurs questions, le responsable de cette structure demande aux experts comment leurs données ont pu se retrouver dans la rue sans leur autorisation ?.

INTRODUCTION

Avec le développement de l'utilisation d'internet, de plus en plus d'entreprises ouvrent leur système d'information à leurs partenaires ou leurs

fournisseurs, il est donc essentiel de connaître les ressources de l'entreprise à protéger et de maîtriser le contrôle d'accès et les droits des utilisateurs du système d'information. Il en va de même lors de l'ouverture de l'accès de l'entreprise sur internet.

La sécurité informatique, d'une manière générale, consiste à assurer que les ressources matérielles ou logicielles d'une organisation sont uniquement utilisées dans le cadre prévu. La sécurité informatique vise à réduire au maximum le **risque** qui est une combinaison des notions (menace, vulnérabilité et contre-mesure). La menace représente le type d'action susceptible de nuire dans l'absolu, tandis que la **vulnérabilité** (en anglais « *vulnerability* », appelée parfois *faille* ou *brèche*) représente le niveau d'exposition face à la menace dans un contexte particulier. Enfin la **contre-mesure** est l'ensemble des actions mises en œuvre en prévention de la menace.

I. OBJECTIFS DE LA SECURITE INFORMATIQUE

La sécurité informatique vise généralement cinq principaux objectifs :

- ☞ L'**intégrité**, c'est-à-dire garantir que les données sont bien celles que l'on croit être ; consiste à déterminer si les données n'ont pas été altérées durant la communication (de manière fortuite ou intentionnelle).
- ☞ La **confidentialité**, consistant à assurer que seules les personnes autorisées aient accès aux ressources échangées ; consiste aussi à rendre l'information inintelligible à d'autres personnes que les seuls acteurs de la transaction.
- ☞ La **disponibilité**, permettant de maintenir le bon fonctionnement du système d'information ; L'objectif de la disponibilité est de garantir l'accès à un service ou à des ressources.
- ☞ La **non répudiation**, permettant de garantir qu'une transaction ne peut être niée ; La **non-répudiation** de l'information est la garantie qu'aucun des correspondants ne pourra nier la transaction.
- ☞ L'**authentification**, consistant à assurer que seules les personnes autorisées aient accès aux ressources. L'authentification consiste à assurer l'identité d'un utilisateur, c'est-à-dire de garantir à chacun des correspondants que son partenaire est bien celui qu'il croit être. Un contrôle d'accès peut permettre (par exemple par le moyen d'un mot de passe qui devra être crypté) l'accès à des ressources uniquement aux personnes autorisées.

II. LES ATTAQUES

Il existe quatre catégories principales d'attaque :

📁 **Les attaques par l'accès.** C'est une tentative d'accès à l'information par une personne non autorisée.

Exemples : **Sniffing** cette attaque est utilisée par les pirates informatiques pour obtenir des mots de passe. **Craquage de mot de passe** il consiste à faire de nombreux essais jusqu'à trouver le bon mot de passe

📁 **Les attaques par modification,** elle consiste, pour un attaquant à tenter de modifier des informations. Quelques exemples de ce type d'attaque : **Virus et les vers etc...** (Un vers comme un programme caché dans un autre qui peut s'exécuter et se reproduire en infectant d'autres programmes ou d'autres ordinateurs il se déplace à travers un réseau.)

📁 **Les attaques par déni de service,** elle consiste à envoyer des milliers de messages depuis des dizaines d'ordinateurs, dans le but de submerger les serveurs d'une société, de paralyser pendant plusieurs heures son site Web et d'en bloquer ainsi l'accès aux internautes.

Exemple Le flooding Cette attaque consiste à envoyer à une machine de nombreux paquets IP de grosse taille. **Le smurf** c'est une attaque qui s'appuie sur le ping et les serveurs de broadcast. On falsifie d'abord son adresse IP pour se faire passer pour la machine cible.

📁 **Les attaques par répudiation.** La répudiation est une attaque contre la responsabilité. Autrement dit, la répudiation consiste à tenter de donner de fausses informations ou de nier qu'un événement ou une transaction se soit réellement passé.

Ces attaques sont basées sur les types de virus suivant :

☞ **virus:** un programme malveillant ayant pour but de nuire au bon fonctionnement de l'ordinateur.

☞ **Ver(worm):** programme malveillant qui se propage en utilisant un réseau informatique (comme Internet)

☞ **cheval-de-troie (trojan):** un programme malveillant qui ouvre une porte dérobée dans un système afin de l'exploiter ultérieurement.

☞ **Spam:** courrier indésirable envoyé dans l'intention de créer des déni de service, ou alors de distraire et de faire oublier l'essentiel.

III. CONCEPTS DE CYBERSECURITE ET CYBER CRIMINALITE

La cyber-sécurité renvoie à la protection des personnes, des idées et des données dans le cyber espace. La sécurisation des données informatiques ou cyber sécurité, passe par une sécurisation des matériels et logiciels informatiques constituant le support de transmission, de stockage et de traitement de l'information. L'augmentation de la sécurité des matériels et logiciel informatique, permettrait donc d'augmenter la sécurité des informations.

Deux types de personnes animent ces concepts :

 **Black hat hacker:** pirate des systèmes informatiques, s'introduisant de façon furtive ou masquée pour voler des informations sensibles ou créer des dénis de service. Ces genres de personnes font dans ce qu'on appelle **cybercriminalité** (le fait d'utiliser ses connaissances informatique pour commettre des délits des actes préjudiciables).

 **white-hat hacker :** ce sont des « hackers éthiques » des experts en sécurité informatique qui utilisent leurs capacités à des fin honnêtes éthiques et du côté de la justice. Un **hacker white-hat** qui trouve une faille de sécurité dans une application la rapportera à son développeur lui permettant ainsi d'améliorer la sécurité de celle-ci avant qu'elle ne soit compromise. Ces genres de personnes font dans ce qu'on appelle **cybersécurité** (le fait d'utiliser ses connaissances informatiques pour empêcher la défaillance, détecter et fermer les failles d'un système informatique dans une entreprise).

IV. PROTECTION DES DONNEES

La protection des données consiste à déployer des moyens et des dispositifs visant à sécuriser le système d'information ainsi que de faire appliquer les règles définies dans la politique de sécurité. Les principaux dispositifs permettant de sécuriser un réseau contre les intrusions sont :

 **Les mots de passe** Lors de la connexion à un système informatique, celui-ci demande la plupart du temps un **identifiant** (en anglais *login* ou *username*) et un **mot de passe** (en anglais *password*) pour y accéder. Ce couple *identifiant/mot de passe* forme ainsi la clé permettant d'obtenir un accès au système. Pour des données sensibles à protéger il est conseillé d'utiliser un mot de passe d'au moins 14 caractères contenant des lettres, des chiffres et des caractères spéciaux.

 **Les systèmes pare-feu.** Dispositif matériel et logiciel qui protège un système informatique connecté à internet des tentatives d'intrusion qui pourraient en provenir. Le pare feu définit les types de communications autorisés, surveille et contrôle les applications et les flux de données. Néanmoins ce type de dispositif ne protège pas la confidentialité des données circulant sur le réseau.

 **La cryptographie** discipline de la cryptologie s'attachant à protéger les messages en utilisant des clés. La cryptographie vise à rendre le message inintelligible à toutes autres personnes sauf le destinataire du message. Elle permet ainsi de garantir la confidentialité des échanges. On s'appuie généralement sur plusieurs algorithmes cryptographiques tels que DES, RSA, etc..

 **Antivirus** ce sont des programmes permettant de détecter et de neutraliser éliminer tout programme malveillant présent dans l'ordinateur. Il existe plusieurs antivirus tels que : Kaspersky, Avast, Norton, Avira etc..

 **Le VPN** (Virtual Private Network) correspond à la mise en place de tunnels sécurisés. Ce système permet de créer un lien direct entre des ordinateurs distants qui isole leurs échanges du reste du trafic se déroulant sur le réseau. (permet d'obtenir un niveau de sécurisation supplémentaire dans la mesure où l'ensemble de la communication est chiffrée.

 **Droit d'accès et privilèges** les droits d'accès sont des privilèges attribués à un utilisateur pour accéder à des informations. L'utilisateur peut bénéficier des privilèges suivants : lecture, écriture ou modification, privilège d'administration dans le cas des serveurs ou des appareils de sécurité comme le firewall (pare feu matériel). L'attribution des privilèges peut se faire à un utilisateur spécifique ou à un groupe d'utilisateurs. Il existe plusieurs niveaux de privilèges :

- **Le super administrateur** : c'est celui qui est propriétaire de tous les dossiers et fichiers du système
- **L'administrateur** : reçoit du super administrateur le droit aussi d'accéder et de modifier les données sur tous les fichiers
- **L'invité** : dispose d'un accès limité dans le système ; ne peut pas modifier ou accéder aux données dont il n'a pas créé.

CONCLUSION

La sécurité d'un système informatique doit être abordée selon une approche globale. Ainsi, une porte blindée est inutile dans un bâtiment si les fenêtres sont ouvertes sur la rue.

Cela signifie que la sécurité doit être abordée dans un contexte global et notamment prendre en compte les aspects suivants :

-  **La sensibilisation des utilisateurs aux problèmes de sécurité**
-  **La sécurité logique**, c'est-à-dire la sécurité au niveau des données, notamment les données de l'entreprise, les applications ou encore les systèmes d'exploitation.
-  **La sécurité des télécommunications : technologies réseau, serveurs de l'entreprise, réseaux d'accès, etc.**
-  **La sécurité physique**, soit *la sécurité au niveau des infrastructures matérielles* : salles sécurisées, lieux ouverts au public, espaces communs de l'entreprise, postes de travail des personnels, etc.



EXERCICES

Exercice I

Question1 : Définir la notion d'intégrité des données ainsi que les objectifs du contrôle d'intégrité

Question2 : quelles relations existent entre les critères d'intégrité et de confidentialité ?

Question3 : Dans quelle mesure peut-on considérer des principes d'éthique comme faisant partie d'une démarche de sécurité informatique ?

Question4 : Sur quels principes se fonde la réalisation des attaques informatiques ?

Question5 : Quelle est la place de la maîtrise de la cybercriminalité dans une démarche de cyber sécurité ?

Question6 : Ressortez clairement la différence entre un virus, un ver, un cheval de Troie, une bombe logique.

Question7 : Parmi les infrastructures qui composent un système d'information laquelle ne peut être concernée par une cyberattaque ?

A - Matérielle B- Réseau C- Logicielle D- Exploitation E- Structurelle

Question8 : Parmi les attaques informatiques suivantes quelle est celle qui peut être qualifiée d'attaque passive ?

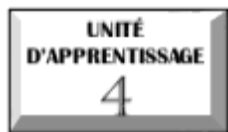
A - modification B- interception C- Fabrication D- Interruption E- Destruction

Question9 : Parmi les propositions suivantes laquelle correspond le mieux à la définition d'une infraction informatique ?

A – « Tout comportement illégal, immoral ou non autorisé qui implique la transmission et/ou le traitement automatique des données »

B- « tout comportement illégal qui implique l'usage de l'informatique et des télécommunications »

Question10 : Expliquez comment le concept de « séparation des tâches » contribue à la sécurité informatique d'une organisation.

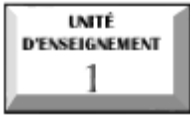


UTILISER LES FICHIERS MULTIMÉDIAS

U.E I : Utiliser les fichiers multimédias

F
I
C
H
I
E
R
S

M
U
L
T
I
M
É
D
I
A
S



Utiliser les fichiers multimédias

Compétences visées :

- ☞ Décrire les caractéristiques (définition, résolution, taille) d'une image
- ☞ Calculer la résolution d'une image matricielle ;
- ☞ Calculer la taille d'un fichier multimédia (image, son, vidéo) ;

Situation problème :

Lors du mariage du frère de Paul, il le charge du reportage (enregistrement audio, photos, vidéos, etc.) qu'il réalise grâce à un appareil adéquat et les stocke dans son ordinateur. Puisqu'il doit partager les fichiers, il vérifie les caractéristiques et obtient les informations parmi lesquelles : la résolution, la définition, la taille. Paul ne comprenant pas ces différents éléments viens vous demandez votre aide pour :

- i. Décrire les différentes caractéristiques d'une image numérique.
- ii. Décrire comment calculer la résolution d'une image ;
- iii. Expliquer comment calculer la taille d'un fichier (image, audio, vidéo)

INTRODUCTION

L'ordinateur est généralement utilisé pour les activités communes comme écouter la musique, regarder les films, jouer. Cette utilisation fait de l'ordinateur un appareil multimédia car il allie simultanément du texte, de l'image, du son, de la vidéo qui sont tous des médias. Un média est un support de communication

I. LES CARACTÉRISTIQUES D'UNE IMAGE

Une image est la représentation d'un objet sous forme de dessin ou photo. Elle peut être acquise grâce à une webcam, un appareil photo numérique, un scanner, etc.

I. Les types d'image

Il existe deux principaux types d'images qui sont :

 **Les images matricielles** : image constituée d'un ensemble de points appelés pixels (chaque point possède une ou plusieurs valeurs décrivant sa couleur) ; ici, une transformation géométrique d'une image bitmap entraîne une perte de sa qualité.

 **Les images vectorielles** : Ce sont des images constituées uniquement d'entités géométriques. Il est facilement de leurs appliquées des transformations géométriques (zoom, étirement...), sans risque de perte de la qualité d'image.

2. Caractéristiques d'une image

Une image présente plusieurs caractéristiques parmi lesquelles :

a. La définition

C'est le nombre total de pixel d'une image ; elle est notée largeur $\times$ hauteur. (Exemple : $200 \times 100 = 20000$ pixel ;). Il peut aussi s'exprimer en mégapixel (MPX) sachant que $1 \text{ mpx} = 1 \text{ million de pixel}$.

b. La résolution

C'est la finesse de détails atteinte par un moniteur ou une imprimante pour la production d'une image. La résolution est exprimée en pixel par pouce (ppp, $1 \text{ pouce} = 2.54 \text{ cm}$).

c. La taille ou poids

C'est la taille numérique d'un fichier image mesurée en Ko. La taille est proportionnelle aux dimensions en pixels de l'image. C'est le produit du nombre de pixel par le poids ou taille d'un pixel (pour une image de 320×200 et une taille de pixel de 1 bit on a une taille de 7.8 Ko) ;

$320 \times 200 = 64000 \text{ px}$ or $1 \text{ px} = 1 \text{ bit}$ et $1 \text{ octet} = 8 \text{ bit}$ d'où $64000 \text{ px} = 64000 \text{ pixel}$ et $64000 / 8 = 8000 \text{ o}$; $1 \text{ ko} = 1024 \text{ o}$, donc $8000 \text{ o} / 1024 = 7,8125 \text{ ko}$ donc environ 7,8 ko.

II. CALCULER LA RÉOLUTION D'UNE IMAGE MATRICIELLE

La résolution c'est le nombre de points ou pixels par unité de surface, de manière conventionnelle, on parle de **dpi** (dot per inch, ou pixel par pouce). Généralement, l'unité de mesure (ou de surface) est le pouce mais peut aussi être le centimètre.

La résolution d'une image matricielle correspond au nombre de pixel contenu sur l'image dans sa longueur divisée par sa taille de l'image en longueur ou correspond au nombre de pixels contenu sur l'image dans sa hauteur divisée par la taille de l'image en hauteur.

Pour calculer la taille en dpi d'une image, il faut prendre deux paramètres : la définition de l'image et la taille de l'impression qu'on souhaite obtenir. Exemple : pour connaître la résolution d'impression d'une image de 600×400 px (définition de 0,24Mpx) tirée au format ; tirée au format ($29,7 \times 42$ cm) un calcul s'impose (Taille en pixels $\times 2,54$) / (taille en cm) = dpi soit $(600 \times 2,54) / 42 = 36,28$ dpi et $(400 \times 2,54) / 29,7 = 34,2$ dpi

III. CALCULER LA TAILLE D'UN FICHIER MULTIMÉDIA

I. Audio

Le débit c'est la taille du fichier par rapport à la durée du son. Débit (kbit/s) = fréquence d'échantillonnage (kHz) $\times$ quantification (bit) $\times$ nombre de canaux

Exemple : le CD audio :

- 📁 Fréquence d'échantillonnage : 44100Hz
- 📁 Résolution : 16 bits ;
- 📁 Canaux : 2 (stéréo)

Le débit est donc : débit = $44100 \times 16 \times 2 = 1411200$ bit/s. Soit environ

- 📁 1411kbit/s, ou 1378kibibit/s
- 📁 Par minute, on a 84,7Mbit ou 80,7 Mibibit ;
- 📁 Par heure, on a 5,1Gbit

Poids

Taille d'un MP3 à 192kbit/s d'une durée de 3,75 minutes : [temps (s)] $\times$ débit (kbit/s) / 8000 : $[3,75 \times 60] \times 192 / 8000 = 5,4$ Mo.

2. Vidéos

Le poids d'une vidéo dépend de son débit d'encodage et de sa durée. Si le poids s'exprime généralement en méga-octets (Mo), le débit est généralement connu en kilobits/seconde (Kbps). Pour effectuer les conversions, on sait qu'un octet = 8

bits et qu'un Mo = 1024 Ko. On a alors la relation : poids (Mo) = débit (Kbps) x durée (s) / 8 / 1024

Exemple : le poids en Mo d'une vidéo de 2' encodée à 500Kbps est de $500 \times 120 / 8 / 1024 = 7,32\text{Mo}$

Lorsqu'une vidéo est encodée à plusieurs débits, le poids total qu'occuperont ses différentes versions correspond au poids d'une vidéo dont le débit serait la somme des débits disponibles.

3. Image (photo)

La taille d'un fichier image est calculée en multipliant la surface d'un document (hauteur × largeur) à numériser par la profondeur de bit et le dpi au carré. La taille de l'image étant exprimée en bytes ou octets, le nombre obtenu est divisé par 8.

Formule 1 : Taille du fichier = (hauteur × largeur × profondeur de bit × dpi²)/8

Si les dimensions en pixels sont données, multipliez-les entre elles et par la profondeur de bits pour définir le nombre de bits d'une image. Par exemple, si une image 24 bits est capturée avec un appareil photo numérique aux dimensions de 2048 par 3072, alors la taille du fichier est égale à $(2048 \times 3072 \times 24)/8$, soit 18.874.368 bytes

Formule 2 : Taille du fichier = (dimensions en pixels × profondeur de bit)/8

1 kilo-octet (KB ou Ko) = 1024 bytes ou octets ;

1 Mega-octet (MB ou Mo) = 1024 KB (ou Ko) ;

1 giga-octet (GB ou Go) = 1024 MB (ou Mo) ;

1 tera-octet (TB ou To) = 1024 GB (ou Go)



EXERCICES

Exercice 1

1. Définition : image, définition, résolution et taille d'une image.
2. Donner les deux principaux type d'image.
3. Citer les principales caractéristiques d'une image.

Exercice 2

1. Déterminer le nombre de pixel d'une image 800×400 . Déterminer la largeur et la hauteur de cette image sachant qu'elle présente une résolution de 72 ppp.
2. Calculer la résolution d'une image bitmap carrée de côté 10 cm et de définition 800×800 .

Exercice 3

Un son audio a une fréquence d'échantillonnage de 44,1 kHz (44 100 Hz) et une hauteur de quantification sur 16 bits sachant qu'on a deux piste. Calculer le débit binaire de cet audio.

Exercice 4

On veut réaliser une carte postale de format 10cm x 15cm, orientation portrait, qualité professionnelle

- a) Quelle doit être la résolution de cette image ?
- b) Calculer le nombre de pixel pour cette image. On prend 1 pouce = 2,5cm
- c) Calculer le nombre de lignes de pixels pour cette photo. On prend 1 pouce = 2,5cm

Exercice 5

La notice d'un appareil photographique numérique indique une « résolution » du capteur CCD de 3072×2048 , soit 6,3 mégapixels (Mpx). Une image est dite de qualité « photo » quand la taille du pixel est suffisamment petite pour qu'un œil normal n'en perçoive pas les détails. On considère qu'un œil normal peut percevoir des détails lorsque les rayons lumineux issus de ces détails arrivent dans l'œil avec un angle supérieur à une minute. En codage normal, un pixel est codé en RVB 24 bits.

Données : 1 Mio = 1 024 Kio et 1 Kio = 1 024 octets ; 1 pouce = 2,54 cm ; 1 minute d'angle = $(1/60)^\circ$.

1. a. Qu'appelle-t-on une image numérique ?

b. Par abus de langage, les fabricants utilisent le terme « résolution ». Quel est celui qui convient en réalité ?

c. Le constructeur affiche une « résolution » de 3 072 x 2 048. Que représentent ces valeurs ?

2. a. Combien d'octets sont utilisés pour coder un pixel ?

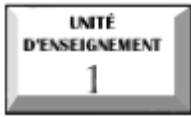
b. Déterminer la taille d'une image correspondant à la « résolution » indiquée par le fabricant. Exprimer le résultat en Mio.



DÉCRIRE LES CONCEPTS DE BASE DES SYSTÈMES D'INFORMATION

 U.E I : Introduction aux systèmes d'information

 U.E 2 : Méthodes d'analyse et de conception des systèmes d'information



Introduction aux systèmes d'information

Compétences visées :

- ☑ Décrire les systèmes d'une entreprise,
- ☑ Énumérer les composants d'un SI,
- ☑ Décrire les fonctions d'un SI.

Situation problème :

Vous avez été sélectionné pour faire un stage de vacance dans une boulangerie de fabrication du pain dans votre quartier. A la fin de votre stage, la série de question suivante vous êtes posée afin de vérifier vos compétences acquises.

1. De quoi a besoin l'entreprise pour fabriquer du pain ? Comment fait-elle pour acquérir ce besoin ?
2. Proposer quelques exemples d'actions du dirigeant de cette entreprise
3. Donner quelques exemples d'actions des employés de cette entreprise
4. Comment l'entreprise fait-elle pour mémoriser, traiter et diffuser ses informations ?
5. Quel nom donne-t-on à l'ensemble des ressources de l'entreprise permettant de recueillir, d'enregistrer, de traiter et de diffuser les informations ?

INTRODUCTION

Toute entreprise recherche une bonne organisation afin d'atteindre facilement ses objectifs. L'entreprise s'attarde beaucoup à l'information car cette dernière lui permet de prendre des décisions. Une information représente un élément de connaissance pouvant être collecté, traité, conservé et diffusé dans une organisation. Pour gérer l'information, l'entreprise dispose d'une composante capable de collecter, de stocker, de traiter et diffuser les informations : C'est le système d'information (SI). Ainsi, Un SI est l'ensemble de ressources (matérielles, logicielles, humaines) structurées pour collecter, traiter, mémoriser et diffuser l'information au sein d'une organisation. Dans la suite de notre cours, nous

décrierons les différents systèmes d'une entreprise, ensuite nous présenterons les composants et les fonctions d'un système d'information.

I- DESCRIPTION DES SYSTEMES D'UNE ENTREPRISE

Une entreprise est généralement composée de trois composantes (ou système) en interaction qui sont : le système de pilotage, le système opérant et le système d'information.

1. Le système de pilotage :

Il est chargé de diriger l'entreprise et de prendre des décisions sur les actions à conduire. Il contrôle le système opérant en décidant de son comportement compte tenu des objectifs de l'entreprise.

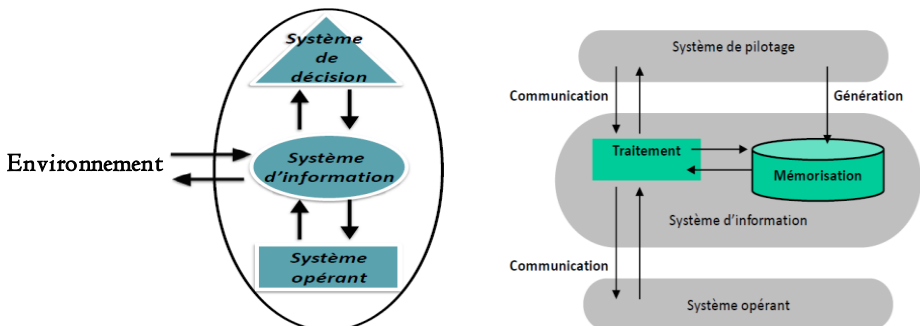
2. Le système opérant :

Ce système assure l'exécution des tâches en transformant les flux d'entrée en flux de sortie. Un flux est un échange d'information entre 2 acteurs. Il communique avec le Système de pilotage par l'intermédiaire du système d'information.

3. Le système d'information :

Ce système s'occupe de collecter, de traiter, de stocker et de diffuser l'information dans l'entreprise. Il reçoit du système opérant des données collectées, les transforme en informations utiles et informe le système de pilotage des performances du système opérant.

Illustration : Les systèmes d'une entreprise



II-LES COMPOSANTES D'UN SYSTEME D'INFORMATION (SI)

Un Système d'information contient généralement trois composantes à savoir, les ressources humaines, matérielles et logicielles.

I. Les ressources humaines :

Il existe deux types de ressources humaines dans un Système d'Information : l'utilisateur final et les informaticiens.

a. L'utilisateur final :

Il s'agit de toute personne qui utilise le système d'information ou l'information qu'il produit. Ce type d'utilisateur peut aussi contribuer à l'acquisition, au stockage et au traitement de l'information.

Exemples : Les commerciaux, les clients, les Direction des Ressources Humaines, les Dirigeants, etc.

b. Les informaticiens : ces sont des spécialistes (analyste, programmeurs, operateurs, etc.) des SI capables de concevoir, d'implanter et de faire fonctionner le SI.

Exemples :

-  Les analystes : conçoivent le Système d'information en fonction des besoins d'information des utilisateurs ;
-  Les programmeurs : élaborent des programmes informatiques en fonction des exigences des analystes ;
-  Les operateurs : exploitent sur ordinateur les systèmes d'information.

2. Les ressources matérielles :

Elles englobent tous les dispositifs physiques et toutes les machines qui servent au traitement de l'information. On distingue principalement :

-  les ordinateurs portables et de bureau,
-  les serveurs,
-  les supports de stockage (Disque dur, clé USB, etc.) pour la sauvegarde des données,

- 🖨 les équipements de réseaux informatiques (commutateur, routeur, concentrateur, modem,),
- 🖨 les imprimantes,
- 🖨 les dispositifs de sécurité (camera de surveillance, lecteur de badge, lecteur de carte, lecteur d'emprunte, etc.),
- 🖨 les écrans géants,
- 🖨 le vidéo projecteur, etc.

3. Les ressources logicielles :

Il s'agit de l'ensemble de logiciels utilisés dans l'entreprise. On distingue les logiciels de base et les logiciels d'application :

a. Les logiciels de base : ce sont des programmes qui gèrent et commandent le matériel informatique. On distingue :

🖨 Le Système d'exploitation : Exemples : Windows, Ubuntu, Fedora, Mac OS, Debian, etc.

🖨 Les Utilitaires : Programme informatique permettant de configurer et d'optimiser un ordinateur.

Exemples : Utilitaire d'installation/désinstallation d'un logiciel, défragmentation d'un disque, etc.

🖨 Les pilotes : Programmes informatique permettant au système d'exploitation de communiquer avec les périphériques.

b. Les logiciels d'application : Programme informatique permettant à l'utilisateur d'effectuer un traitement particulier. On en distingue plusieurs catégories :

- 🖨 Antivirus, Exemples : Avast, Kapersky, Norton,
- 🖨 Navigateur, Exemples : Mozilla firefox, Opéra, Google Chrome,
- 🖨 Tableur : Exemples : Excel, Lotus,
- 🖨 Serveur d'Application, Exemples : Apache Tomcat, JBoss, Jonas,
- 🖨 Système de Gestion de Base de Données (SGBD), Exemples : MySQL, Oracle,

 Progiciel de Gestion Intégré (P.G.I) en anglais E.R.P (Enterprise Resource Planning) : logiciel qui regroupe en son sein les principales composantes fonctionnelles d'une entreprise.

Exemples : SAP, SAGE ERP,

 etc.

III- LES FONCTIONS D'UN SYSTEME D'INFORMATION

Les principales fonctions d'un système d'Information sont :

I – Collecter l'information : C'est enregistrer l'information dans un support (papier, informatique) avant son traitement. Les informations peuvent provenir des partenaires de l'entreprise (clients, partenaires, fournisseurs, sous traitants, etc.) : ce sont des sources externes. Elles peuvent aussi provenir des activités de l'entreprise (Approvisionnement, production, comptabilité, ventes, etc.) : ce sont des sources internes.

2 – Stocker l'information : Consiste à conserver l'information dans un support (Disque Dur, clé USB, DVD, papier, etc.) pour une utilisation ultérieure. Ce stockage doit être durable et fiable.

3 – Traiter l'information : C'est effectuer des opérations sur un ensemble de données pour faciliter la prise de décision. Comme exemples d'opérations de traitement, nous avons : les calculs, le classement, le tri, les ajouts, la modification, la suppression, l'extraction de l'information.

NB : Ce traitement peut être manuel ou automatique. Dans le cas où il est automatique, on parle de système d'information informatisé. L'informatisation du SI consiste à automatiser toutes les tâches pour faciliter le travail des utilisateurs.

4 – Diffuser l'information : C'est transmettre l'information à la bonne personne dans les meilleurs délais après son traitement. Les moyens de diffusion de l'information sont multiples : support papier, support numérique, forme orale, internet et l'interconnexion des S.I, etc.



EXERCICES

Exercice I

1. Définir les expressions suivantes : entreprise, Information, donnée, Système d'information, Système informatique, ERP,
2. Décrire les différentes composantes d'un SI,
3. Donner 4 fonctions d'un SI,
4. Décrire brièvement chacun des systèmes suivants : Système de pilotage, Système opérant,
5. Illustrer à l'aide d'un schéma, les différents systèmes d'une entreprise,
6. Proposer quelques exemples d'acteurs (interne/ externe) dans une organisation,
7. Proposer quelques exemples de Cellule de Direction dans une entreprise
8. Quelle relation existe-t-il entre le Système d'information et le système informatique ?

Exercice 2 : SI d'un lycée

Pour vous inscrire dans votre lycée, vous avez déposé un dossier d'inscription qui a été étudié. Après quoi, vous avez reçu un courrier d'acceptation de ce dossier et votre inscription a été validée.

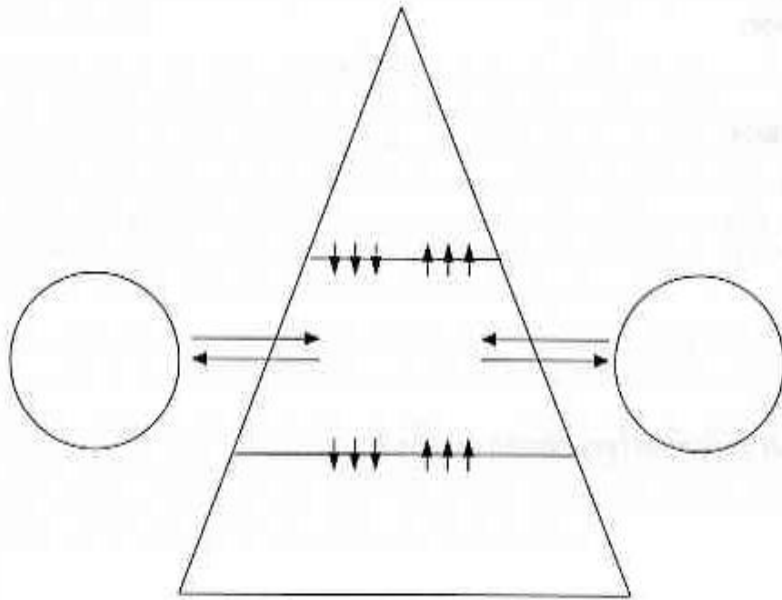
Au début du mois de septembre, une réunion du Conseil d'administration, présidée par votre proviseur, a déterminé des dates qui bornent chacun des trimestres de l'année scolaire qui débute. Ces dates sont inscrites dans le règlement intérieur de l'établissement scolaire.

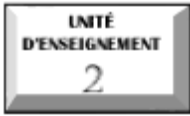
Le proviseur fixe les dates des conseils de classe de chaque trimestre en fonction de ces bornes. Les professeurs doivent avoir rempli les bulletins trimestriels 3 jours avant la date du conseil de classe. Les professeurs doivent également relever les absences de leurs élèves, et les faire remonter à la scolarité qui établit des synthèses à l'attention du proviseur.

Vos parents (si vous êtes mineurs) sont informés de vos résultats par l'intermédiaire du bulletin trimestriel qui leur est adressé. Lorsque vous formulez un choix d'orientation, celui-ci doit également être confirmé par vos parents.

- I. Déterminez quels sont les éléments qui font partie dans un lycée du :
 - a. système de pilotage,

- b. système d'information,
- c. système opérant.
- 2. Quels sont les acteurs externes de l'organisation du lycée ?
- 3. Complétez le schéma d'analyse systémique suivant, en recherchant les flux d'information qui circulent entre les différents sous-systèmes du lycée.





Méthodes d'analyse et de conception des systèmes d'information

Compétences visées :

- ☞ Décrire les intérêts d'un SI
- ☞ Énumérer quelques méthodes d'analyse et de conception d'un SI

Situation problème :

Vous avez été désigné chef de votre salle de classe, et comme première tâche vous êtes censé mettre sur pied un moyen pour gérer la discipline dans la salle de classe.

- 1- Quelle démarche allez-vous adopter?
- 2- De quels outils allez-vous vous en servir?
- 3- Quelles sont les méthodes adaptées à la réalisation de cette tâche?

INTRODUCTION

De nos jours, l'utilisation combinée des moyens, des procédures, et des outils informatiques permet selon les besoins et les intentions exprimées par une entreprise un accompagnement dans l'automatisation et la dématérialisation de toutes les opérations incluses dans ses activités. Toutes fois l'intérêt et la réussite du système d'information d'une entreprise repose sur l'appuie des méthodes d'analyse et de conception.

I- INTÉRÊT D'UNSI

Le système d'information contribue au pilotage de l'organisation ou de ses activités en fournissant de l'information pour son management, il aide ainsi a la prise de décisions

Un bon système d'information permet à une organisation d'accéder de comprendre et de réagir à une information rapidement et efficacement.

Pour les entreprises, la capacité de modifier les données en informations est utile, et inestimable pour le succès de ses affaires dans la résolution des problèmes qu'elle rencontre.

Le système d'information permet de contrôler l'évolution de l'organisation par la détection des dysfonctionnements et des anomalies internes.

II- MÉTHODES D'ANALYSE ET DE CONCEPTION D'UN SI

Une méthode d'analyse et de conception est un procédé qui a pour objectif de formaliser les étapes préliminaires du développement d'un système afin de le rendre plus fidèle. Les méthodes d'analyse et de conception sont basées sur des modèles, un modèle étant une représentation abstraite et simplifiée d'un objet du monde réel en vue de le décrire, de l'expliquer, ou de le prévoir.

Exemple: si on modélise une voiture de manière abstraite, on obtient la représentation suivante.



I- Rôle d'une méthode d'analyse

Le besoin d'une méthode est née de la volonté d'uniformiser la résolutions de certains problème en matière d'analyse de système. Une méthode d'analyse et de conception de SI a un double rôle :

- 📖 Elle guide et indique comment aborder les problèmes.
- 📖 Elle propose des normes ou des standards de présentation de résultat

2- Quelques méthodes d'analyse et de conception

Il existe plusieurs méthodes d'analyse et de conception des systèmes d'information. Chaque méthode répond à une approche bien définie. Les plus connues sont :

METHODE	APPROCHE	DESCRIPTION
MERISE : <i>Méthode d'Etude et de Réalisation Informatique pour les Systèmes d'Entreprise</i>	<i>systémique</i>	Cette méthode est adaptée pour les projets internes aux organisations se limitant à un domaine précis.

SADT : <i>Structured Analysis Design Technic</i>	<i>Cartésienne</i>	Elle permet de décrire les tâches d'un projet et leurs interactions, mais aussi le système que le projet vise à étudier.
UML : <i>Unified Modeling Language</i>	<i>Orientée-objet</i>	Cette méthode a été développée dans le but de définir la notation standard pour la modélisation des applications construites à l'aide d'objets.
OMT : <i>Objet Modeling Technic</i>	<i>Orientée-objet</i>	Cette méthode permet de couvrir l'ensemble des processus d'analyse et de conception en utilisant le même formalisme

CONCLUSION

Il existe de nombreuses méthodes d'analyse et de conception des SI. Chacune d'entre elles a ses particularités. De ces particularités découlent plusieurs modalités tels que la modélisation systémique, la modélisation cartésienne, la modélisation orientée objet... avec comme exemple MERISE, UML, SADT, OMT. Le choix de la méthode d'analyse et de conception adapte à un problème se fait par rapport aux objectifs de la solution que l'on veut développer. ordinateur, une maison, un arbre...

EXERCICES

Exercice 1

- 1- décrire deux intérêts du système d'information dans une organisation.
- 2- définir les concepts suivants : modèle, méthode d'analyse et de conception.
- 3- citer 4 exemples de méthode d'analyse et de conception de SI, et pour chacune d'elle donner son approche.

Exercice 2

Donner un modèle abstrait des objets du monde réels suivant: une personne, un ordinateur, une maison

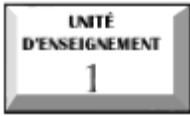


DÉCRIRE LES CONCEPTS DES BASES DE DONNÉES

-  U.E 1 : Généralités sur les bases de données
-  U.E 2 : Les caractéristiques d'une table
-  U.E 3 : Les fonctions d'une base de données

Situation problème :

Le censeur chargé des premières de votre lycée a du mal à gérer les enseignants, les apprenants, les notes et les cours de son niveau. Il pense qu'un outil informatique pourrait l'aider dans l'automatisation de sa tâche quotidienne.



Généralités sur les bases de données

Objectifs

- ☞ Décrire les composants d'un SGBD
- ☞ Énumérer quelques exemples de SGBD ;
- ☞ Énumérer quelques domaines d'application des bases de données ;
- ☞ Déterminer quelques utilités des bases de données ;

Activité :

- De quel outil aurait-il besoin.
- Qu'est-ce qu'une base de données ?
- Donner l'importance d'une base de données
- Quel logiciel permet la définition et la manipulation des bases de données ?
- Citer des exemples de SGBD

Une base de données est une collection structurée et organisée d'information ayant un objectif commun, stockées de manière à limiter les redondances et à faciliter son exploitation.

I. UTILITÉS D'UNE BASE DE DONNÉES

Une base de donnée est un outil informatique robuste et capable de supporter de forts volumes de données ; ces données peuvent être locale, c'est-à-dire utilisable sur une machine par un utilisateur, ou bien répartie, c'est-à-dire que les informations sont stockées sur des machines distantes et accessibles par réseau.

Une base de données permet de gérer finement les droits d'accès à la lecture et la modification des données : elle permet de mettre des données à la disposition d'utilisateurs pour une consultation, une saisie ou bien une mise à jour, tout en

s'assurant des droits accordés à ces derniers. Cela est d'autant plus utile que les données informatiques sont de plus en plus nombreuses.
L'avantage majeur de l'utilisation de bases de données est la possibilité de pouvoir être accédées par plusieurs utilisateurs simultanément.

II. DOMAINE D'APPLICATION DES BASES DE DONNÉES

Les bases de données sont utilisées dans divers domaines de la vie parmi lesquels :

-  Domaine de la **santé** : pour la gestion des patients, des hôpitaux, ...
 -  Domaine de l'**économie** : pour la gestion d'un supermarché, gestion de la facturation
 -  Domaine de l'**éducation** : pour la gestion des notes, gestion des emplois de temps, ...
 -  Domaine **administratif** : pour la gestion du personnel, gestion des matricules, ...
- La gestion de l'accès à une base de données est assurée par un logiciel spécialisé appelé SGBD.

III. LES SGBD

Un **système de gestion de base de données (SGBD)** est un logiciel conçu pour créer et gérer les bases de données. Il offre aux utilisateurs et aux programmeurs les outils nécessaires pour créer, modifier, supprimer les données dans la base de données.

I Les composants d'un SGBD

Un SGBD est composé de 03 langages : un langage de définition de données, un langage de manipulation de données et un langage de contrôle de données. Il permet alors de :

-  **La définition des données** (création des données) à travers un langage de définition de données (LDD) : le SGBD garantit la cohérence et la non redondance des données grâce aux différentes contraintes d'intégrité définies à niveau.
-  **La manipulation des données** (insertion, maj, consultation des données) à travers un langage de manipulation des données (LMD)

 **Le contrôle des données** : grâce au Langage de Contrôle de données qu'il intègre, le SGBD permet de définir les droits d'accès aux différents utilisateurs de la base de données, assurant ainsi la sécurité des données.

2 Exemples de SGBD

On distingue plusieurs SGBD parmi lesquels :

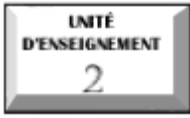
-  Microsoft Access
-  MySQL
-  MicrosoftSQL
-  Oracle
-  DB2
-  Sybase
-  PosgresSQL



EXERCICES

Exercice I

1. Définir base de données, SGDB
2. Quelle différence faites-vous entre une base de données et un fichier texte ordinaire ?
3. Citer 03 utilités d'une base de données tout en les expliquant clairement
4. Citer 05 exemples de SGDB
5. Décrire les 03 fonctions d'un SGBD
6. Quels sont les composants d'un SGBD ?



Les caractéristiques d'une table

Objectifs

- ☞ Décrire les éléments caractéristiques d'une table
- ☞ Décrire les contraintes d'intégrités

Activité

Après avoir pris en compte l'avis de plusieurs d'entre vous, votre censeur pense qu'il aura besoin d'une table ELEVE.

- Qu'est-ce qu'une table ?
- Citer quelques éléments caractéristiques de cette table
- Sachant qu'une contrainte d'intégrité est un ensemble de règles qui définit la cohérence d'une donnée ou d'un ensemble de données de la base de données, citer quelques contraintes d'intégrité que peuvent respecter la table ELEVE

I. ÉLÉMENTS CARACTÉRISTIQUES D'UNE TABLE

Une **table** ou **relation** est un ensemble de données organisées sous la forme d'un tableau à deux dimensions où les colonnes représentent les catégories d'informations et les lignes à des enregistrements.

Un **enregistrement** ou **ligne** est une ligne d'une table dans un modèle relationnel ; c'est une occurrence, un exemple d'une table.

Un **attribut** ou **colonne** est une propriété caractéristique d'une table, une colonne d'une table dans un modèle relationnel ;

Une **clé primaire** est un attribut (ou un ensemble d'attributs) permettant d'identifier de manière unique une table. Elle est généralement soulignée.

Une **clé étrangère** d'une table est un attribut qui est clé primaire dans une table et qui se retrouve dans une autre table. Elle est généralement précédée par le symbole « # »

Le tableau suivant est un exemple de schéma d'une relation :

Eleve			
Matricule	Nom	Prenom	Classe
19M001	SIMO	Sandrine	PC
19M002	OUAFO	Peguy	PA

 **Table** : Eleve(Matricule, Nom, Prenom, Classe) ;

 **Liste des attributs** : Matricule, Nom, Prenom, Classe ;

 **Liste des enregistrements** : (19M001, SIMO, Sandrine, PC) et (19M002, OUAFO, Peguy, PA) ;

 **Clé primaire** : Matricule ;

 **Clé étrangère** : aucune clé étrangère pour cette table

Pour assurer la cohérence dans les données d'une base de données, les SGBD définissent un ensemble de règles à respecter.

II. NOTION DE CONTRAINTES D'INTÉGRITÉ

Une **contrainte d'intégrité** est une règle qui définit la cohérence d'une donnée ou d'un ensemble de données de la base de données. Les contraintes d'intégrités peuvent s'appliquer à une colonne ou à une table entière. On distingue les contraintes suivantes : clé primaire, valeur autorisée, valeur nulle, valeur par défaut, unicité de valeur, format de valeur. Nous distinguons les contraintes d'intégrité suivantes :

 **Valeur nulle** (NULL ou NOT NULL) pour interdire ou autoriser l'insertion d'une valeur **null**, elle permet de spécifier si la valeur d'une colonne peut être vide ou pas

 **Valeur autorisée** (CHECK) : vérifie pendant l'insertion que la valeur de l'attribut satisfait la condition ;

 **Valeur par défaut** (DEFAULT) : pour spécifier la valeur par défaut des occurrences d'une colonne.

 **Unicité de valeur** (UNIQUE ou NOT UNIQUE) : pour spécifier que toutes les occurrences d'une colonne doivent être unique ou non ; UNIQUE interdit à 02 enregistrement d'avoir la même valeur pour cet attribut ;

 **Clé primaire** (PRIMARYKEY) définit l'attribut comme clé primaire : un attribut clé primaire satisfait 02 contraintes : UNIQUE et NOT NULL

 **Format de valeur**,

Le **format d'une donnée** représente le type, la nature des données à stockée ; le format d'une donnée peut être :

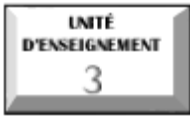
-  Entier (INTEGER) pour les données de types entiers ;
-  Réel (REAL) pour les données de types réel ;
-  Caractères (CHAR ou VARCHAR) pour les données de types chaîne de caractère à longueur fixe ;
-  Date (DATE) pour stocker les données de type date ;
-  Texte (TEXT) pour les chaînes de caractères de longueur variable ;



EXERCICES

Exercice I

1. Définir table, attribut, enregistrement, clé primaire, contrainte d'intégrité
2. Décrire les éléments caractéristiques d'une table
3. Soit la table « client », représentant un client d'un super marché.
 - Citer 03 propriétés de cette table
 - Quelle peut être la clé primaire ?
 - Donner 02 exemples de client
 - Définir 02 autres contraintes sur les colonnes de la table



Les fonctions d'une base de données

Objectifs

- ☑ Décrire les fonctions de manipulation d'une base de données
- ☑ Décrire les fonctions de définition d'une base de données

Activité :

Monsieur le censeur sait désormais qu'il aura besoin de la table ELEVE, et aimerai créer cette table.

- 📖 Que doit-il prendre en compte au moment de créer la table ?
- 📖 Quelles sont les opérations qu'il peut réaliser pour modifier la structure de la table ELEVE ?
- 📖 Quelles sont les opérations qu'il peut réaliser pour manipuler les données de la table ELEVE ?

En plus du contrôle de l'accès aux données, les SGBD permettent la définition et la manipulation des données.

I. LES FONCTIONS DE DÉFINITION D'UNE BASE DE DONNÉES

Grâce au langage de définition de données (LDD) qu'il intègre, le SGBD permet de définir la structure de la base de données, de créer et de supprimer les objets dans la BD (tables, contraintes, vue, ...); Il permet également de définir le format des données.

Parmi les fonctions de définition de données, on distingue :

- 📖 **Création de la base de données :**
- 📖 **Création d'une table :** pour créer une table, il faut préciser les différents éléments caractéristiques de cette table au moment de sa création et les différentes contraintes d'intégrité.

Exemple : table élève

ÉLÈVE				
<u>matricule</u>	Nom	Prénom	Age	Tel

ELEVE(matricule : entier, nom : caractère, prenom : texte, age : entier, tel : entier)

 **Ajout de colonne d'une table** : on ne peut ajouter qu'une colonne dans une table existante :

Exemple : ajout de l'attribut classe dans la table ELEVE

ÉLÈVE					
<u>matricule</u>	Nom	Prénom	Classe	Age	Tel

ELEVE(matricule : entier, nom : caractère, prenom : texte, Classe : caractère, age : entier, tel : entier)

 **Renommage des colonnes d'une table** : consiste à changer le nom d'une colonne ;

Exemple : modifier la colonne Age en date_naissance

ÉLÈVE					
<u>matricule</u>	Nom	Prénom	Classe	<u>date_naissance</u>	Tel

ELEVE(matricule : entier, nom : caractère, prenom : texte, Classe : caractère, date_naissance : entier, tel : entier)

 **Modification du type de donnée d'une colonne** : consiste à modifier le format des valeurs d'une colonne.

Exemple : Changer le type de « matricule », d'entier à caractère

ELEVE(matricule : caractère, nom : caractère, prenom : texte, Classe : caractère, date_naissance : entier, tel : entier)

 **Suppression des tables** : lorsqu'on supprime une table, elle n'existe plus et si elle contenait des données, elles sont perdues

 **Suppression de la base de données** : lorsqu'on supprime une base de données, elle n'existe plus et tous les données qu'elles sont perdues

II. LES FONCTIONS DE MANIPULATION D'UNE BASE DE DONNÉES

Grâce au langage de manipulation de données (LMD) qu'il intègre, le SGBD permet l'ajout, la suppression, et la modification des lignes, la visualisation du contenu des tables. Autrement dit, la fonction de manipulation inclut :

 **L'ajout ou l'insertion des données dans une table** : consiste à insérer une ligne dans une table.

Exemple : Ajouter dans la table ELEVE 02 exemples d'élèves

ÉLÈVE					
matricule	Nom	Prénom	Classe	date_naissance	Tel
09Y001	SIMO	Sandrine	PC	26-12-2005	
09Y002	OUAFO	Peguy	PA		

 **La suppression des données dans une table** : consiste à supprimer l'élève dont le nom est « OUAFO » de la table ELEVE

ÉLÈVE					
matricule	Nom	Prénom	Classe	date_naissance	Tel
09Y001	SIMO	Sandrine	PC	26-12-2005	

 **La modification ou mise à jour des données dans une table** : consiste à modifier une information contenue dans une table.

Exemple : modifier la classe de l'élève SIMO en « Tle C »

ÉLÈVE					
matricule	Nom	Prénom	Classe	date_naissance	Tel
09Y001	SIMO	Sandrine	Tle C	26-12-2005	

 **La visualisation ou consultation du contenu d'une table** : permet d'afficher le contenu d'une ou de plusieurs tables en fonction de certaines conditions.



EXERCICES

Exercice I

Soit la base de données suivante :

La base de données **Gestion_commande** est utilisée dans un super marché pour l'éditer des factures de ses clients et de pour gérer ses stocks. Le schéma de la base de données client :

Client(numClt, nom, adresse, ville, compte)

Produit(numPro, libelle, prix, qteStock)

Commande(numCmde, numClt, dateCmde)

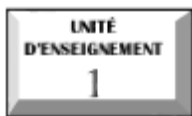
Détail(numPro, numPro, qteCmde)

1. Combien de table compte cette BD ? décrive chacune de ces tables en ressortant la clé primaire, le type de chaque attribut, ...
2. Citer 04 contraintes d'intégrité que doivent respecter certains attributs de ses tables ;
3. Donner 01 exemple de client, de produit, de commande et de détail de commande
4. Modifier la structure d'une table client en y ajoutant l'attribut « Catégorie » ;
5. Renommer une table « Détail » en « détail_Commande » ;



UTILISER LES STRUCTURES ALGORITHMIQUES

-  U.E 1 : Les structures itératives
-  U.E 2 : Les structures conditionnelles
-  U.E 3 : Généralités sur les structures de données
-  U.E 4 : Les tableaux



Les structures itératives

OPO

- ☑ Définir structures itératives
- ☑ Donner la syntaxe des différentes structures itératives.

Situation problème :

Lors d'une course de résistance, votre enseignant vous demande d'effectuer 4 tours du stade de jeux votre lycée. Pour cela il se dote d'un chronomètre pour vérifier le temps mis et note à chaque fois le nombre de tours faits sur un tableau (1, 2, 3 ...).

Mettez-vous en groupe de trois, et répondez aux questions suivantes :

- 1- Quelle est l'action qui sera à chaque fois répétée ?
- 2- Pour qu'elle condition doit t'on arrêter de répéter l'action identifiée plus haut ?
- 3- Compléter la phrase suivante :
Tant que nous continuerons à courir.
- 4- Quelles sont les structures de contrôles mises en exergue dans les questions précédentes ?
Le compteur du nombre de tours augmente chaque fois d'une unité (1, 2, 3)
- 5- A structure de contrôle cela renvoie t'il ?

INTRODUCTION

Les opérations élémentaires relatives à la résolution d'un problème, peuvent en fonction de leur enchainement, être organisées suivant une famille de structures algorithmiques fondamentales. nous pouvons citer :

- 📁 Les structures séquentielles ;
- 📁 Les structures alternatives ;
- 📁 Les structures de choix ;
- 📁 Les structures itératives ou répétitives

Nous allons dans le cadre de cette leçon, nous appesantir sur les structures itératives.

Il s'agit de la structure **pour, tant que - faire et répéter jusqu'à**.

I- RAPPEL

La structure itérative ou boucle répète l'exécution d'une instruction ou d'un ensemble d'instructions. Sa forme dépend de la connaissance qu'on a du nombre de répétition de l'action. On distingue deux cas.

I. Le nombre de répétitions n'est pas connu ou est variable

On distingue deux structures de bases :

 La structure RÉPÉTER JUSQU'A

 La structure TANT QUE FAIRE

2- le nombre de répétitions est connu ou invariable

On utilise la structure POUR

II-LA STRUCTURE REPETER...JUSQU'A

Dans la structure REPETER ...JUSQU'A, l'action est exécutée une première fois puis sa répétition se poursuit jusqu'à ce que la condition soit vérifiée. Elle se présente sous la forme suivante :

SYNTAXE:

Répéter

Action ;

Jusqu'à condition vrai

III- LA STRUCTURE TANT QUE...FAIRE

SYNTAXE:

TANT QUE condition Faire

Action ;

FIN TANT QUE

Exemple :

I=I ;

TANT QUE $i < 5$ FAIRE

AFFICHER (" bonjour à tous") ;

FIN TANT QUE

IV- LA STRUCTURE POUR SYNTAXE:

POUR **indice** ALLANT DE **v1** à **v2** PAS **n** FAIRE
ACTION;

FIN POUR

Où **indice** représente le compteur et s'incrémente en fonction du pas.

Exemple Pour i allant de 0 à 9 Pas de 1 faire
Afficher(i);
Fin pour



EXERCICES

Exercice 1

- 1) Quelle différence y a-t-il entre les boucles REPETER et TANT QUE ?
- 2) Quels sont les données indispensables à l'utilisation de la boucle POUR ?
- 3) Donner les syntaxes des boucles POUR, TANT QUE et REPETER.

Exercice 2

Écrire l'algorithme permettant d'afficher à l'écran tous les nombres impairs inférieurs à 200. Utiliser la boucle POUR et TANT QUE.

Exercice 3

Que produit l'algorithme suivant :

algorithme

Variable N :Entier

Debut

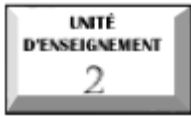
$N \leftarrow 0$;

Repeter

$N \leftarrow N + 3$;

Jusqu'à $(N > 45)$ vrai

Fin



Les structures alternatives

OPO

- ☞ Énumérer les structures conditionnelles qui existent
- ☞ Utiliser les structures conditionnelles pour résoudre des problèmes.

I. LES STRUCTURES ALTERNATIVES

L'instruction si permet de programmer une structure dite de choix (ou alternative), permettant de choisir entre deux instructions ou plus, suivant la valeur d'une expression jouant le rôle de condition. La seconde partie, introduite par le mot clé SINON, est facultative, de sorte que l'instruction SI présente deux formes. On distingue deux types de Structures alternatives :

I. La structure alternative complète

Dans cette structure l'exécution d'un des deux traitements distincts ne dépend que du résultat d'un test effectué sur la condition qui peut être une variable ou un événement.

Sa syntaxe est la suivante :

Si (condition) alors

action1 ;

Sinon

action2 ;

Fin si

Ceci signifie que :

- Le test de la condition est effectué
- Si le test est positif, action1 est exécuté
- Si le test est négatif, action2 est exécuté
- Sortir de la structure.

II. LA STRUCTURE ALTERNATIVE RÉDUITE CONDITIONNELLE SIMPLE

La structure alternative réduite se distingue de la précédente par le fait que seule la situation correspondant à la validation de la condition entraîne l'exécution du traitement, l'autre situation conduisant systématiquement à la sortie de la structure.

Sa syntaxe est la suivante :

Si (condition) alors

Action ;

Fin si

Ceci signifie que :

Le test de la condition est effectué

Si le test est positif, action est exécutée

Sortir de la structure



EXERCICES

Exercice 1

Votre papa décide de vous primer par rapport à la fin d'année selon votre mention.

Pour ce faire il vous impose les conditions suivantes :

- Pour la mention p (passable) il vous achète des bonbons ;
- Pour la mention AB il vous offre un vélo ;
- Pour la mention B Il vous offre une console de jeux ;
- Pour la mention TB il vous offre un billet d'avion pour Paris.

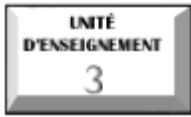
Écrire un algorithme qui permet d'effectuer cette opération en utilisant :

- Les structures de choix ;
- Les structures conditionnelles.

On utilisera les opérations primitives suivantes : acheter_bonbon, offrir_vélo, offrir_console, offrir_billet pour les opérations respectives décrites plus haut

Exercice 2

Écrire l'algorithme qui prend en entrée deux nombres et affiche le plus grand.



Généralités sur les structures de données

OPO

- ① Définir structures de données ;
- ② Énumérer les différentes structures de données ;

INTRODUCTION

En informatique, une structure de données est une structure logique destinée à contenir des données, afin de leur donner une organisation permettant de simplifier leur traitement. Différentes structures de données existent pour des données différentes ou répondant à des contraintes algorithmiques différentes. Parmi celles-ci nous pouvons citer des tableaux, des enregistrements, des listes des piles...

Dans ce cours nous allons présenter de manière sommaire quelques unes.

I- ENREGISTREMENTS

Les enregistrements sont des structures de données dont les éléments peuvent être de type différent. Pour créer des enregistrements, il faut déclarer un nouveau type (type structuré), basé sur d'autres types existants.

Après avoir défini un type structuré, on peut l'utiliser comme un type normal en déclarant une ou plusieurs variables de ce type.

Les variables de type structuré sont appelées enregistrements.

Les éléments qui composent un enregistrement sont appelés champs

Sa déclaration est la suivante :

Type

Structure nom_type

nom_champI: type_champI

...

nom_champN: type_champN

FinStruct

Exemple

Type

Structure Tpersonne

nom : chaîne

adresse : chaîne

âge : entier

FinStruct

II-LISTES

Les listes chaînées sont des structures de données semblables aux tableaux sauf que l'accès à un élément par un pointeur au lieu d'indice. Chaque élément étant repéré par ses voisins auxquels il est relié.

Une liste étant complètement définie par :

-  tete : Pointeur vers le premier élément
-  Suivant : chaque élément possède un pointeur vers un autre nœud
-  Valeur : valeur(s) de la donnée (des données) d'un nœud
-  Fin : Le pointeur suivant du dernier élément doit pointer vers NULL (la fin de la liste)

Sa déclaration se fait de la manière suivante :

```
typedef struct noeud
{
    char *nom;
    int num ;
    struct noeud * suivant;
} Noeud;
Noeud * tete;
Tete=NULL ;
```

III- PILES

La pile est une structure de données, qui permet de stocker les données dans l'ordre LIFO (Last In First Out) - en français Dernier Entré Premier Sorti).

Une pile a un seul point d'accès, les données sont ajoutées ou retranchées par l'intermédiaire de la tête d'accès appelé sommet de la pile.

Les opérations caractéristiques d'une pile sont empiler pour ajouter un élément et dépiler pour retirer un élément.

Une pile est une liste sur laquelle on autorise opérations:

-  Accès au sommet de la pile
-  Tester si la pile est vide
-  Empiler un élément, le mettre au sommet de la pile
-  Dépiler un élément (par le sommet)

Pour représenter un élément de la pile, il suffit de reprendre la structure d'un élément d'une liste doublement chaînée.

```
struct element {  
    int donnee;  
    struct pile *next;  
};  
typedef struct element Element;
```

IV- LES TABLEAUX

Il s'agit d'une structure contenant les éléments de même type. Ceci fera l'objet d'une autre leçon.

CONCLUSION

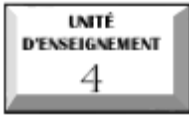
Au terme de notre leçon nous avons défini la notion de structure de données, énuméré et décrit les structures telles que les enregistrements, listes et piles. Quand aux tableaux ils feront l'objet du prochain cours.



EXERCICES

Exercice 1

Exercice 2



Les tableaux

OPO

-  Déclarer un tableau à une dimension ;
-  Parcourir un tableau pour effectuer la lecture, l'affichage, la recherche ;
-  Exécuter un algorithme simple utilisant les tableaux.

Situation problème :

Au cours d'informatique l'enseignant demande d'écrire un algorithme permettant de saisir sept notes et de les afficher après avoir multiplié toutes les notes par 2.

Voici notre solution :

Algorithme Note

Variables N1, N2, N3, N4, N5, N6, N7 : Réel

Début

Ecrire ("Entrer la valeur de la 1er note")

Lire (N1)

Ecrire ("Entrer la valeur de la 2eme note")

Lire (N2)

Ecrire ("Entrer la valeur de la 3eme note")

Lire (N3)

Ecrire ("Entrer la valeur de la 1er note")

Lire (N4)

Ecrire ("Entrer la valeur de la 1er note")

Lire (N5)

Ecrire ("Entrer la valeur de la 1er note")

Lire (N6)

Ecrire ("Entrer la valeur de la 1er note")

Lire (N7)

Ecrire ("La note 1 multipliée par 2 est : ", N1 *2)

Ecrire ("La note 2 multipliée par 2 est : ", N2 *2)

Ecrire ("La note 3 multipliée par 2 est : ", N3 *2)

Ecrire ("La note 4 multipliée par 2 est : ", N4 *2)

Ecrire ("La note 5 multipliée par 2 est : ", N5 *2)

Ecrire ("La note 6 multipliée par 2 est : ", N6 *2)

Ecrire ("La note 7 multipliée par 2 est : ", N7 *2)

Fin

Nous allons remarquer que la même instruction se répète sept fois. Imaginons que si l'on voudrait réaliser cet algorithme avec une centaine de notes, cela deviendrait fastidieux.

Étant donné que les variables portent des noms différents on ne peut pas utiliser de boucle, quel type de données pourrions-nous utiliser pour résoudre ce problème ?

INTRODUCTION

En anglais **Array**, un tableau est une suite d'éléments de même type. Il utilise plusieurs cases mémoire à l'aide d'un seul nom. Comme toutes les cases portent le même nom, elles se différencient par un numéro ou un indice. Dans ce cours on verra comment déclarer un tableau et l'utiliser pour exécuter des algorithmes.

I- DÉCLARATION D'UN TABLEAU A UNE DIMENSION

Un tableau est une structure de donnée T qui permet de stocker un certain nombre d'éléments T [i] repérés par un index i. Les tableaux vérifient généralement les propriétés suivantes :

-  tous les éléments ont le même type de base ;
-  le nombre d'éléments stockés est fixé.

La déclaration d'un tableau permet d'associer à un nom une zone mémoire composée d'un certain nombre de cases mémoires de même type.

Syntaxe : Variable identificateur : tableau [indice_min.. indice_max] de type

Ou bien Variable identificateur : tableau [taille] de type

La déclaration des tableaux est susceptible de varier d'un langage à l'autre. Certains langages réclament le nombre d'éléments, d'autre le plus grand indice...

Le premier élément d'un tableau porte l'indice zéro ou l'indice I selon les langages

La valeur d'un indice doit être un nombre entier inférieure ou égale au nombre d'éléments du tableau.

Exemple I : L'instruction suivante déclare un tableau de 100 éléments de type réel :

Variable Note : tableau [I.. 100] de Réels

Note : c'est le nom du tableau (identificateur)

I : c'est l'indice du premier élément du tableau.

100 : c'est l'indice du dernier élément du tableau (nombre d'éléments du tableau).

Exemple2 : l'instruction suivante permet de déclarer un tableau de 12 éléments de chaîne de caractères :

Variable T : tableau [11] de chaînes de caractères

T : c'est le nom du tableau (identificateur)

11 : c'est le plus grand indice de ce tableau de 12 éléments. On à considérer que le premier indice du tableau est 0.

Exercice d'application : Écrire l'instruction qui permet de déclarer un tableau contenant les 6 voyelles de l'alphabet.

II-LES OPÉRATIONS DANS UN TABLEAU

On peut créer des tableaux contenant des variables de tous types : tableaux de numériques, bien sûr, mais aussi tableaux de caractères, tableaux de booléens, tableaux de tout ce qui existe dans un langage donné comme type de variables. Par contre, hormis dans quelques rares langages, on ne peut pas faire un mixage de types différents de valeurs au sein d'un même tableau.

Les éléments d'un tableau sont des variables indicées qui s'utilisent exactement comme n'importe quelles autres variables classiques. Elles peuvent faire l'objet d'une affectation, elles peuvent figurer dans une expression arithmétique, dans une comparaison, elles peuvent être affichées et saisies etc.

L'utilisation de ces éléments se fait en suite, via le nom du tableau et son indice. Ce dernier peut être soit une valeur (exemple : Note [3]), soit une variable (exemple : Note[i]) ou encore une expression (exemple : Note [i+1]).

Exemple 1 : opération d'affectation

On peut affecter à une variable un élément du tableau et la syntaxe pour le faire est la suivante : $\text{nom_variable} \leftarrow \text{nom_tableau} [\text{indice de l'élément}]$

L'instruction suivante affecte à la variable X la valeur du premier élément du tableau Note :

$X \leftarrow \text{Note} [1]$

Exemple 2 : opération d'affichage

On peut afficher le contenu d'un élément du tableau et la syntaxe pour le faire est la suivante : $\text{Ecrire ou Afficher} (\text{nom_tableau} [\text{indice de l'élément}])$

L'instruction suivante affiche le contenu du quatrième élément du tableau Note :

$\text{Afficher} (\text{Note} [4])$

Exemple 3 : opération de lecture

On peut récupérer et affecter une valeur introduite par l'utilisateur à un élément du tableau et la syntaxe pour le faire est la suivante :

$\text{Lire ou Saisir} (\text{nom_tableau} [\text{indice de l'élément}])$

L'instruction suivante affecte une valeur introduite par l'utilisateur à l'élément trois du tableau Note :

$\text{Saisir} (\text{Note} [3]).$

Parcours complet d'un tableau Le fait que les éléments d'un tableau soient indicés permet de parcourir tous ces éléments avec une boucle, en utilisant une variable qui sert d'indice et s'incrémente à chaque tour de boucle.

III-UTILISATION DES TABLEAUX POUR EXÉCUTER DES ALGORITHMES

Exercice 1 :

Écrire un algorithme permettant de saisir 20 notes et de les afficher après avoir multiplié toutes ces notes par un coefficient fourni par l'utilisateur :

Solution :

Algorithme tableau_note

Variable Note : tableau[19] de Réels

Coef, i : entier

Début

Ecrire ("Entrer le coefficient")

Lire (Coef)

//Remplissage du tableau Note

Pour i Allant de 0 à 19 Faire

Ecrire ("Entrer la valeur de la note", i)

Lire (Note[i])

Fin Pour

*//Affichage des notes * Coef*

Pour i Allant de 0 à 19 Faire

Affiche(Note[i] * Coef

Fin Pour

Fin

Exercice 2 :

Écrire un algorithme qui déclare un tableau de 12 notes, dont on fait ensuite saisir les valeurs par l'utilisateur, calcul la moyenne des notes et affiche à l'écran.

Solution :

Algorithme tableau_note

Variable Tab : tableau[8] de Réels

S, i : entier

Début

$S \leftarrow 0$

Pour i allant de 0 à 8 faire

 Ecrire ("Entrez la note numéro ", $i + 1$)

 Lire (Tab [i])

$S \leftarrow S + \text{Tab}[i]$

Fin pour

Ecrire ("Moyenne :", $S / 9$)

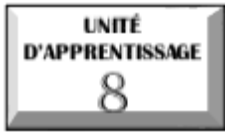
Fin



EXERCICES

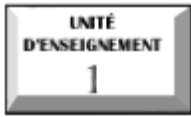
Exercice 1

Exercice 2



UTILISER LES FONCTIONS ET PROCÉDURES

🖱 U.E I : Notions de procédure et fonction



Notions de procédure et fonction

Compétences visées :

- ☞ Identifier et déclarer une procédure et une fonction.
- ☞ Savoir passer des paramètres à une procédure et une fonction.
- ☞ Exécuter un algorithme en utilisant une procédure et une fonction.

Situation problème

Votre professeur titulaire fait appel à vous pour l'aider dans le calcul de moyenne de chaque élève de votre salle de classe. Pour ce faire, il opte pour une solution algorithmique et vous devez l'aider dans l'élaboration de cette solution. L'intitulé de chaque matière et son coefficient sont connus. Lors d'une exécution, l'utilisateur devra donc entrer le nom de l'élève, pour chaque matière entrer sa note ; l'algorithme affichera à l'écran la note finale de chaque matière, la note totale et la moyenne.

Quelle méthode peut-on prendre pour calculer la note finale de tous les élèves sans avoir à reprendre la même séquence d'instructions pour chaque matière ?

INTRODUCTION

Un algorithme est une séquence ordonnée et finie d'instructions dont une exécution correcte permet de résoudre un problème spécifique appartenant à une classe de problèmes connus. Cette séquence d'instructions peut être constituée de plusieurs sous-séquences d'instructions (les blocs). Dès lors, on peut constater plusieurs répétitions d'un même bloc. Le corps de l'algorithme est donc très vaste, la détection d'une erreur sur un bloc répétitif nécessite la correction sur tous les blocs qui se ressemblent.

Pour contourner ces problèmes générés par les séquences d'instructions qui se répètent, on utilise les procédures et les fonctions.

I. LES PROCEDURES

Une procédure est un sous-algorithme assurant de manière autonome la résolution d'une tâche particulière. L'algorithme principal ou un autre sous-algorithme peut utiliser à plusieurs reprises cette tâche : il doit juste faire un appel de la procédure.

La notion de procédure comporte deux aspects :

-  **Déclaration de la procédure** : c'est la définition de la procédure ;
-  **Appel de la procédure** : c'est l'utilisation de la procédure.

Une grosse procédure peut être décomposée en plusieurs petites procédures. La structure générale d'une procédure n'est pas très différentes de celle d'un algorithme normal : elle comporte une tête, une partie déclaration et un corps. La tête de la procédure commence par le mot réservé **procédure** suivi du nom de la procédure et d'une liste de paramètres placée entre parenthèses. Les paramètres assurent la communication entre les différents modules d'un algorithme.

La syntaxe d'une procédure est la suivante :

Procédure nom_de_la_procedure (paramètre1:type I; paramètreN: type n)

Déclaration des variables et constantes

Début

/*Corps de la procédure*/

Finprocédure

1 Paramètre d'une procédure

Un paramètre est une variable d'entrée de la procédure. Ces paramètres vont permettre l'appel de cette procédure dans une procédure principale ou appelante. La déclaration des paramètres est similaire à celle des variables dans l'algorithme principal

2 Déclaration et appel d'une procédure

La déclaration d'une procédure est faite dans la zone de déclarations des fonctions et procédures.

Lors de l'appel d'une procédure les paramètres s'appellent les arguments ; un certain nombre de règles doivent être respectées :

-  Les arguments doivent apparaître dans l'ordre choisi pendant la déclaration.
-  Les arguments doivent être du même type que les paramètres qu'ils représentent.

Exemple : Ecrire une Procédure qui renvoie la somme deux nombres et l'appeler dans un algorithme.

procédure addition (a : Réel, b : Réel)

Var S : Réel; /* le résultat qu'on va retourner*/

début

S ← a + b ;

afficher ("La somme est égale à :", S);

fin

Appelons notre procédure dans un algorithme.

Algorithme exemple

Var x, y, P: Réel;

Procédure addition ;

Début

Ecrire ("Veuillez saisir deux nombres ")

;

Lire (x); **Lire** (y);

addition (x, y);

Fin

II. LES FONCTIONS

Une **fonction** est un sous-programme qui effectue une tâche ou un calcul et qui renvoie ou retourne un résultat. Dans la plupart des cas, elle contient des paramètres (informations qui seront récupéré par la **fonction** et qui serviront à l'exécution de la tâche ou du calcul.

I La syntaxe d'une fonction est la suivante :

Fonction nom_de_la_fonction (paramètreI, ..., paramètreN) :Type valeur de retour

Déclaration des variables et constantes

Début

/*Corps de la de la fonction*/

retourner /*nom d'une variable*/ ;

FinFonction

Ainsi l'objectif principal d'une fonction est de calculer et retourner une valeur à l'algorithme appelant. Les paramètres et l'appel d'une fonction obéissent aux mêmes règles que ceux d'une procédure. Cependant, au moment de l'appel, la fonction peut apparaître dans la partie droite d'une affectation ; la fonction est considérée comme une valeur du même type que sa valeur de retour.

2 Syntaxe d'appel d'une fonction

Une fonction retourne toujours une information au programme appelant, c'est pourquoi l'appel de la fonction se fait à l'aide de l'instruction d'affectation :

Variable ← Nom Fonction (Var1, Var2...);

Exemple : *Ecrire une fonction qui renvoie la somme deux nombres et l'appeler dans un algorithme.*

<u>fonction</u> addition(a : Réel, b : Réel) : Réel <u>Var</u> S : Réel; /* le résultat qu'on va retourner*/ <u>début</u> S ← a + b; retourner S; <u>fin</u>	Appelons notre fonction dans un algorithme. <u>Algorithme</u> exemple <u>Var</u> x, y, P: Réel; <u>Fonctions</u> addition; <u>Début</u> Ecrire ("Veuillez saisir deux nombres"); Lire(x); Lire(y); P ← addition(x, y); Ecrire ("La somme vaut : ", P); <u>Fin</u>
--	---

REMARQUE:

Il faut bien faire la différence par la suite entre une **Variable locale** (variable privée) et **variable globale** (variable publique). En Algorithmique, une **variable locale** est une **variable** qui ne peut être utilisée que dans la fonction / procédure où elle est définie. La **variable locale** s'oppose à la **variable globale** qui peut être utilisée dans tout le programme. La variable locale n'a donc de visibilité qu'à l'intérieur de la procédure / fonction.

 Les noms des procédures et fonctions respectent les mêmes règles que celles des noms des algorithmes.

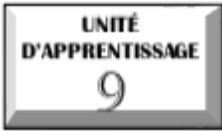
CONCLUSION

Les procédures et fonctions permettent de : réduire la taille des algorithmes, faciliter la lecture des algorithmes, faciliter la correction des erreurs, Une procédure est une fonction qui ne retourne pas de valeur. Pour utiliser une procédure ou une fonction, on doit la définir au préalable puis l'appeler chaque fois que le besoin s'impose. L'ordre, le type et le nombre des paramètres doivent à chaque appel d'une procédure ou d'une fonction être cohérent à sa déclaration. Dans une fonction récursive, on doit vérifier si la condition d'arrêt est possible pour n'importe quelle valeur des paramètres.

EXERCICES

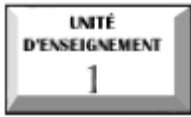
Exercice I

- 1) Donner l'avantage des procédures et fonctions, les critères à respecter lors d'un appel.
- 2) Qu'elle est la différence fondamentale entre une fonction et une procédure ?
- 3) Écrire une fonction qui à base des notes et coefficient, retourne la moyenne d'un élève
- 4) Écrire une procédure qui permet à un utilisateur de saisir la note pour une matière



PROGRAMMER EN HTML

-  U.E 1 : Insérer les fichiers multimédias dans une page web
-  U.E 2 : Insérer des tableaux et formulaires en HTML
-  U.E 3 : Organiser les dossiers du site et mise en place du CSS



Insérer les fichiers multimédias dans une page web

Compétences :

- ☞ Insérer les fichiers sons, vidéo, images dans une page web

Situation problème :

Lors de la création de votre site web, vous voulez réserver une page pour le téléchargement des musiques, des vidéos et photos à vos visiteurs. Comment allez-vous insérer ces éléments dans votre page ?

I. INSÉRER UNE IMAGE EN HTML

Il fallait utiliser la balise `<img>` pour insérer une image dans un fichier HTML.

La balise `<img>` est représenté par une balise orpheline et possède deux attributs obligatoires : l'attribut **src**, qui sert à indiquer le chemin (relatif ou absolu) de l'image, et l'attribut **alt** qui sert à décrire l'image, dans le cas où l'utilisateur ne pourrait pas la voir.

Commençons donc par placer une image dans le même dossier que notre fichier HTML et essayons de l'afficher. Ici l'image s'appelle « mon-lycee.png ».

```

```

II. INSÉRER UN SON EN HTML

Ici on utilise la balise `<audio>` qui possède un attribut obligatoire : l'attribut **controls** qui ne possède qu'une valeur (que nous ne sommes donc pas strictement obligés de préciser car celle-ci est implicite). Celui-ci va tout simplement afficher les contrôles de base (lecture, pause, retour au début et volume) de notre piste audio.

Afin de laisser le choix au navigateur d'utiliser le fichier qu'il souhaite, nous allons utiliser en plus de la balise `<audio>` une balise `<source>` (représenté sous forme de balise orpheline) avec son attribut **src**.

```
<audio controls>
  <source src="musique.mp3">
</audio>
```

III. INSÉRER UNE VIDÉO EN HTML

La balise `<video>` va s'utiliser exactement comme la balise `<audio>`. De plus, il possède les mêmes attributs que ce dernier à savoir : Un attribut **controls** pour ajouter les contrôles de base de la vidéo (lecture, arrêt, etc.) ; une balise `<source>` précisant l'adresse de la vidéo

```
<video controls>
  <source src="video.mp4">
</video>
```

EXERCICES

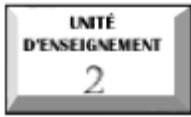
Exercice 1

Exercice 2

Exercice 3

Exercice 4

Exercice 5



Insérer les tableaux et formulaires en HTML

Compétences :

- ☑ Insérer un tableau dans une page web
- ☑ Créer un formulaire

Situation problème :

Le club informatique de votre lycée veut permettre à chaque candidat à la coopérative de s'inscrire dans le site web du lycée en renseignant ses informations personnelles (nom, prénom, âge, ...) dans une page web pour les organiser plutard et les afficher en lignes et colonnes. Quels sont les éléments permettent de recueillir les informations des différents candidats ? Que peut-on utiliser pour organiser les données sous forme de lignes et colonnes ?

I. LES TABLEAUX

Un tableau qu'est-ce que c'est ? Tout simplement une suite de lignes et de colonnes qui forment un ensemble de cellules. Les tableaux servent avant tout à présenter des données tabulaires. Un tableau est délimité par les balises `<table>` `</table>`. Le contenu d'un tableau HTML est généré horizontalement. C'est-à-dire qu'il n'est pas créé colonne par colonne mais ligne par ligne. Pour créer un tableau, nous commençons par créer des lignes, puis nous insérons dans ces lignes les cellules du tableau. Le contenu (textes, images, autres balises) se trouve uniquement dans les cellules et ne doit se trouver en aucun cas en dehors.

Les balises utiles

Par défaut, les bordures ne sont pas visible, il faut rajouter l'attribut **border** à la balise `<table>` pour les faire apparaître. On peut évidemment épaissir les bordures en augmentant la valeur.

Donc pour construire un tableau, nous avons besoin des balises suivantes :

- `<table>` `</table>`. C'est cette balise qui permet d'indiquer le début et la fin

d'un tableau.

- `<caption> </caption>` : Titre du tableau
- `<th> </th>` : En-tête du tableau
- `<tr> </tr>` : indique le début et la fin d'une ligne du tableau ;
- `<td> </td>` : indique le début et la fin du contenu d'une cellule.

Exemple :

```
< table border= '1'>
  < caption >Passagers du vol 377 </ caption >
< tr >
  < th >Nom </ th >
  < th >Âge </ th >
  < th >Pays </ th >
</ tr >
< tr >
  < td >Carmen </ td >
  < td >33 ans </ td >
  < td >Espagne </ td >
</ tr >
< tr >
  < td >Michelle </ td >
  < td >26 ans </ td >
  < td >États - Unis </ td >
</ tr >
</ table >
```

Passagers du vol 377

Nom	Âge	Pays
Carmen	33 ans	Espagne
Michelle	26 ans	États - Unis

II. LES FORMULAIRES

Toute page HTML peut être enrichie de formulaires interactifs, qui invitent vos visiteurs à renseigner des informations : saisir du texte, sélectionner des options, valider avec un bouton . . . tout est possible ! Nous arrivons cependant aux limites du langage HTML car il faut ensuite pouvoir analyser les informations que le visiteur a saisies. ... et cela ne peut pas se faire en langage HTML. Comme nous allons le voir, le traitement des résultats doit s'effectuer dans un autre langage, par exemple le PHP.

I. Déclaration d'un formulaire

La balise `<form>` `</form>` est la balise principale du formulaire, elle permet d'en indiquer le début et la fin. Comment envoyer le texte saisi par le visiteur ? Une fois que les données ont été envoyées, comment les traiter ?

Pour fournir les réponses à ces questions, vous devez ajouter deux attributs à la balise `<form>` :

▪ **method** : cet attribut indique par quel moyen les données vont être envoyées (réponse à la question no 1). Il existe deux solutions pour envoyer des données sur le Web :

✓ **method="get"** : c'est une méthode en général assez peu adaptée car elle est limitée à 255 caractères. La particularité vient du fait que les informations seront envoyées dans l'adresse de la page (`http://...`).

✓ **method="post"** : c'est la méthode la plus utilisée pour les formulaires car elle permet d'envoyer un grand nombre d'informations. Les données saisies dans le formulaire ne transitent pas par la barre d'adresse.

▪ **action** : c'est l'adresse de la page ou du programme qui va traiter les informations (réponse à la question no 2).

Cette page se chargera de vous envoyer un e-mail avec le message si c'est ce que vous voulez, ou bien d'enregistrer le message avec tous les autres dans une base de données. Cela ne peut pas se faire en HTML et CSS, on utilisera en général un autre langage tel que PHP.

On aura alors comme exemple de déclaration de formulaire le code suivant :

```
< p >Texte avant le formulaire </ p >
  < form method = " post " action = " traitement . php " >
    < p >Texte à l ' int é rieur du formulaire </ p >
  </ form>
< p >Texte après le formulaire </ p >
```

2. Les éléments de formulaires

Les éléments de formulaires sont répartis en 3 classes :

- **Input** : Champs de saisie de texte et différents types de boutons
- **Select** : Listes (menus déroulants et ascenseurs)

- **Textarea** : Zone de saisie de texte libre

Chaque élément doit être nommé, pour cela on utilise l'attribut **NAME**. Le nom ainsi défini permet d'identifier les données lors de leur envoi. Il doit être unique, sauf dans le cas d'un ensemble composé de plusieurs éléments (boutons radio par exemple). Un autre attribut **VALUE** est commun à ces balises, mais son rôle est différent selon le type d'élément utilisé.

Les éléments input

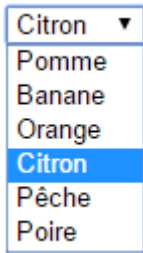
Cet élément est utilisé pour définir des zones d'entrée de texte simple, des boutons, des cases à cocher ou des boutons radio. Le type d'élément dont il s'agit sera précisé en utilisant l'attribut **TYPE**

Type	Syntaxe
text	<code><input type="text"/></code>
(avec valeur par défaut)	<code><input type="text" value="par défaut" /></code>
password	<code><input type="password" name="pass"/></code>
date	<code><input type="date" name="date"/></code>
range	<code><input type="range" /></code>
email	<code><input type="email" /></code>
submit	<code><input type="submit" value="Envoyer"/></code>
reset	<code><input type="reset" value="Effacer"/></code>
Checkbox	<code><input type="checkbox" name="pfm" value="linux" checked/> Linux
</code> <code><input type="checkbox" name="pfm" value="dos"/> Dos
</code> <code><input type="checkbox" name="pfm" value="win"/> Windows</code>
Radio	<code><input type="radio" name="media" value="cd" checked/> CD-ROM
</code> <code><input type="radio" name="media" value="dk"/> Disquette</code>

Remarque : Pour le type “radio”, les attributs “name” doivent porter la même valeur sinon il y aura possibilité de choisir plusieurs valeurs comme pour les cases à cocher

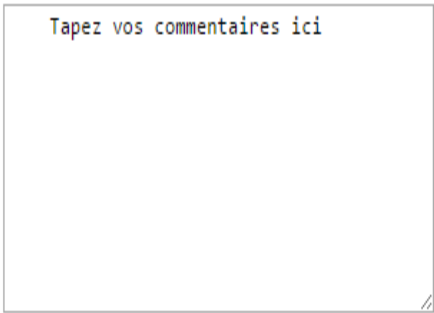
L'élément SELECT

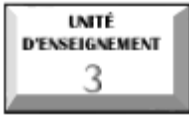
Cet élément sert à définir des listes (menus déroulant ou ascenseurs). Elle s'utilise avec la balise **OPTION**.

Exemple	résultat
<pre><select name="menu"> <option> Pomme </option> <option> Banane</option> <option> Orange</option> <option selected> Citron </option> <option> Pêche </option> <option> Poire </option> </select ></pre>	

L'élément TEXTAREA

Permet de créer une zone de texte en spécifiant sa taille grâce aux attributs **ROWS** et **COLS**.

Exemple	Résutat
<pre><textarea name="comm" rows=10 cols=40> Tapez vos commentaires ici </textarea></pre>	



Organiser les dossiers du site et mise en place du CSS

Compétences visées :

- ☞ Créer un dossier principal et des sous-dossiers en fonction de fichiers utilisés et des thèmes traités
- ☞ Regrouper les fichiers à utiliser en fonction de leur nature et en fonction du thème ou des parties du site
- ☞ Créer un fichier de style externe

Problème : Le club informatique de votre établissement lance le projet de développement de son site Web, ce site doit entre-autre faire une présentation générale de l'établissement mais aussi les activités des autres clubs tels que club journal, club anglais, club santé... Chaque club dans le site web doit avoir un espace pour la publication de ses activités et ses éventuelles photos et fichiers.

Proposez une organisation structurée de ce site web ainsi que les fichiers présents de ce site

Solution

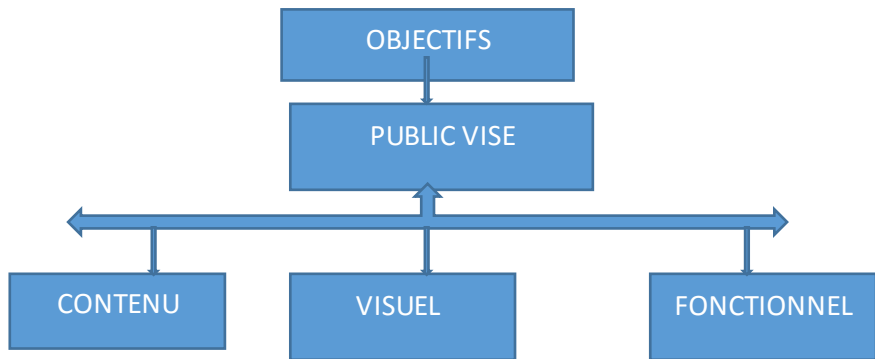
On aura un dossier global qui représente le site web

Dans ce dossier on va créer un dossier pour chaque club et chacun de ces dossiers contiendra un dossier pour les images, un dossier pour les fichiers audio, un dossier pour les vidéos...

I- CONSTRUCTION DU SITE WEB

Réaliser une page d'accueil html avec un menu permettant d'accéder aux différentes parties qui constituent le site web. Chacune de ces parties représente un dossier

De façon générale, voici comment s'organise un site web :



II- INSERTION DU CSS DANS UNE PAGE HTM

Le **CSS** (Cascading Style Sheet) est un langage permettant la mise en forme des pages Web. Le css agit sur des éléments de langage html nettement sur des balises qui représentent des sélecteurs.

Dans un document html, on peut insérer le code css soit :

Dans une balise html sous forme **d'attribut**

syntaxe : <balise style="propriété : valeur">

Exemple: <p style="color :blue">

Dans l'entête de document html : entre les balises <head>.....</head> inserer la balise <style type="text/css" >

Saisir le code css ici

</style>

Dans un fichier de style externe .css

Dans les deux derniers cas, on utilise la syntaxe générale du css qui est la suivante :

Sélecteur {

Propriété : valeur ;}

Exemple

p{

font-size:12px;

color :blue ;

}

Pour faire agir le css sur le html, il faut ajouter à l'entête du fichier html la balise
`<link rel="stylesheet" href="chemin/fichier.css" type="text/css">` l'attribut **href**
 permet d'indiquer l'emplacement du fichier css dans l'ordinateur

EXERCICES

Exercice I

On veut ajouter dans le site web de votre lycée une page web permettant à chaque élève d'écouter ses chansons préférées ou de suivre des documentaires durant la semaine culturelle du mois de février

Saisir la structure minimale d'une page html

Reproduire le code suivant entre les balises `<body>` et `</body>`

```
<table border="1">
<tr>
<td
colspan="2"
>ligne1, colonne 1 et 2</td>
<td>ligne1, colonne3</td>
</tr>
<tr>
<td>ligne2, colonne1</td>
<td>ligne2, colonne2</td>
<td>ligne2, colonne3</td>
</tr>
</table>
```

Enregistrer sous votre nom .html dans un dossier qui porte le nom de votre classe sur le bureau de votre ordinateur

Ouvrir la page avec votre navigateur et décrire ce que vous voyez

Modifier votre code comme suite :

```
<table border="1">
<tr>
<td rowspan="2">ligne 1 et 2, colonne1</td>
<td>ligne1, colonne2</td>
<td>ligne1, colonne3</td>
</tr>
<tr>
<td>ligne2, colonne2</td>
<td>ligne2, colonne3</td>
</tr>
</table>
```

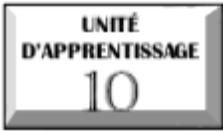
Remplacer les données de vos tables par les balises `<audio>` et `<video>` pour obtenir un tableau de données éventuellement de vidéo, au préalable prenez les fichiers audio et vidéo que vous enregistrez au préalable dans le dossier contenant votre page web

Insertion d'audio

```
<audio controls>
  <source src="hype_home.mp3">
  <source src="hype_home.ogg">
</audio>
```

Insertion de vidéo

```
<video controls>
  <source src="sintel.mp4">
  <source src="sintel.webm">
  <source src="sintel.ogv">
</video>
```

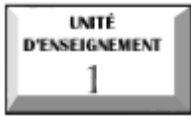


PROGRAMMER EN JAVASCRIPT

- 🖱 U.E 1 : Introduction au JavaScript
- 🖱 U.E 2 : Notions de base du JavaScript
- 🖱 U.E 3 : Structures de contrôles, tableaux et fonctions en JavaScript
- 🖱 U.E 4 : Les événements en JavaScript

Situation problème

Votre camarade vient de créer un formulaire en HTML permettant d'entrer les données par un utilisateur et souhaiterait contrôler et manipuler ces données (vérification de la saisie, convertir les données numériques, ...). Peut-il le faire en HTML ? sinon proposez lui un langage approprié.



Introduction au JavaScript

Compétences visées :

- ☞ Énoncer les limites du HTML liées à l'interactivité
- ☞ Définir JavaScript
- ☞ Énoncer les avantages et limites de JavaScript
- ☞ Utiliser la balise script

Activité

1. Donner le rôle des balises **form**, **input**, **a**
2. Donner le rôle des attributs **name**, **value**, **id**, **type**
3. Justifier cette affirmation : le HTML et le CSS permettent de créer des pages web statiques
4. Quand dit-on qu'une page web est interactive
5. Qu'entendez-vous par **côté client**, **côté serveur**

INTRODUCTION

Le html et le CSS nous permettent de créer et mettre en forme des pages web dites statiques car elles ne peuvent pas interagir avec l'utilisateur (récupérer ou vérifier les données saisies par un utilisateur par exemple). Pour Remédier à ce problème, des langages tels que, ASP, PHP, ou JavaScript ont vu le jour. Le JavaScript a été développé par Netscape en 1995 sous le nom de LiveScript et s'exécute directement sur le navigateur de l'utilisateur.

I. DÉFINITION, IMPORTANCE ET LIMITE DE JAVASCRIPT

I Définition

Un **script** est une suite d'instructions permettant d'automatiser certaines tâches.

Le **JavaScript** est un langage de script orienté objet exécutable par les navigateurs web. Pour cela on dit que c'est un langage qui s'exécute côté client.

2 Importance

JavaScript permet aux programmeurs web **de créer les pages dynamiques** ; c'est-à-dire :

-  **Interagir avec l'utilisateur**: JavaScript permet au navigateur de traiter directement (sans accéder au serveur) certaines actions de l'utilisateur ;
-  **Embellir les pages**: Afficher/masquer du texte, faire défiler des objets, créer des info-bulles, gérer des menus;
-  **Réagir aux événements de la souris**
-  **Effectuer des calculs et des tests.**

3 Limite

Les principales limites de ce langage sont : ne permet pas la connexion à une base de données, ne permet pas de lire ou écrire dans un disque dur, n'est pas autonome car ne peut fonctionner sans le HTML, est très dépendant du navigateur.

II. UTILISATION DE LA BALISE SCRIPT

Le code JavaScript est placé entre les balises : `<script >` et `</script>` selon l'une des structures suivantes :

<pre><script type="text/javascript"> taper des instructions JavaScript ici </script></pre>	<pre><script language="javascript"> taper des instructions JavaScript ici </script></pre>
--	---

Notons qu'il existe plusieurs façons d'utiliser un script dans une page web :

-  **De manière interne** : dans l'entête ou le corps de la page
-  **De manière externe** : dans un fichier externe (avec une extension `.js`) à la page web.

Dans ce second cas, la balise `<SCRIPT>` contiendra un attribut **src** faisant référence au fichier externe : `<script src="mon_fichier_externe.js"> code ici </script>` ; Ceci a pour avantages entre autres d'alléger la page HTML la rendant plus lisible, et rendre le code réutilisable et plus facilement maintenable.

Remarque : Il est vivement conseiller d'inclure des commentaires personnels dans vos codes JavaScript comme pour tous les langages de programmation. JavaScript utilise le symbole `« // »` pour des commentaires de fin de ligne et `« /* ... */ »` pour des commentaires sur plusieurs lignes.

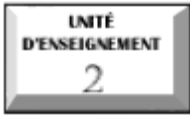
Ne pas confondre les commentaires JavaScript et les commentaires Html (<!-- ...-->).



EXERCICES

Exercice I

- 1) Définir script, javascript
 - 2) Pourquoi utilise-t-on du javascript dans les pages web
 - 3) Que signifie l'expression « langage coté client »
 - 4) Donner deux avantages ainsi que deux limites du langage javascript
 - 5) Soit l'affirmation suivante : « le code javascript peut être insérer partout dans une page web »
 - a. Dire en justifiant si cette affirmation est vraie
 - b. Peut-on utiliser le code javascript en dehors de la page web ? si oui comment ?
 - 6) Les commentaires permettent entre autres d'assurer la maintenabilité du code ; comment insérer un commentaire en javascript
-



Notions de base du JavaScript

Compétences visées :

- ☑ Déclarer les variables en JavaScript
- ☑ Effectuer les opérations d'entrées/sorties en JavaScript
- ☑ Effectuer les conversions de type
- ☑ Utiliser les opérateurs en JavaScript

Activité :

1. Qu'est-ce qu'un algorithme
2. Dans l'écriture des algorithmes,
 - a. Quelle différence faites-vous entre une constante et une variable
 - b. Quelles sont les conventions dans le nommage d'une variable
 - c. Faire la déclaration de variables nommées rayon (de type entier), perimetre (de type réel), nom (de type chaîne de caractères) et d'une constante PI
 - d. Déduire l'écriture du périmètre d'un cercle
3. Écrire les instructions permettant de saisir le rayon et d'afficher le périmètre
4. Si le périmètre est de 3.1415, quelle est le type du nombre obtenu par troncature d'ordre zéro du périmètre
5. On suppose maintenant que le rayon est de type chaîne de caractère (ex : "2.5première"); peut-on calculer le périmètre avec une telle valeur du rayon ? Si oui, comment ?

INTRODUCTION

Dans l'écriture des scripts, il est très souvent nécessaire d'avoir recours aux variables et constantes ; une variable est considérée comme un espace dans lequel on peut stocker une valeur (un chiffre, un nombre, un caractère ou une chaîne de caractères), qui peut être modifiée pendant l'exécution du programme par opposition à une constante qui ne peut être modifiée. Il est ainsi possible d'appliquer à ces objets différentes opérations dans le but d'effectuer une tâche précise.

I. DÉCLARATION DES VARIABLES EN JAVASCRIPT

En JavaScript, les noms de variable peuvent être aussi longs que l'on désire, mais doivent répondre à certains critères :

- 📁 Commencer par une lettre (majuscule ou minuscule) ou un "_" ;
- 📁 Comporter uniquement des lettres, des chiffres et les caractères _ et \$;
- 📁 Ne peuvent pas être des mots réservés du langage.

Exemples : surface, stylo_rouge ; **Contre Exemples** peri-metre, demi périmètre

NB : JS est sensible à la casse. Attention donc aux majuscules et minuscule !

Les variables peuvent se déclarer de deux façons :

- 📁 Soit de façon explicite : on dit à JS que ceci est une variable. Le mot clé (la commande) de déclaration d'une variable est « var »

Syntaxe : var nom_variable = valeur

Exemples : var nombre = 19 var prenom = "zavier" var note

- 📁 Soit de façon implicite : on écrit directement le nom de la variable suivi de la valeur que l'on lui attribue et JS s'en accommode.

Syntaxe : nom_variable = valeur **Exemples :** nombre = 19 prenom = "zavier"

Le mot clé de déclaration d'une constante est « const ».

Syntaxe : Const nom_constante = valeur

Exemples : Const taux_TVA = 0.1925 Const PI = 3.14

II. LES INSTRUCTIONS DE BASES EN JAVASCRIPT

Une **instruction** est l'opération élémentaire que le processeur peut accomplie en une seule fois. Les instructions d'entrées/sorties en javascript sont généralement effectuées au moyen de boites de dialogues ou de méthodes.

I L'instruction d'affectation.

Cette instruction permet d'attribuer une valeur ou le résultat d'une expression à une variable.

Syntaxe : nom_variable=valeur ;

Exemple : nombre = 19

prenom = "zavier"

2 L'instruction de lecture ou d'entrée.

Cette instruction permet de récupérer les informations saisies par l'utilisateur et de les stocker dans les variables. Pour cela on utilisera la méthode ***prompt()***.

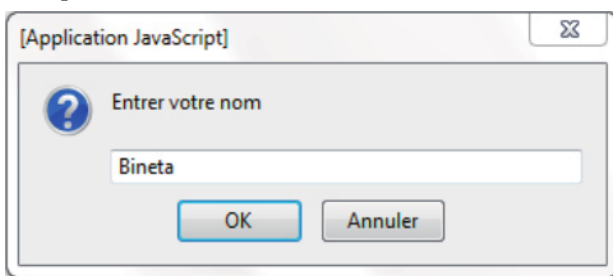
Syntaxe :

variable=**prompt**("texte à afficher", "valeur par défaut").

La valeur par défaut est optionnelle.

Exemple :

Nom = **prompt**("Entrer votre nom", "Bineta"); l'exécution de cette commande fera appel à la boîte de dialogue ci-contre où l'utilisateur peut entrer son nom et cliquer sur **ok** pour valider ou sur **annuler**



3 L'instruction d'affichage ou de sortie ou d'écriture.

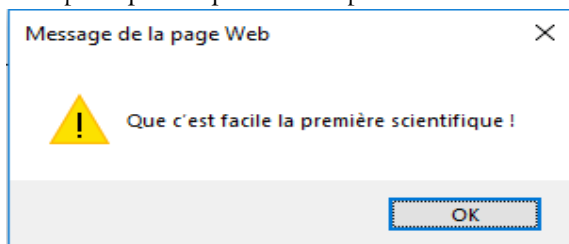
Pour afficher du texte en JavaScript on peut utiliser la méthode ***write()*** ou la méthode ***alert()***

Syntaxe :

document.write("texte à afficher");	ou	alert("texte à afficher");
document.write(nom_variable);	ou	alert(nom_variable);

Exemple :

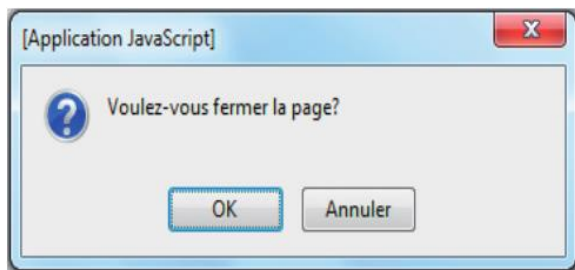
alert("Que c'est facile la première scientifique !"); l'exécution de cette commande va afficher une boîte de dialogue d'alerte similaire à celle présentée ci-contre et l'utilisateur n'a plus qu'à cliquer sur **ok** pour continuer.



NB : Il est aussi possible de demander une confirmation pour une action (suppression d'élément, fermeture de page, ...). On utilise la fonction «confirm» dont la syntaxe est :**confirm("Demande de confirmation")** ;

Exemple :

variable=**confirm**("Voulez-vous fermer la page ?") ;



Remarque : La réponse de la commande **prompt()** est toujours une chaîne de caractères.

Si on désire récupérer un nombre, il faut utiliser la fonction **parseInt** (pour convertir en un nombre entier) ou **parseFloat** (pour la convertir en nombre à virgule).

Syntaxe : **parseInt**(chaîne) ou **parseInt**(var) ; **parseFloat**(chaîne) ou **parseFloat**(var)

Exemples :

Age= **prompt**("Entrer votre age") ;

Ou Age = **parseInt**(**prompt**("Entrer votre ag

parseInt(Age) ;

parseInt("23AZer") et **parseInt**("23") ; //ces deux instructions retournent le nombre 23.

Rayon = **parseFloat**("24.568Bonjour38") ; //cette instruction renvoie le nombre 24.568

III. LES OPÉRATIONS EN JAVASCRIPT

Les opérations sont effectuées au moyen d'opérateurs qui permettent d'élaborer une expression en vue d'effectuer un calcul ou une comparaison ; l'usage des parenthèses est vivement conseillé dans le cas d'expressions complexes. Ainsi on peut distinguer :

2 Les opérateurs arithmétiques

Opérateur	Description
+	Addition
-	Soustraction
*	Multiplication
/	Division
^	Puissance
%	Modulo (a%b renvoie le reste de la division entière de a par b)
++	Incrément // x++ équivaut à x = x + 1
--	Décrément // x-- équivaut à x = x - 1

2 Les opérateurs logiques

Opérateur	Description
&&	ET logique
	OU logique
!	Négation logique

3 Les opérateurs de comparaison

Opérateur	Description
==	Est égal à
!=	Différent de
>	Strictement supérieur à
<	Strictement inférieur à
>=	Supérieur ou égal
<=	Inférieur ou égal

Retourne **True** ou **False** (1 ou 0)

4 L'opérateur de chaîne ou de concaténation

L'opération entre les chaînes de caractère s'appelle la concaténation, c'est-à-dire la juxtaposition des objets de type chaîne. L'opérateur de concaténation est '+'.

Exemples :

```
a=2 ; b=8 ; c=5 ; texte1= "bonjour " ; texte2= "le monde ! " ;
```

```
c = b - c ; → 3          d = (a > 0) && (c < b) ; → False (0)
```

```
alert (texte1 + texte2) ; → affiche bonjour le monde !
```

```
a++ ; → 3          b-- ; → 7          (a + c) == (b - c % 2) ; → True (1)
```

Remarque : L'expression **variable = variable opérateur constante** ($a = a + 3$ par exemple) peut être simplifiée en **variable opérateur = constante** ($a += 3$, qui lui est équivalente).

Ainsi, au lieu d'écrire **nombre = nombre * 26**, on peut écrire simplement **nombre*=26** ; il en est de même pour tous les opérateurs arithmétiques.



EXERCICES

Exercice 1

- 1) Qu'est-ce qu'une variable
- 2) Quelle différence faites-vous entre une variable et une constante
- 3) Justifier l'affirmation suivante : « le javascript est un langage faiblement typé »
- 4) Que signifie cette affirmation : « en javascript, une variable peut être déclarer implicitement »
- 5) Parmi les variables suivantes, identifier celles qui sont non valides et justifier :
_I23 ; pianic26 ; yahoo@ ; première_CD ; lyoss-plus ; grand prof ; 4points ; alert ; facile!_bizarre8

Exercice 2

Soit le script:

```
<script language = "javascript">
```

```
var age = parseInt(prompt("entrer votre age svp")) ;
```

```
</script>
```

- a. Quel est le type de donnée retourné par défaut dans un prompt
- b. Déduire le rôle du parseInt dans ce script

Exercice 3

Soit le script :

```
<script language = "javascript">
    var nombre1 = parseInt(prompt("entrer le premier nombre svp")) ;
    var nombre2 = prompt("entrer le deuxième nombre svp") ;
    alert(nombre1 + nombre2);
</script>
```

a. Quel sera le contenu de la boîte de dialogue *alert* si l'utilisateur entre respectivement

i. 04 et 88

ii. 123bonjour et salut

b. Corriger ce script pour retourner effectivement la somme de nombre1 et nombre2

Exercice 4

Déterminer le contenu des variables d, e et f dans le script suivant :

```
<script language = "javascript">
    a = 25 ; b = 12 ; c = 2 ;
    a++ ; b-- ; c*=3 ;
    d = a%b ; e = (b==c) ; f = e && d
</script>
```

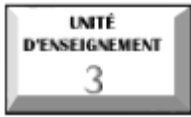
Exercice 5

1) Écrire un script qui demande à l'utilisateur le rayon et la hauteur d'un cylindre et affiche son volume

a. Dans une boîte de dialogue

b. Sur la page

2) Écrire un script qui demande le nom et le prénom de l'utilisateur puis affiche le nom complet



Structures de contrôles, tableaux et fonctions en JavaScript

Compétences visées :

- ☑ Traduire des algorithmes séquentiels, conditionnels et itératifs en JavaScript
- ☑ Déclarer un tableau et une fonction en JavaScript

Activité

Qu'est-ce qu'une structure conditionnelle ? une structure répétitive ?

Qu'est-ce qu'une fonction ? une procédure ?

Soit l'algorithme suivant :

Algorithme sans_nom

Var nombre : entier ;

Début

nombre $\leftarrow$ 4 ;

Tantque (nombre $\neq$ 0) faire

Ecrire ("entrer un nombre pair positif")

Lire (nombre)

Si ((nombre < 0) ou (nombre mod 2 $\neq$ 0)) alors

Ecrire ("le nombre doit être pair et positif")

finSi

finTantque

Fin

1. Que fait cet algorithme
2. Que se passe-t-il si l'utilisateur saisi les valeurs :
 - a. -2
 - b. 5
 - c. 0
3. Relevez dans cet algorithme une structure conditionnelle et une structure itérative
4. Traduire cet algorithme en script JavaScript

INTRODUCTION

En programmation JavaScript tout comme en algorithmique, il est parfois nécessaire d'exécuter une instruction ou un bloc d'instructions (fonction par exemple) uniquement lorsqu'une certaine condition est vérifiée ; pour cela, on utilise des structures de contrôles. En JavaScript on distingue plusieurs instructions qui nous permettent de contrôler l'exécution de nos programmes : **les structures conditionnelles et les structures répétitives ou boucles**

I. LES STRUCTURES CONDITIONNELLES

a Structure avec une alternative (if...)

L'instruction if est la structure de test la plus basique, elle permet d'exécuter les instructions lorsqu'une condition est vérifiée. La syntaxe est la suivante.

```
If(condition) {  
  // liste d'instructions  
}
```

Il est possible de définir plusieurs conditions à remplir avec les opérateurs **ET** et **OU** (&& et ||) et s'il n'y a qu'une instruction, les accolades ne sont pas nécessaires.

Exemple : If(x==2) document.write ("x vaut 2")

b Structure avec deux alternatives (if ... else...)

Elle permet en plus d'exécuter d'autres instructions en cas de non réalisation de la condition. La syntaxe est la suivante

```
if(condition) {  
  // listes d'instructions  
} else {  
  // autres séries d'instructions  
}
```

Exemple :

```
if(moyenne >= 10) {  
  document.write("Admis en classe supérieure");  
} else {
```

```
document.write("Redouble");
}
```

c Structure avec conditions imbriquées

Il arrive que l'instruction dans une structure conditionnelle soit une autre structure conditionnelle ; dans ce cas on parle de structures à condition imbriquée.

Syntaxe : Formulation générale

```
if (<condition1>) {
    <action>
} else if (<condition2>) {
    <action>
...
} else {
    <action>
...
}
```

Exemple :

```
if (reponse == 1) {
    cadeau = "console";
}
else if (reponse == 3) {
    cadeau = "fleurs" ;
}
else {
    cadeau = "rien du tout" ;
}
```

Remarque : Il existe une autre condition nommée « **Switch** » que nous ne verrons pas dans ce cours!

II. STRUCTURES RÉPÉTITIVES (OU BOUCLES OU ITÉRATIVES)

Elles permettent d'effectuer de manière répétitive une série d'opérations jusqu'à ce qu'une condition ne soit plus réalisée. JavaScript propose plusieurs types de boucles itératives : boucle **for**, boucle **while** et boucle **do...while**.

a While

Elle correspond à la structure algorithmique «Tant Que» et permet d'exécuter les <actions> aussi longtemps que <condition> est vraie. Si la condition est fausse au début, aucune instruction n'est exécutée. Les actions doivent permettre d'incrémenter la condition. Sa syntaxe est la suivante :

```
while ( <condition> ) {
<actions>;
...
}
```

Exemple :

```
<script language="JavaScript">
  compt=1;
  while (compt<5) {
    document.write ("ligne : " + compt + "<br>");
    compt++;
  }
  document.write("fin de la boucle");
</script>
```

b Do...While

Elle correspond à la structure algorithmique «Répéter» et permet d'exécuter les <actions> aussi longtemps que <condition> est vraie. Si la condition est fausse au début, les instructions sont quand même exécutées une fois. Les actions doivent permettre d'incrémenter la condition. Sa syntaxe est la suivante :

```
do {
<actions>
...
}
```

```
} while ( <condition> )
```

Exemple :

```
<script language="JavaScript">
  compt=1;
  do{
    document.write ("ligne : " + compt + "<br>");
    compt++;
  }
  while (compt<5)
  document.write("fin de la boucle");
</script>
```

Remarque :

La boucle While doit être utilisée uniquement si on a une condition qui teste l'entrée dans la boucle, et la Do...While si la condition teste la sortie de la boucle (le corps de la boucle doit être exécuter au moins une fois). Il est également recommandé d'utiliser la boucle lorsque le nombre d'itération est connu d'avance.

c For

Elle correspond à la structure algorithmique «Pour» et la syntaxe est la suivante :

```
for (compteur ; condition; modification du compteur) {
  <action>
  ...
}
```

Exemple :

```
<script language="JavaScript">
  var somme=20;
  for (i=10 ;i<=12 ; i++){
    somme+=i ;
    document.write("La somme est : " +somme+"<br>");
  }
</script>
```

III. TABLEAUX ET FONCTIONS EN JS

I Les tableaux

Les tableaux sont des variables qui permettent de stocker plusieurs données à la fois. Ils sont généralement utilisés pour stocker une liste d'éléments (nombres, textes, tableaux). De manière interne, les tableaux sont représentés de la manière suivante :

Indice 0	Indice 1	Indice 2	Indice 3	...	Indice n
Donnée 1	Donnée 2	Donnée 3	Donnée 4	...	Indice n

La taille d'un tableau est dynamique. Elle augmente au fur et à mesure qu'on y ajoute des éléments. Elle est donnée par la propriété «length» (exemple : `tableau1.length`)

syntaxe

Pour créer (déclarer) un tableau on utilise les syntaxes ci-après :

`tableau1 = new Array( ); // sans initialisation`

`tableau2 = new Array("donnée 1", "donnée 2", "donnée 3", "donnée 4");`

`tableau3 = ["donnée 1", "donnée 2", "donnée 3", "donnée 4"];`

Pour accéder à un élément du tableau, on utilise la syntaxe suivante : `tableau1[indice]`.

N.B : la position *I* est repérée par l'indice 0

Exemple : `tab [3]` indique la valeur contenue à la position 4 du tableau `tab`.

```
<script language="JavaScript">
    tab = new Array( ); //
    tab [0]= " Ella " ;
    tab [1]= "Anicet" ;
    tab [0]= " Raissa " ;
    tab.sort( );
    alert ("Prénoms par ordre alphabétique")
    for (indice=0 ; indice<tab.length ; indice++) {
        alert (tab [indice]);
    }
</script >
```

Interprétation :

-  La première ligne du script déclare un tableau vide ;
-  Ensuite on insère les prénoms «Ella», «Anicet» ;
-  On remplace la valeur d'indice 0 (c'est-à-dire «Ella») par «Raissa» ;
-  On range les données par ordre alphabétique ;
-  On entre dans une boucle «For» avec pour compteur «indice» initialisée à 0, la condition de continuation «indice<2» car la taille du tableau est 2 ;
-  La boucle va afficher les valeurs du tableau indice après indice.

2 Les fonctions

On appelle **fonction** un sous-programme qui permet d'effectuer un ensemble d'instruction par simple appel de la fonction dans le corps du programme principal. Une fonction qui ne retourne aucune valeur est appelée **procédure**. Dans JavaScript, les fonctions et les procédures sont définies par le mot clé **function**.

NB : Avant d'être utilisée, une fonction ou procédure doit être définie car pour l'appeler dans le corps du programme il faut que le navigateur la connaisse, c'est-à-dire son nom, ces arguments (paramètres) et les instructions qu'elle contient.

La **déclaration** d'une fonction ou procédure se fait en respectant la syntaxe suivante :

Syntaxe :

```
function nom_fonction(parametre 1, parametre 2, ...) {
    <instructions >
    <instructions >
    ....
    [ return resultat; ] /* présent si c'est une fonction */
}
```

Exemple : fonction calculant le carrée d'un nombre qu'on lui passe en paramètre.

```
function carre(nombre){
    res = nombre*nombre ;
    return res;
}
```

Pour utiliser une fonction, il suffit de faire **appel** à elle en écrivant son nom (tout en respectant la case) suivie d'une parenthèse ouvrante (éventuellement ces arguments) puis d'une parenthèse fermante. **Exemple** : `a = carre(5)` ;

Remarques:

a Fonctions prédéfinies

JavaScript dispose aussi de fonctions mathématiques tels que : `Math.abs(x)`, `Math.sqrt(x)`, `Math.pow(x,y)`, `Math.sin(x)`, `Math.log(x)`, `Math.random()`, `Math.max(x,y)`, `Math.min(x,y)` etc.

b Portée d'une variable

Toute variable déclarée en dehors de toute fonction, est dite **variable globale** et peut donc être exploitée partout dans le script tandis qu'une variable déclarée par le mot clé **var** dans une fonction aura une portée limitée à cette seule fonction et ne pourra donc pas être exploitée ailleurs dans le script. D'où le nom de **variable locale**. Par contre, toujours dans une fonction, si la variable est déclarée de manière implicite (sans utiliser le mot `var`), sa portée sera globale.



EXERCICES

Exercice 1

QCM (questions à choix multiples)

1) En JavaScript, l'instruction permettant d'affecter la valeur 1 dans la 2^e case du tableau nommé tab est :

a- tab[1]=1 ; b- tab(1)=2 ; c- tab[2]=1 ; d- tab(1)=2

2) L'instruction permettant d'afficher la taille du tableau nommé tab en JavaScript est : a- Alert(tab.taille); b- Document.write(tab.size); c- Alert (Tab.length); d- Alert (tab.length);

3) L'une des méthodes suivantes permet de convertir un tableau en chaîne de caractère :

a- concat(); b- sort(); c- join(); d- length()

4) Une structure permettant d'effectuer répétitivement une série d'opérations avec une condition est :

a- une fonction ; b- une boucle ; c- une procédure ; d- une itération

5) Selon la hiérarchie des objets en JS, le bouton but1 du formulaire form1 peut être accéder par :

a- window.form1.but1; b- document.form1.but1;
c- document.window.form1.but1; d- window.but1 ;

Exercice 2

Répondre par Vrai ou Faux

1) On peut déclarer un tableau sans initialiser en JavaScript

2) La méthode prompt() est une instruction d'écriture ou de sortie

3) Le mot clé return est obligatoire dans la déclaration d'une procédure

Exercice 3

On considère le code JavaScript suivant. Répondre clairement aux questions suivantes

```

<html>
  <head><title>Bacc    ESG</title>
  <script    language="javascript">
    var note1 = 12 ;var note2 = 13;
decision;
    function moyenne(note1, note2){
      var moyenne = (note1+note2)/2 ;
      return    moyenne    ;}
    moyenne(note1,    note2)    ;
    if(moyenne    >=10){
      decision="vous avez reussi au
Bacc"
      document.write(decision) ;}
    else{
      decision    =    "Echec";
      document.write(decision) ;}
    </script>
  </head>
  <body> <h1> Exo </h1> </body>
</html>

```

1. Quels langages a-t-on utilisé pour écrire ce code ?
2. Donner deux exemples de logiciels permettant d'éditer ce code.
3. Quel titre aura le document généré par ce code lorsqu'il sera interprété ?
4. Combien de variables compte ce programme ?
5. Quel sera le contenu de la variable moyenne lorsque le code sera interprété ?
6. Relever dans ce code :
 - a. Une instruction d'initialisation
 - b. Une instruction d'affichage
 - c. *Un nom de fonction*
 - d. Une structure de contrôle
7. Dans quel endroit de la page HTML a été inséré le code JavaScript ?

Exercice 4

Le proviseur du lycée a besoin d'un élève de terminale parce qu'il aimerait calculer directement depuis une des pages web du site du lycée le nombre total des élèves et vous allez l'y aider.

1. Donner la syntaxe d'une **fonction** en JavaScript
2. Donner la syntaxe d'une **boucle for** en JavaScript
3. Écrire une fonction nommée **som_tab** qui prend un tableau d'entiers en paramètre et retourne la somme de ses valeurs
4. Déclarer sans initialiser un tableau nommé **nbr_el_class** qui enregistrera chaque fois le nombre d'élèves d'une classe
5. Que fais ce bout de code ?

```
j=0 ;
```

```
do {
```

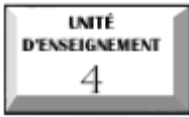
```
    var a= parseInt(prompt("entrer un nombre d'eleves et 0           lorsque vous
avez parcouru toutes les classes") );
```

```
Tab[j]=a;
```

```
j=j+1;
```

```
}while (a != 0);
```

En utilisant les **questions 3, 4 et 5**, écrire un script pour afficher le nombre d'élèves de l'établissement au Boss



Les événements en JavaScript

Compétences visées :

- ☑ Utiliser l'évènement onclick
- ☑ Utiliser les zones de saisie en javascript
- ☑ Programmer un convertisseur

Activité

1. Qu'est-ce qu'un formulaire ?
2. Donner quelques éléments que nous pouvons retrouver sur un formulaire
3. Comment déclare-t-on un formulaire sur une page web ?
4. Écrire le code HTML permettant de créer un formulaire comportant :
 - a. Une zone de texte libellée *taille en cm*
 - b. Un bouton de validation avec la valeur *convertir*
 - c. Une autre zone de texte libellée *taille en pouce*
5. On souhaite maintenant que lorsque la page est visualisée, l'utilisateur saisi sa taille en centimètre dans le champ prévu à cet effet, puis il clique sur convertir et sa taille en pouce doit être affichée dans le champ pour la taille en pouce. Comment procéder ?

INTRODUCTION

Le langage JavaScript associé au langage HTML permet de rendre les pages web interactives ; (non seulement les utilisateurs peuvent prodiguer des informations à une page web mais la page peut aussi réagir aux informations saisies par un utilisateur). Les boîtes de dialogues comme nous l'avons déjà vu permettent par exemple d'envoyer des messages aux utilisateurs ou d'obtenir des informations des utilisateurs ; cette nouvelle leçon sera consacrée à l'étude d'un autre élément d'interactivité dans JavaScript : **les évènements**

I Définition et syntaxe

Les **événements** sont des actions de l'utilisateur qui vont pouvoir donner lieu à une interactivité. L'événement par excellence est **le clic** de souris. Il est possible d'associer des actions à des événements aux moyens des **gestionnaires d'événements**. Le nom d'un gestionnaire d'événement commence toujours par le mot « **on** » ; on peut par exemple citer :

-  **onclick / ondblclick** : lorsqu'on clique ou double clique avec la souris
-  **onfocus** : lorsqu'un champ est actif ou sollicité
-  **onchange** : lorsqu'un champ perd le focus
-  **onsubmit** : lorsqu'on clique sur un bouton du type submit
-  **onkeydown/onkeyup** : lorsqu'on presse / relâche une touche du clavier
-  etc.

Les gestionnaires d'événements sont associés à des objets, et leur code s'insère dans la balise de ceux-ci ; la syntaxe d'utilisation d'un gestionnaire d'événement est la suivante :

onEvenement= "Action_Javascript ou Fonction()";

Exemple : onclick= "alert("Bonjour");" onchange="mafonction()";

2 Accès aux éléments de la page par l'attribut « id »

Pour accéder à un élément de la page, on lui donne une valeur d'attribut « **id** » dans le code HTML. Ensuite, dans le script JavaScript, on utilise la syntaxe suivante :

document.getElementById("id") // où *id* désigne la valeur de l'attribut id

Exemple :

```
document.getElementById("lienProduits").href="products.html";
document.getElementById("img").src = "photo.jpg";
document.getElementById("txt").value= "les ways forts";
```

Remarque : il est également possible d'accéder aux éléments d'une page par l'attribut « **name** » mais cela donne généralement lieu à des codes plus long ; nous n'aborderons donc pas cette notion dans cette leçon.

3 Exemple d'application : réalisation d'un convertisseur

Confère activité question n°5

The screenshot shows a web browser window with the title 'JavaScript en action'. The address bar shows a local file path. The page content is titled 'Mon simple convertisseur'. It features two text input fields: 'Taille en cm : 190' and 'Taille en mètre : 1.9'. A 'Convertir' button is positioned below these fields.

```

<html>
<head>
  <meta charset = "utf-8">
  <title>validation de formulaire avec javascript</title>
</head>
<body >
  <font face="times new roman">
    <center><h2> mon simple convertisseur</h2></center>
    <hr> <br>
    <form name="frm" method="get">
      <center>
        <label for="h_cm">taille en cm :</label> <input type="text" name="h_cm"
id="h_cm" > &nbsp;
        <label for="h_m">taille en mètre : </label><input type="text" name="h_m"
id="h_m"> <br><br>
        <input type="button" value="convertir" onclick="convertir();" >
      </center>
    </form>

<script language="javascript">
  function convertir(){
    var h_cm = document.getElementById("h_cm").value;
    var h_m = h_cm / 100;
    document.getElementById("h_pd").value = h_m;
  }
</script>

```

</body>

</html>



EXERCICES

Exercice 1

- 1) Définir évènement
- 2) Quel est l'évènement qui est supporté nativement par le langage HTML
- 3) Donner le rôle d'un gestionnaire d'évènement
- 4) Décrire brièvement quatre gestionnaires d'évènement

Exercice 2

Soit un formulaire avec une zone de texte (avec cet attribut `id = txtZone`)

- a. Écrire une instruction permettant de garder la valeur de la zone de texte (le texte qu'elle contient) dans une variable *txtVal*
- b. Écrire une instruction permettant de remplacer ce contenu par la valeur « *trop cool* »

Exercice 3

En vous inspirant de l'exemple du cours,

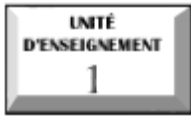
- a. Créer une page web contenant :
 - i. Une zone de texte (libellée **nom complet**)
 - ii. Deux boutons radios pour le sexe (**M** et **F**)
 - iii. Un bouton pour valider (**OK** par exemple).
- b. Par la suite, écrire une fonction permettant d'afficher le message :
« Bienvenue M. `nom_complet` » ou « Bienvenue Mme. `nom_complet` », suivant le sexe choisi (où `nom_complet` désigne le nom saisie par l'utilisateur).

Ex : Bienvenue M. Lionel Sop ou Bienvenue Mme. Raïssa Ngah



PROGRAMMER EN C

-  U.E 1 : Introduction au langage C
-  U.E 2 : Les structures de contrôle en C
-  U.E 3 : Les tableaux en C
-  U.E 4 : Les sous-programmes en C
-  U.E 5 : Exécution d'un programme C



Introduction au langage C

Objectifs pédagogiques :

- ☞ Installer un compilateur C ;
- ☞ Écrire la structure d'un programme C ;
- ☞ Inclure les bibliothèques `stdio.h`, `stdlib.h`, `math.h` et `conio.h` ;
- ☞ Utiliser les fonctions d'entrée/sortie classiques (`scanf`, `printf`, `get`,...);

Situation problème :

Votre ami souhaite utiliser l'ordinateur pour exécuter ses algorithmes. Pour cela, votre petit frère lui propose la traduction de ces algorithmes en langage C avant de les exécutés. Ne connaissant rien sur ce langage, il fait donc appel à vous dans le but de l'expliquer quelques notions sur langage C.

Consignes :

1. Définir langage de programmation
2. A part le langage C, quel autre langage de programmation connaissez-vous ?
3. Comment appelle-t-on un algorithme déjà traduit en langage de programmation ?
4. Comment appelle-t-on l'application qui permet d'exécuter un programme sur un ordinateur ?
5. Donner la structure du programme écrit en langage C.
6. Donner le rôle des bibliothèques en langage C. Puis énumérer quelques exemples.
7. Donner une fonction utilisée en langage C pour :
 - Afficher un message
 - Lire une variable.

I. DÉFINITION :

Programmation : c'est la traduction d'un algorithme en un langage de programmation.

Langage de programmation : ensemble des mots et symboles permettant d'écrire un programme.

Programme : Suite d'instructions écrite dans un langage de programmation quelconque et permettant de réaliser une ou plusieurs tâches.

II. INSTALLATION D'UN COMPILATEUR

Un **compilateur** : est une application qui transforme le code source d'un programme en un fichier binaire exécutable par la machine.

Exemple de compilateur : GNU Assembler, Turbo Pascal, Turbo C, GNU Pascal, Delphi, javac, GCJ (Gnu Compiler for Java), Jikes, Visual Basic, FreeBasic...

NB : chaque langage de programmation a son compilateur approprié. Par exemple Turbo Pascal, Delphi pour le langage **Pascal** et Javac , GCJ pour le langage **Java**.

Pour écrire des programmes, il est nécessaire d'installer différents outils. Le premier et le plus important est **l'installation d'un compilateur**.

Certains compilateurs sont déjà disponibles sur plusieurs SE, alors que d'autres sont spécifiques d'un système. Pour le plus connus sur Desktop : GCC (pour Windows, Android, Linux), Microsoft Visual Studio (MSVC).

D'autres compilateurs sont intégrés dans les environnements de développement (IDE). Il suffit de les cocher lors de l'installation de ces IDE.

Exemples des IDE: Code:: Blocks, Visual Studio, Qt Creator, Dev C++, Dev Pascal, Eclipse...

III. STRUCTURE GÉNÉRALE D'UN PROGRAMME C

Définition :

Une expression : est une suite de composants élémentaires syntaxiquement correcte. Elle est toujours suivie d'un point-virgule (;)

En C on n'a pas une structure syntaxique englobant tout, comme la construction « **Algorithme ... Fin.** ». Un programme n'est qu'une collection de fonctions assortie d'un ensemble de variables globales. Ainsi un programme C est structuré comme suit :

```

[Directives au préprocesseur]
[Déclarations de variables externes]
[Fonctions secondaires]

int main ()
{
    Déclarations de variables internes
    instructions
}

```

La ligne : `int main()` se nomme un "**en-tête**". Elle précise que ce qui sera décrit à sa suite est en fait le "programme principal". Elle peut avoir des paramètres formels.

Le **programme (principal)** proprement dit est constitué des variables internes et des instructions et vient à la suite de cet en-tête. Il est délimité par les accolades "{" et "}".

Exemple de programme C affichant "Bonjour".

```

1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      printf("Bonjour tout le monde !\n" ) ;
7      return 0;
8  }

```

La première ligne de notre programme : `#include <stdio.h>` est en fait un peu particulière. Il s'agit d'une "directive" qui est prise en compte avant la traduction (compilation) du programme.

Ces directives, contrairement au reste du programme, doivent être écrites à raison d'une par ligne et elles doivent obligatoirement commencer en début de ligne. Leur emplacement au sein du programme n'est soumis à aucune contrainte (mais une directive ne s'applique qu'à la partie du programme qui lui succède). D'une manière générale, il est préférable de les placer au début.

La directive demande en fait d'introduire (avant compilation) des instructions (en langage C) situées dans le fichier `stdio.h`. Notez qu'un même fichier

en-tête contient des déclarations relatives à plusieurs fonctions. En général, il est indispensable d'incorporer `stdio.h`.

Quelques bibliothèques (ou directives de préprocesseur) utilisés en langage C sont :

Bibliothèques	Rôle
<code><stdio.h></code>	Fournit la capacités centrales d'entrées/sorties
<code><math.h></code>	Pour calculer des fonctions mathématiques
<code><stdlib.h></code>	Pour exécuter les opérations de conversion, l'allocation des mémoires, le contrôle de processus, le tri, la recherche, ...
<code><string.h></code>	Pour manipuler les chaînes de caractères

NB : cette liste des bibliothèques standard C n'est pas exhaustive.

IV. NOTION DES VARIABLES ET OPERATEURS

Variable

Une variable est un objet manipulé dans l'exécution d'un programme. Elle est caractérisée par son **identificateur** (nom), son **type** et parfois sa **valeur** (le cas de constante).

Quelques types prédéfinis en langages C sont :

- ***int*** (*entier naturel*)
- ***Char*** (*caractères*)
- ***float, double*** (*nombre à virgule*)

Une variable doit être déclarée avant son utilisation. En C, toute instruction composé d'un spécificateur de type et d'une liste d'identificateurs séparés par une virgule est une **déclaration**. Ainsi en C, une variable est déclarée de la manière suivante : *type identificateur ;*

Exemples : ***int a ;*** et ***char a ;*** sont de déclarations

Operateur

Le tableau ci-dessous résume quelques operateurs utilisés en langage C

Operateurs	Symboles	Operateurs	Symboles
Affectation	=	égal	==
Reste de la division(modulo)	%	diffèrent	!=
Multiplication	*	ET logique	&&
Inférieur ou égal	<=	OU logique	

V. LES ENTRÉES/SORTIES EN C

Durant l'exécution d'un programme, le processeur, qui est le cerveau de l'ordinateur, a besoin de communiquer avec le reste du matériel. Il doit en effet recevoir des informations pour réaliser des actions et il doit aussi en transmettre. Ces échanges d'informations sont les entrées et les sorties (ou input / output en anglais).

Les sorties

Le tableau ci-dessous décrit les trois fonctions d'affichage de données

Fonctions	Rôle
Printf()	Pour écrire une chaîne de caractères formatée
puts()	Pour écrire une chaîne de caractères toute simple
Putchar()	Pour écrire un caractère

La syntaxe de l'utilisation de ces fonctions est :

fonction ("texte a afficher... ") ;

Exemples :

```
printf("Bonjour le monde");
```

```
puts("Hello world !") ;
```

```
put("c");
```

La fonction **Printf** permet non seulement d'afficher des chaînes de caractères simples, mais également la valeur d'une variable passée en paramètre. Pour ce faire, il

suffit d'utiliser un indicateur de conversion : il s'agit du caractère spécial % suivi d'une lettre qui varie en fonction du type de la variable.

Le tableau ci-dessous donne quelques indicateurs de conversion :

Type	Indicateurs de conversions
char	%c
int	%d
long	%ld
short	%hd
float	%f
double	%f
long double	%Lf
unsigned int	%u
unsigned short	%hu
unsigned long	%lu

Après avoir inscrit un indicateur de conversion dans la chaîne de caractère (dans les guillemets ""), il faut indiquer de quelle variable il faut afficher la valeur. Il suffit de rajouter une virgule après les ces derniers, suivis du nom de la variable, comme ceci :

```
printf ("% [lettre]", variable_ a_afficher);
```

Exemples :

```

1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int variable = 20;
7
8      printf("%d\n", variable);
9      return 0;
10 }
```

Remarque : Plutôt qu'appeler plusieurs fois la fonction `printf` pour écrire du texte, on peut ne l'appeler qu'une fois et écrire plusieurs lignes. Pour cela, on utilise le signe `\` à chaque fin de ligne.

Les entrées

Pour récupérer les valeurs saisies par l'utilisateur, nous utilisons les fonctions suivantes : `scanf()` et `get()`

La syntaxe de l'utilisation de la fonction `Scanf()` est :

```
scanf("%[lettre]", &variable qui_contiendra_notre_valeur);
```

NB : La fonction `scanf` a besoin de connaître l'emplacement en mémoire de nos variables afin de les modifier. Afin d'effectuer cette opération, on utilise le symbole `&`. Donc, si vous oubliez le `&`, le programme plantera quand vous le lancerez, car vous tenterez d'accéder à une adresse mémoire inexistante !

Exemple :

```

1  #include <stdio.h>
2
3
4  int main(void)
5  {
6      int age;
7
8      printf("Quel âge avez-vous ? ");
9      scanf("%d", &age);
10     printf("Vous avez %d an(s)\n", age);
11     return 0;
12 }
```



EXERCICES

Exercice 1

Vous souhaitez écrire un programme C qui demande à l'utilisateur de saisir un nombre puis affiche ce nombre.

1. Donner la structure d'un programme C
2. Quelles sont les bibliothèques standard que vous avez besoin dans votre programme C
3. Écrire l'algorithme qui permet de réaliser cette tâche.
4. Traduire votre algorithme en langage C

Exercice 2

1. Définir le terme suivant : compilateur, programme, programmation, langage de programmation.
2. Citer deux exemples de compilateur C
3. Donner deux exemples de langage de programmation.
4. Donner le rôle des bibliothèques standard suivant : `<stdio.h>`, `<stdlib.h>` et `<math.h>`
5. Donner 03 exemples des fonctions C utilisées pour afficher les textes à l'écran.
6. Donner deux exemples des fonctions C qui permet la récupération des données de l'utilisateur.
7. Donner la différence entre les fonctions `puts()` et `printf()`
8. Quel rôle joue le symbole `&` dans la récupération de la valeur d'une variable ?

Exercice 3

Écrire un programme C qui affiche « bonjour » suivi du nom de l'utilisateur.

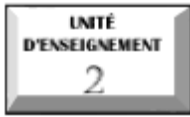
Exercice 4

Votre petit frère qui apprend la programmation vient d'écrire le programme suivant :

```
1 #include <math.h>
2 int main() {
3 int a ;
4 put("entrer un nombre");
5 scanf(%d,a) ;
6 printf("le nombre saisi est:",a);
7 return 0 ;
8 )
```

Travail demandé :

Ce programme est -il correct ? Si non relevez toutes les lignes incorrectes puis corriger.



Les structures de contrôle en C

Objectifs pédagogiques :

- ☑ Écrire la syntaxe des structures for, while et do...while en C;
- ☑ Utiliser l'opérateur conditionnelle (?) et virgule ;

Situation problème :

Votre amie souhaite écrire un programme C qui calcule et affiche la table de multiplication d'un nombre positif saisi par l'utilisateur. Elle souhaite donc que ce programme affiche à chaque fois un message à l'utilisateur s'il saisi un nombre négatif. Rencontrant des problèmes sur l'écriture de ce programme, elle donc appelle à vous dans le but de l'aider.

Consignes :

1. Qu'entend-on par structure de contrôle ?
2. Énumérer les différentes structures de contrôle utilisées en programmation.
3. Quelle structure utilise-t-on dans le cas où la condition se traite en deux cas possibles ?
4. Quelle structure utilise-t-on dans chacun des cas suivants :
 - La répétition connue d'un bloc instructions d'un programme ?
 - La répétition non connue d'un bloc instructions d'un programme ?

En programmation informatique, une **structure de contrôle** est une instruction particulière d'un langage de programmation impératif pouvant dévier le flot de contrôle du programme la contenant lorsqu'elle est exécutée. Le langage C propose plusieurs structures de contrôle différentes, qui nous permettent de nous adapter à tous les cas possibles. Il permet d'utiliser les conditions (structures alternatives), des boucles (structures itératives).

Structure alternative (if...else...)

Il s'agit de tester une condition et d'effectuer une série d'instructions si la condition est vraie et si elle est fausse, effectuer une autre série d'instructions. Il existe deux formes de structures conditionnelles :

La forme réduite

```
if (condition) {
    bloc instructions 1
}
```

Exemple : le programme C qui prend en entrée un nombre puis affiche le message « positif » si c'est ce nombre est supérieur à 0 est :

```
#include <Stdio.h>
int main (void) {
    int n ;
    printf ("saisir un nombre");
    scanf ("%d",&n);
    if (n>0) {
        printf ("Positif");
    }
    Return 0;
}
```

La forme complète

```
if (condition) {
    bloc instructions 1
} else {
    bloc instructions 2
}
```

Exemple : le programme C qui permet d'étudier la parité d'un nombre saisi par l'utilisateur est :

```
#include <Stdio.h>
int main (void) {
    int n ;
    printf ("saisir un nombre");
    scanf ("%d",&n);
    if (n%2==0) {
        printf ("%d",n,"est paire");
    } else {
        printf ("%d",n,"est impaire");
    }
    Return 0;
}
```

L'opérateurs conditionnel ?

L'opérateur conditionnel ? est un opérateur ternaire. Sa syntaxe est la suivante :
condition ? *expression 1* : *expression 2* ;

Cette expression est égale à *expression 1* si *condition* est satisfaite, et à *expression 2* sinon.

Exemple : le programme C qui calcule la valeur absolue d'un nombre est :

```
#include <Stdio.h>
int main (void) {
    int x, abs;
    abs=((x>=0)? x : -x) ;
    printf(" abs= %d",abs);
}
Return 0;
}
```

L'opérateur virgule

Une expression peut être constituée d'une suite d'expressions séparées par des virgules : *expression 1, expression 2,, expression n* ;

Cette expression est alors évaluée de gauche à droite. Sa valeur sera la valeur de l'expression de droite. Par exemple, le programme

```
#include <Stdio.h>
int main (void) {
    int a, b ;
    b=((a=3),(a+2)) ;
    printf(" b= %d",b);
}
Return 0;
}
```

retournera 5 à la sortie.

NB : la virgule séparant les arguments d'une fonction ou les déclarations des variables n'est pas l'opérateur virgule.

Structures itératives

Toutes les structures itératives répètent l'exécution de traitement(s). Deux cas sont cependant à envisager, selon que :

- Le nombre de répétitions est connu à l'avance : c'est le cas des boucles itératives.
- Le nombre de répétitions n'est pas connu ou est variable : c'est le cas des boucles conditionnelles.

Boucle for

Cette structure est une boucle itérative ; elle consiste à répéter un certain traitement un nombre de fois fixé à l'avance. Cette structure est donnée par :

```
for (initialisation ; condition ; incrémentation ) {
    bloc instructions;
```

```
}
```

Exemple : Affichage de nombres plus petit ou égal à 5.

```
for (i=0 ; i<=5 ; i++ ) {  
    printf(" i=%d",i);  
}
```

Boucle while

Parfois, pour réaliser une tâche, on doit effectuer plusieurs fois les mêmes instructions, sans que le nombre de fois soit déterminé à l'avance. On utilise alors une boucle conditionnelle. Dans cette structure, le même traitement est effectué tant qu'une condition reste valide ; la boucle s'arrête quand celle-ci n'est plus remplie. Cette structure répétitive est ainsi formulée :

```
while (condition vraie) {  
    bloc instructions;  
}
```

Exemple : affichage des nombres plus petit que 5

```
i=0 ;  
while (i < 5) {  
    printf("i=%d",i);  
    i++;  
}
```

Boucle do...while

Une variante de structure répétitive avec boucle conditionnelle consiste à répéter un traitement jusqu'à ce qu'une certaine condition soit vérifiée. Dans ce cas, la boucle est exécutée au moins une fois, après quoi on teste la condition. On la traduit par l'instruction :

```
do {
    bloc instructions;
} While (condition);
```

Exemple : affichage des nombres plus petit que 5

```
i=0;
do {
printf("i=%d",i) ;
i++;
} While (i<5) ;
```



EXERCICES

Exercice 1

Soit le programme suivant :

```
#include <stdio.h>
int main(void)
{
    int i,n,som;
    som = 0;
    for(i=0;i<4;i++)
    {
        printf(''donnez un entier '');
        scanf('' %d '',&n);
        som+=n;
    }
    printf(''somme : %d\n'',som);
    return 0 ;
}
```

Ecrire un programme C réalisant exactement la même chose, en utilisant :

- Une instruction while;
- Une instruction do ... while

Exercice 2

Écrire un programme C pour calculer la moyenne de notes fournies au clavier. Le nombre de notes n'est pas connu à priori et l'utilisateur peut en fournir autant qu'il le désire. Pour signaler qu'il a terminé on convient de fournir une note négative non comptée dans la moyenne.

Exercice 3

Écrire un programme C pour Déterminer si un nombre entier fourni en donnée est premier ou non

Exercice 4

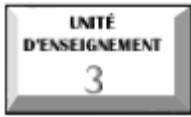
Écrire un programme qui affiche la table de multiplication des nombres de 1 à 10

Exercice 5

Soit le programme C ci-dessous :

```
#include <Stdio.h>
int main (void) {
    int a,b,m;
    m =((a>b)? a : b) ;
    printf("%d",m);
}
Return 0;
}
```

- Que fait ce programme ?
 - Traduire ce programme en utilisant la structure alternative.
-



Les tableaux en C

Objectif pédagogique :

📄 Déclarer un tableau en C ;

Situation problème :

Votre camarade Ali souhaite écrire un programme C qui calculera la moyenne arithmétique de 50 nombres. Votre petit frère lui conseille d'utiliser le tableau dans son programme pour stocker les 50 nombres. Ne connaissant pas comment utiliser le tableau en C, il fait appel à vous dans le but de l'aider.

Consignes :

1. Définir tableau
2. Donner l'intérêt de l'utilisation d'un tableau
3. Donner les caractéristiques d'un tableau
4. Donner la syntaxe de déclaration d'un tableau en langage C
5. Expliquer comment, on remplit un tableau avec ses éléments en C
6. Expliquer comment on accède aux différents éléments d'un tableau en C.

I. DÉCLARATION D'UN TABLEAU

Un tableau est un ensemble fini d'éléments de même type, stockés en mémoire à des adresses contiguës.

La déclaration d'un tableau à une dimension se fait de la façon suivante :

type nom-du-tableau[nombre-éléments] ;

où ***nombre-éléments*** est une expression constante entière positive. Par exemple : ***int tab[10] ;*** indique que ***tab*** est un tableau de 10 éléments de type ***int***.

Pour plus de clarté, il est nécessaire de donner un nom à la constante ***nombre-éléments*** par une directive au préprocesseur, par exemple ***#define nombre-éléments 10***

De façon similaire, on peut déclarer un tableau à plusieurs dimensions. Par exemple pour un tableau à deux dimensions :

type nom-du-tableau[nombre-lignes][nombre-colonnes] ;

Exemple : *int tab[10][8]* est un tableau à deux dimensions de 10 lignes et de 8 colonnes.

II. AJOUT DES ÉLÉMENTS D'UN TABLEAU

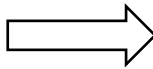
Pour remplir les différents éléments du tableau, on pourra affecter à chaque case de ce tableau une valeur.

Exemple :

```
int tab[2] ;
```

```
tab[0]=10 ;
```

```
tab[1]= 9 ;
```



10	9
----	---

Évidemment, à ce rythme-là, l'affectation est longue, surtout si votre tableau est de grande taille. C'est pourquoi on utilise la boucle itérative **for**.

On peut initialiser le tableau lors de sa création par une liste de constantes de la façon suivante :

type nom-du-tableau[N]={ constante-1,...,constante-N} ;

Par exemple, on peut écrire :

```
#define N 4
```

```
int tab[N]={1,2,3,4} ;
```

1	2	3	4
---	---	---	---

III. ACCÈS AUX ÉLÉMENTS D'UN TABLEAU

On accède à un élément du tableau en lui appliquant l'opérateur []. Les éléments d'un tableau sont toujours numérotés de 0 à *nombre-éléments-1*

Par exemple l'instruction `printf("%d",tab[2]);` affiche le troisième élément du tableau (contenu du tableau à l'indice $i=2$).

On pourra afficher tous les éléments du tableau en utilisant la boucle **for**. Le programme ci-dessous réalise la tâche.

```
#include <Stdio.h>
#define N 10
int main () {
    int tab[N];
    int i;
    .....
    for (i=0;i<N;i++) {
        printf("tab[%d]=%d\n",I,tab[i]);
    }
    Return 0;
}
```



EXERCICES

Exercice 1

Écrire un programme C qui calcule la moyenne de 5 nombres saisi par l'utilisateur.

Exercice 2

1. Définir tableau puis donner 03 éléments qui caractérisent un tableau.
2. Quel inconvénient présente le remplissage d'un tableau en utilisant l'affectation ? Quelle autre méthode proposeriez-vous pour résoudre ce problème ?

Exercice 3

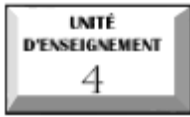
1. Écrire un programme C qui permet de rechercher le plus petit élément d'un tableau à 20 éléments.
2. Écrire un programme C qui permet de rechercher le nom d'un candidat parmi les noms de 50 candidats admis à un concours.
3. Écrire un programme C qui permet de faire la somme de 10 éléments d'un tableau.
4. Écrire un programme C permettant de créer les tableaux ci-dessous :

Tableau I :

12	13	14	15	222
----	----	----	----	-----

Tableau 2 :

2	3	4	5	22	1	2	1	2	3
11	22	11	22	3	4	12	14	14	6



Les sous-programmes en C

Objectif pédagogique :

- ☞ Déclarer et appeler un sous-programme en C ;

Situation problème :

Votre ami Isaac a utilisé le langage C pour créer une calculatrice qui effectue les opérations d'addition, de soustraction, de division et de multiplication. Il constate que le code source de son programme est touffu et volumineux ce qui lui rend difficile la compréhension de son code. Il cherche donc un moyen très efficace pour mieux structurer son programme. Pour cela il fait appel à vous dans le but de l'aider.

Consignes :

1. Que doit faire votre ami Isaac pour organiser et mieux comprendre son code ?
2. Comment appelle-t-on le sous-programme en C ?
3. Donner la syntaxe de définition d'une fonction en C
4. Comment fait-on appel à une fonction en langage C ?

I. DÉFINITION D'UNE FONCTION

L'écriture et la compréhension d'un programme peut s'avérer difficile lorsque le nombre d'instructions devient trop important. A ce moment, le programme devient lourd et son exécution par l'ordinateur devient beaucoup plus lente. Pour pallier ce problème, on a mis sur pied la notion de sous-programme informatique dont l'objectif principal est de scinder un programme complexe et touffu en plusieurs sous-programmes plus simples et plus efficaces. En C, les sous-programmes sont appelés les **fonctions**.

A part la fonction principale `main()` qui est obligatoire, les autres fonctions d'un programme C sont dites fonctions secondaires.

Une fonction en C se définit de la manière suivante :

type nom-fonction(type-I argument-I, ..., type-n argument-n)

{

[déclarations des variables locales]

listes d'instructions

}

La première ligne de cette définition est l'**en-tête** de la fonction. Dans cet en-tête, **type** désigne le type de la fonction, c'est-à dire le type de la valeur qu'elle retourne.

NB :

📁 Contrairement à d'autres langage de programmation, en C, il n'y a pas la notion de procédures et de sous-programme. Une fonction qui ne renvoie pas de valeur est une fonction dont le type est spécifié par le mot clé **void**. Les arguments de la fonction sont appelés **paramètres formels**, par opposition aux **paramètres effectifs**.

📁 Si la fonction ne possède pas de paramètres, on remplace la liste de paramètres formels par le mot clé **void**.

📁 Le corps de la fonction débute par des déclarations de variables locales à cette fonction. Il se termine par l'instruction de retour à la fonction appelante, **return**, dont la syntaxe est : **return(expression);**

La valeur **expression** est la valeur que retourne la fonction. Son type doit être de le même que celui qui a été spécifié dans l'entête de la fonction. Si la fonction ne retourne pas de valeur (fonction de type void), sa définition s'achève par **return ;**

Exemples :

📁 Une fonction calculant le produit de deux nombres est :

```
int produit (int a, int b) {
    int p ;
    p=a*b;
    return p ;
}
```

📁 Une fonction qui imprime les N éléments d'un tableau est :

```
void imprime-tab (int *tab, int N) {
    int i ;
    for(i=0; i<N; i++){
        printf("%d \t ", tab[i]);
    }
    return ;
}
```

II. APPEL D'UNE FONCTION

L'appel d'une fonction se fait par l'expression

nom-fonction(paramètre-1,...,paramètre -n)

NB : L'ordre et le type des paramètres effectifs doivent concorder avec les ceux donnés dans l'en-tête de la fonction.

III. DÉCLARATION D'UNE FONCTION

La déclaration d'une fonction en langage C peut être faite soit avant, soit après la fonction principale **main()**. Toutefois, il est indispensable que le compilateur connaisse la fonction au moment où elle celle-ci est appelée. Si une fonction est définie après son premier appel (en particulier si sa définition est placée après la fonction main), elle doit être impérativement déclarée au préalable. Une fonction secondaire est déclarée par son prototype, qui donne le type de la fonction et celui de ses paramètres, sous la forme :

type nom-fonction(type-1,...,type-n);

Exemple :

<code>int produit (int , int) ;</code>	}	Déclaration de la fonction
<code>int produit (int a , int b){</code>	}	Définition de la fonction
<code> return a*b;</code>		
<code>}</code>		
 <code>int main() {</code>	}	Appel de la fonction
<code>int c=2,d=5 ;</code>		
 <code>printf("%d\n",produit(c,d));</code>		
<code>}</code>		

IV. NOTION DE VARIABLE LOCALE ET GLOBALE

Définitions :

Une **variable globale** : est une variable déclarée en dehors de toute fonction. Elle est connue d compilateur dans toute la portion de code qui suit sa déclaration.

Une **variable locale** : est une variable déclarée en l'intérieur d'une fonction (ou d'un bloc d'instructions) du programme.

NB : Les variables locales n'ont en particulier aucun lien avec des variables globales de même nom.



EXERCICES

Exercice 1

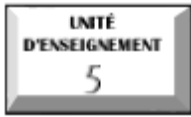
On veut écrire un programme C qui calcule la moyenne de 50 nombres saisis par l'utilisateur.

- a. Écrire une fonction nommée **Lecture** qui permet de lire les 50 nombres puis les stockent dans un tableau de taille 50.
- b. Écrire une autre fonction nommée **somme** qui permet de sommer les 50 nombres stockés dans le tableau.
- c. Écrire une autre fonction nommée **moyenne** qui calcule la moyenne de ces 50 nombres.
- d. Écrire la fonction principale faisant appel à ces fonctions pour résoudre le problème posé.

Exercice 2

1. Définir fonction
2. Quel mot clef utilisé en C et qu'on place devant le nom de la fonction pour signifier qu'elle ne retourne pas une valeur ?
3. Donner la différence entre une variable globale et une variable locale.
4. Écrire une fonction en C qui permet de calculer x^n où x et n sont les nombres entiers.
5. Écrire une fonction en C qui permet de déterminer le type d'une personne selon son âge.

NB : mineur (si âge ≤ 14), adulte (si âge > 14)



Exécution d'un programme C

Objectif pédagogique :

- ☞ Tester un programme dans un compilateur

Situation problème :

Vous amie Claudine a écrit un programme qui calcule la somme des 10 nombres en utilisant le langage C. Elle souhaite donc tester ce programme pour savoir s'il réalise vraiment la tâche demandée. Rencontrant des difficultés, elle fait appel à vous dans le but de l'aider.

Consignes :

1. De quoi a-t-elle besoin pour tester son programme ?
2. Donner le rôle d'un compilateur.
3. Décrire les étapes qu'elle doit suivre pour résoudre son problème.

I. TESTER UN PROGRAMME C EN UTILISANT LE COMPILATEUR GCC

Le C est un langage compilé (par opposition aux langages interprétés). Cela signifie qu'un programme C est décrit par un fichier texte, appelé fichier source. Ce fichier n'étant évidemment pas exécutable par le microprocesseur, il faut le traduire en langage machine. Cette opération est appelée la **compilation** et est effectuée par un programme appelé compilateur. Elle se décompose en fait en 4 phases successives :

1. **Le traitement par le préprocesseur** : le fichier source est analysé par le préprocesseur qui effectue des transformations purement textuelles (remplacement de chaînes de caractères, inclusion d'autres fichiers source ...).
2. La **compilation** : la compilation proprement dite traduit le fichier généré par le préprocesseur en assembleur.
3. **L'assemblage** : cette opération transforme le code assembleur en un fichier binaire, c'est-à-dire en instructions directement compréhensibles par le processeur.
4. **L'édition de liens** : un programme est souvent séparé en plusieurs fichiers source, pour des raisons de clarté mais aussi parce qu'il fait généralement appel à des bibliothèques des fonctions standard déjà écrites. Une fois chaque code source assemble, il faut

donc lier entre eux les différents fichiers objets. L'édition de liens produit alors un fichier dit exécutable.

Pour compiler votre code source C en utilisant le compilateur **gcc**, on utilise la commande suivante : **gcc[options]fichier.c[-llibrairies]**

Par défaut, le fichier exécutable s'appelle **a.out**. Le nom de l'exécutable peut être modifié à l'aide de l'option **-o nom-de-fichier**

fichier.c est le nom de votre programme C que vous avez tapé dans votre éditeur texte. Un programme C est toujours enregistré sous l'extension **.c**

Les éventuelles librairies sont déclarées par la chaîne **-llibrairie**. Dans ce cas le système recherche le fichier **liblibrairie.a** dans le répertoire contenant les librairies pré-compilées (généralement /usr/lib/). Par exemple, pour lier le programme avec la librairie mathématique, on spécifie **-lm**. Le fichier objet correspondant est **libm.a**

Lorsque les librairies pré-compilées ne se trouvent pas dans le répertoire usuel, on spécifie leur chemin d'accès par l'option **-L**

Exemple : la commande **gcc -o programmeI programmeI.c** compile le code source du programme nommé **programmeI.c** pour produire un fichier exécutable nommé **programmeI**

Après compilation de votre programme C, il faut donc ouvrir le fichier exécutable pour tester votre programme.

II. TESTER UN PROGRAMME C EN UTILISANT UN IDE : CAS DE CODE::BLOCKS

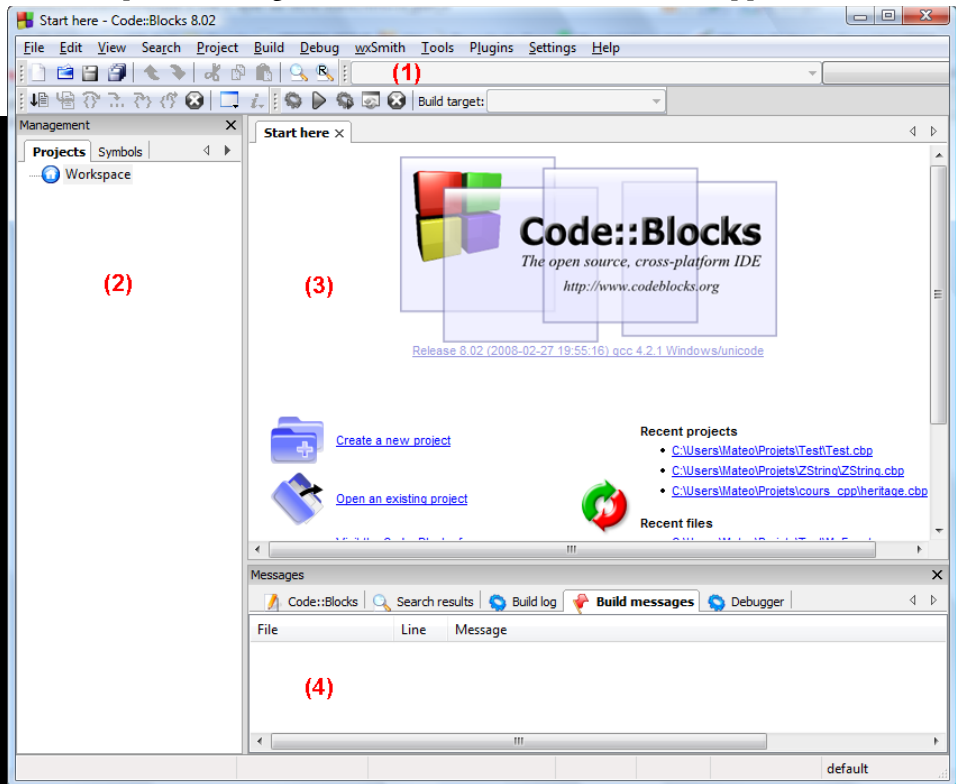
L'exécution d'un programme écrit en langage C se fait traditionnellement en utilisant un compilateur (gcc). Les fichiers sources sont écrits en utilisant un éditeur de texte et les programmes seront compilés depuis un terminal ou une invite de commande.

Pour faciliter la compilation des programmes C, l'on a mis sur pied des outils adaptés appelés **environnements de développement intégré** (IDE : Integrated development environment).

Un environnement de développement intégré permet de prendre en charge les différentes opérations de compilation : il regroupe donc l'édition de texte (créer le code source) et les outils de compilation (activation du compilateur gcc par exemple) directement depuis le même logiciel. Il évite ainsi le lancement d'un terminal ou d'une invite de commande pour lancer les opérations de compilation.

En outre, l'environnement de développement intégré facilite la correction du programme (debug) en lançant l'outil de debug (gdb) auquel il ajoute la convivialité de l'environnement graphique (visualisation de l'évolution des variables en temps réel, par exemple).

Après démarrage de code::Blocks, l'interface suivante apparaît :



Ses différentes parties sont :

1. **La barre d'outils** : elle comprend de nombreux boutons.
2. **La liste des fichiers du projet** : c'est à gauche que s'affiche tous les fichiers sources de votre programme
3. **La zone principale** : c'est là que vous pourriez écrire votre code en langage C
4. **La zone de notification** : aussi appelée « la zone morte », c'est ici que vous pourriez voir les erreurs de compilation.

Sur la barre d'outils, on y trouve les boutons suivants :

Boutons	 Compiler	 Exécuter	 Compiler & exécuter	 Tout recompiler
Signification	Tous les fichiers source de votre programme sont envoyés au compilateur qui va se charger de créer un fichier exécutable	Cette icône lance juste le dernier exécutable que vous avez compilé, cela vous permettra de tester votre programme	Permet de lancer la compilation ensuite le fichier exécutable produit par le compilateur (raccourcis en utilisant la touche F9 du clavier)	Permet de recompiler tous les fichiers de votre code source

Exemple :

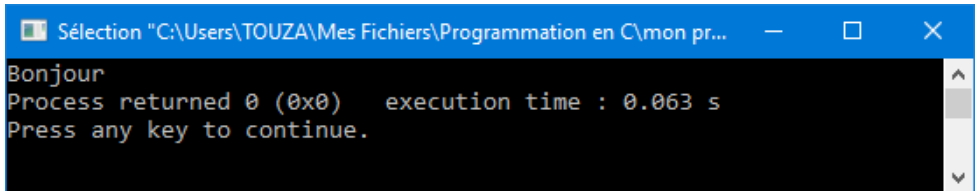
Un programme C qui affiche bonjour est :

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  int main( )
4  {
5      printf( "Bonjour" ) ;
6      return 0;
7  }
8

```

Après compilation et exécution en cliquant sur le bouton  , on obtient :



```

Sélection "C:\Users\TOUZA\Mes Fichiers\Programmation en C\mon pr...
Bonjour
Process returned 0 (0x0)   execution time : 0.063 s
Press any key to continue.

```



EXERCICES

Exercice 1

On souhaite écrire et exécuter un programme C qui calcule la somme de trois nombres saisis par l'utilisateur.

1. Écrire un programme C qui réalise la tâche demandée.
2. Démarrer l'IDE installer sur votre machine puis reproduire votre code dans la zone de travail de ce logiciel.
3. Cliquer sur le bouton de compilation pour compiler votre programme.
4. Exécuter ce programme avec les valeurs 2, 3, 23, 4 et 10.

Exercice 2

Utiliser l'IDE code::Blocks pour écrire et exécuter chacun des programmes demandés ci-dessous :

1. Un programme C qui calcule la moyenne de 10 nombres fournis par l'utilisateur.
2. Un programme C qui calcule le PGDC de deux nombres entiers naturels.
3. Un programme C qui détermine le reste de la division de **a** par **b**. Où **a** et **b** sont fournis par l'utilisateur.
4. Un programme C qui affiche la table de multiplication d'un nombre saisi par l'utilisateur.



Le groupe WhatsApp "**Les grandsprofs Informatique**" a été créé le 12/05/2017 afin de réunir les enseignants d'informatique pour un partage d'expériences, de ressources pédagogiques et didactiques. Permettre également les discussions autour des thèmes entre collègues et les réponses aux préoccupations des uns et des autres!



Partage de ressources

Espace de partage de nos différentes épreuves et autres documents pédagogiques.



Discussions

Discussions autour des thèmes et réponses aux préoccupations des collègues.



Détente

Espace conviviale de causerie et détente entre collègues de divers horizons.

Les grandsprofs  Informatique



Contact des administrateurs

(+237) 676 51 94 64
(+237) 699 38 62 59

(+237) 690 70 90 30
(+237) 699 32 26 76